

当传输一个数据报通过某特定网络时，这些参数被用于引导实际服务参数的选择。几种网络提供了服务优先权，该优先权以某种方式把高优先权流量看得比其他流量更为重要。”在 40 多年前，向不同等级的流量提供不同等级的服务的设想就是清晰的！然而，我们又花费了等长的时间去实现这种愿景。

1. 激励思考的场景

下面用几种激励思考的场景来开始我们的提供多种类型服务的网络机制的讨论。

图 7-14 表示了一种简单的网络场景，两个应用分组流产生于位于一个局域网的主机 H1 和 H2，它们的目的地是另一个局域网的主机 H3 和 H4。在这两个局域网上的两台路由器通过一条 1.5Mbps 的链路连接起来。我们假设局域网的速度远远高于 1.5Mbps，并且关注路由器 R1 的输出队列；注意到如果 H1 和 H2 的总计发送速率超过了 1.5Mbps，分组时延和丢包将会出现。

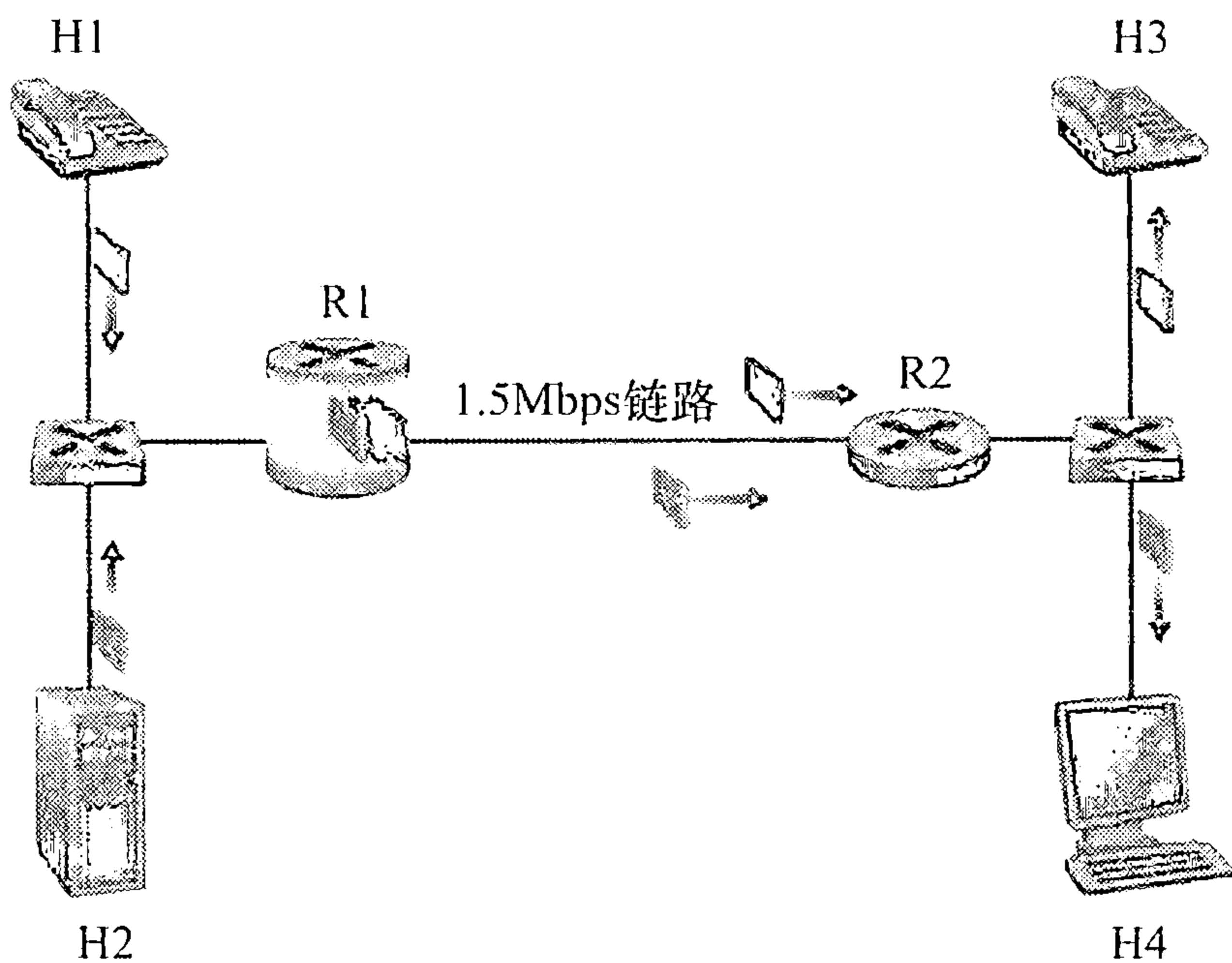


图 7-14 音频应用与 FTP 应用的竞争

我们进一步假设 1Mbps 的音频应用（例如一个 CD 质量的音频呼叫）共享 R1 和 R2 之间 1.5Mbps 的链路，同时从 H2 到 H4 有一个 HTTP Web 浏览应用正在下载一个 Web 网页。

在尽力而为服务的因特网中，该音频和 HTTP 分组在 R1 的输出队列中混合，并且（通常）以先进先出（FIFO）的次序传输。在这种情况下，来自 Web 服务器的突发分组可能潜在地填满这个队列，引起 IP 音频分组过度延迟或者由于 R1 的缓存溢出而丢失。我们应该如何解决这个潜在的问题呢？假定该 HTTP Web 浏览应用没有时间限制，我们的直觉也许是在 R1 为音频分组分配严格的优先级。在一个严格的优先级调度规则下，在 R1 输出缓存的音频分组总是在 R1 输出缓存中的任何 HTTP 分组之前传输。对音频流量而言，从 R1 到 R2 的链路看起来像一条 1.5Mbps 专用链路，而对于 HTTP 流量仅当没有音频流量排队时，才使用 R1 到 R2 的链路。为了让 R1 在它的队列中区分音频和 FTP 分组，每个分组必须被标记为属于这两类流量中的哪一类。这是 IPv4 中服务类型（ToS）字段的最初目的。显而易见，这则是我们对需要提供多种类型流量机制的第 1 个见解。

见解 1：标记分组（packet marking）使得路由器区分属于不同类型流量的分组。

注意到尽管我们的例子考虑到竞争性的多媒体和弹性流，但是相同的见解应用于实现了铂金卡、金卡和银卡服务类型的情况，即仍需要标记分组机制以指示分组属于哪个类型的服务。

现在假设路由器被配置为给标记为属于 1Mbps 音频应用的分组赋予高优先级。因为输出链路速度是 1.5Mbps，即使 HTTP 分组得到较低的优先级，它们仍然可以收到平均 0.5Mbps 的传输服务。但是如果音频应用开始以 1.5Mbps 或者更高的速率（或者恶意的，或者由于应用的差错）发送分组，那会出现什么样的情况呢？在这种情况下，HTTP 分组将挨饿，也就是在 R1 到 R2 的链路上得不到任何服务。如果多个应用（例如，多个音频呼叫）都具有同等的服务类型，共享一段链路带宽，那么也会出现类似问题，即它们也可

能共同饿死该 HTTP 会话。理想情况下，一种服务要与各类流量有隔离度，以保护一种流量类型免受其他流量类型干扰。这种保护能够在网络中的不同地方实现，在每台路由器中，在进入网络的首个入口，或在网络边界域间。这则是我们的第2个见解。

见解2：希望在流量类型之间提供**流量隔离**（traffic isolation）的度，以便一类流量不会受到另一类异常流量的负面影响。

我们将考察在流量类型之间提供这种隔离的特定机制。这里我们注意到，有两大类方法可以使用。首先，可以执行如图7-15所示的**流量监管**（traffic policing）方法。如果流量类型或流必须满足一定的准则（例如，音频流不超过1Mbps的峰值速率），那么可以设置一个监管机制来确保这些准则的确被遵守。如果被监管的应用行为异常，这个监管机制将采取某种行动（例如，丢弃或者延时那些违反这些准则的分组），以便实际进入网络的流量符合这些准则。我们很快将考察的漏桶机制也许是使用最广泛的监管机制。在图7-15中，分组分类和标记机制（见解1）以及监管机制（见解2）都一起在网络的边缘实现，或在端系统中实现，或在边界路由器中实现。

为流量类型之间提供隔离的一种互补的方法是，链路级的分组调度机制为每种类型明确地分配固定量的链路带宽。例如，在R1能够给音频类型分配1Mbps，能够给HTTP流分配0.5Mbps。在这种情况下，音频和HTTP流分别看到了容量为1.0Mbps和0.5Mbps的逻辑链路，如图7-16所示。通过严格执行链路级的带宽分配，一种类型仅能够使用已经分配的带宽量；特别是，它不能利用其他应用现在未使用的带宽。例如，如果音频流静默了（例如，如果谈话者停顿，不产生音频分组），HTTP流在R1到R2的链路上传输带宽仍然不能够超过0.5Mbps，即使音频流分配的1Mbps带宽在那个瞬间没有使用。由于带宽是一种“使用它或丢失它（use-it-or-lose-it）”的资源，没有理由妨碍HTTP流量使用没有由音频流使用的带宽。我们将希望尽可能高效地使用带宽，当能够以别的方法使用它时决不浪费带宽。这引发我们的第3个见解。

见解3：当为流量类型或流之间提供隔离时，希望尽可能有效地使用资源（例如链路带宽和缓存）。

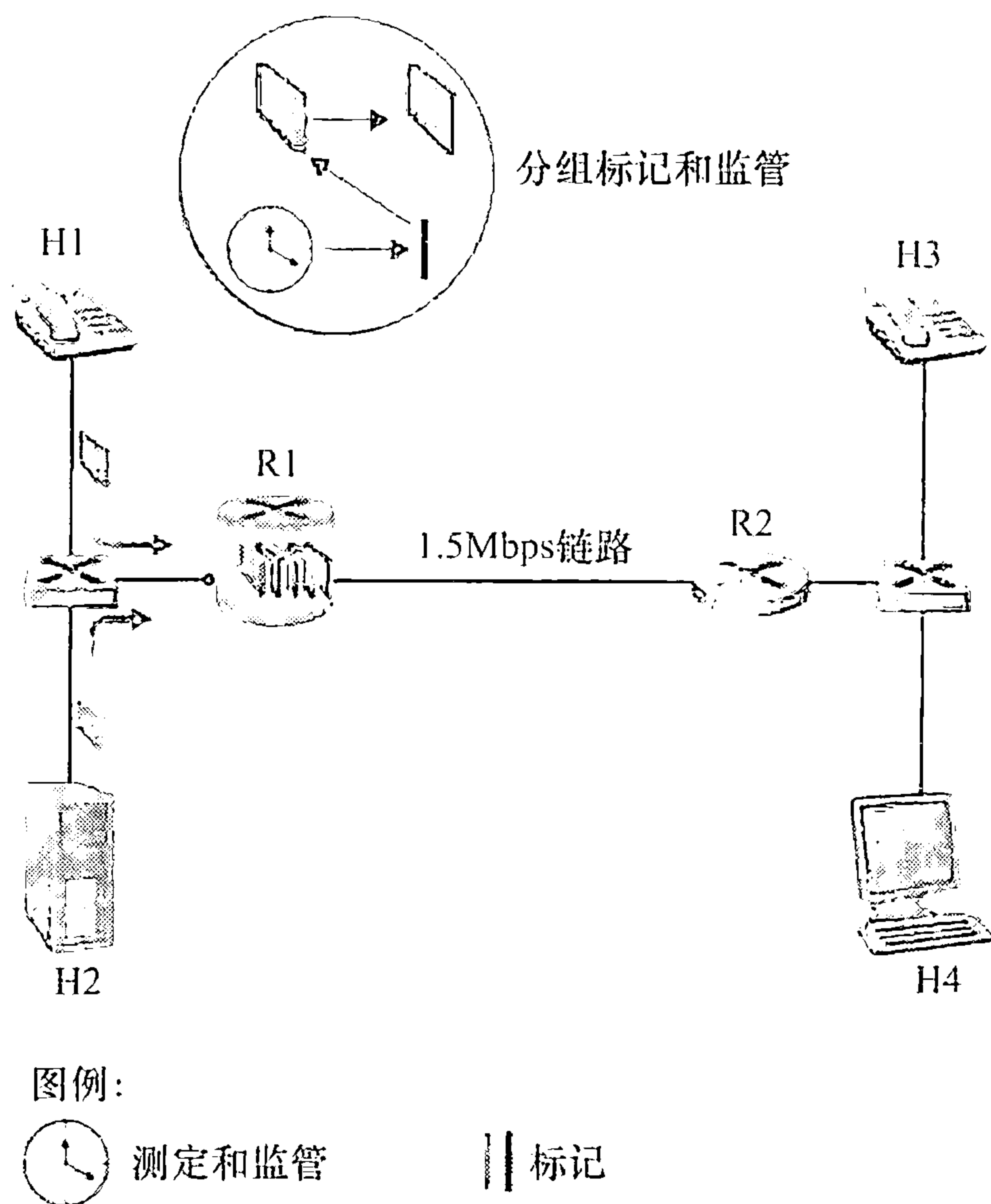


图 7-15 监管（与标记）音频与 HTTP 流量类型

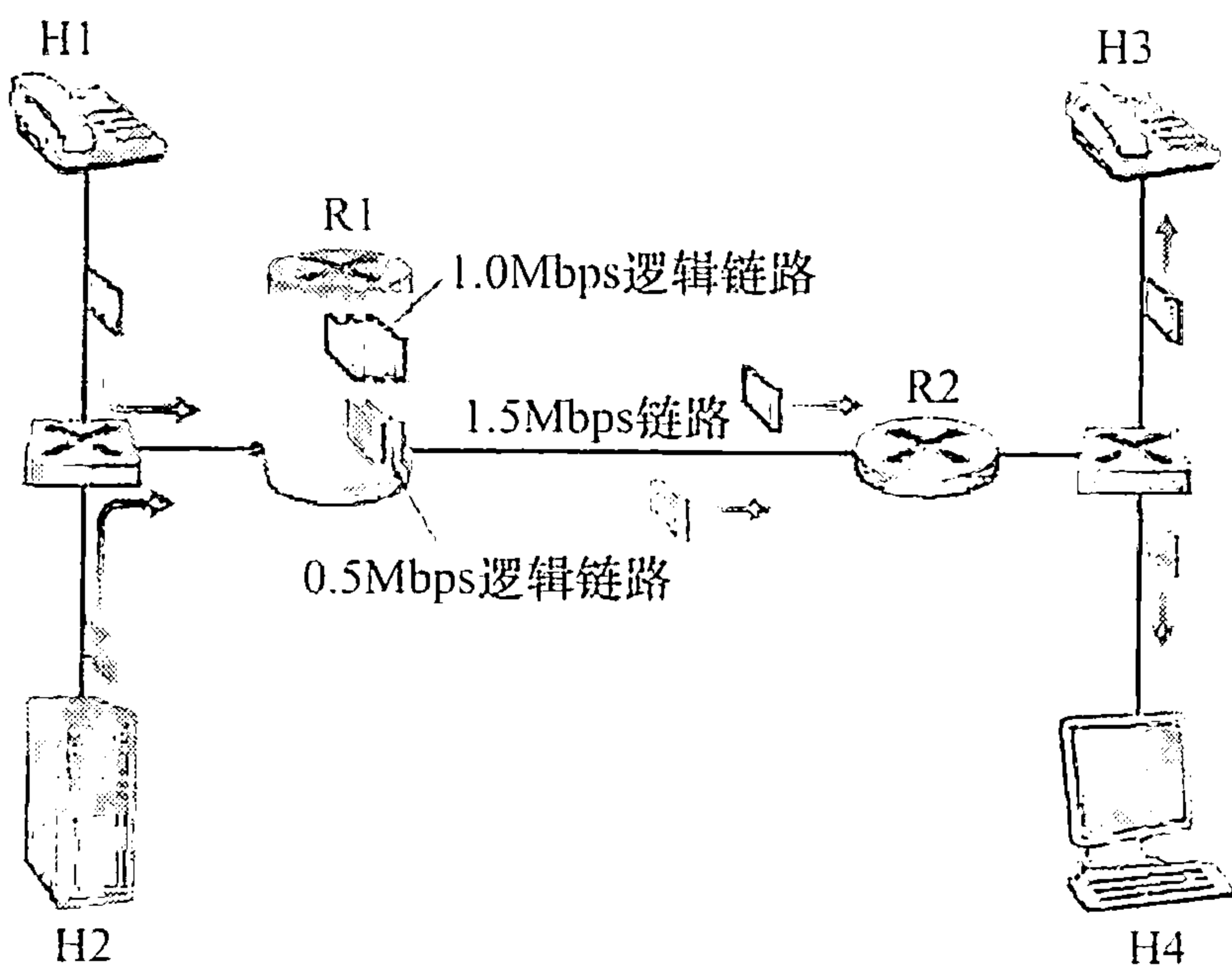


图 7-16 音频与 HTTP 流量类型的逻辑隔离

2. 调度机制

在 1.3 节和 4.3 节我们曾经讨论过，属于各种网络流的分组被复用在一起，并且在与链路关联的输出缓存排队等待传输。选择在链路上传输的排队分组的方式称为链路调度规则（link-scheduling discipline）。我们现在更详细地考虑几个最重要的链路调度规则。

(1) 先进先出

图 7-17 显示了对于先进先出（First-In-First-Out, FIFO）链路调度规则的排队模型的抽象。如果链路当前正忙于传输另一个分组，到达链路输出队列的分组要排队等待传输。如果没有足够的缓存空间来容纳到达的分组，队列的分组丢弃策略（packet-discarding policy）则确定该分组是否将被丢弃（丢失）或者其他分组将从队列中去除以为到达的分组腾出空间。在下面的讨论中，我们将忽视分组丢弃。当一个分组通过输出链路完全传输（也就是接收服务）时，它从队列中去除。

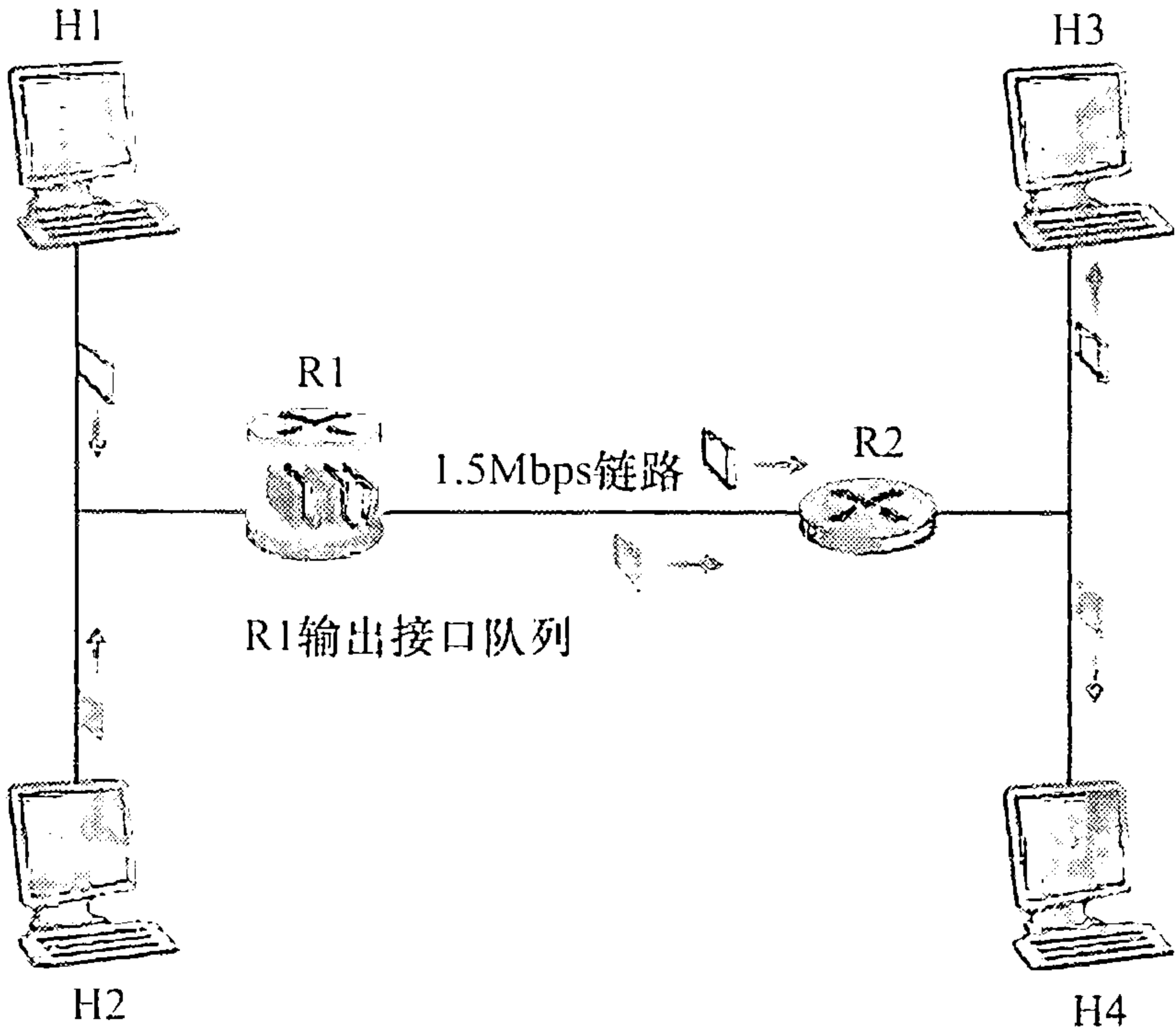


图 7-17 FIFO 排队抽象

FIFO（也称为先来先服务，即 FCFS）调度规则按照分组到达输出链路队列的相同次序来选择分组在链路上传输。我们都很熟悉公共汽车站（尤其在英格兰，那里的排队看起来已经很完善了）或者其他服务中心的 FIFO 排队，在那里到达的顾客加入单一等待队列的最后，保持次序，然后当他们到达队伍的前面时就接受服务。

图 7-18 显示了运行中的 FIFO 队列。分组的到达由上部时间线上带编号的箭头来指示，该编号指示了分组到达的次序。各个分组的离开表示在下部时间线的下面。分组在服务中（被传输）花费的时间是通过这两个时间线之间的阴影矩形来指示的。由于 FIFO 规则，分组按照到达的相同次序离开。注意在分组 4 离开之后，在分组 5 到达之前链路保持空闲（因为分组 1~4 已经被传输并从队列中去除）。

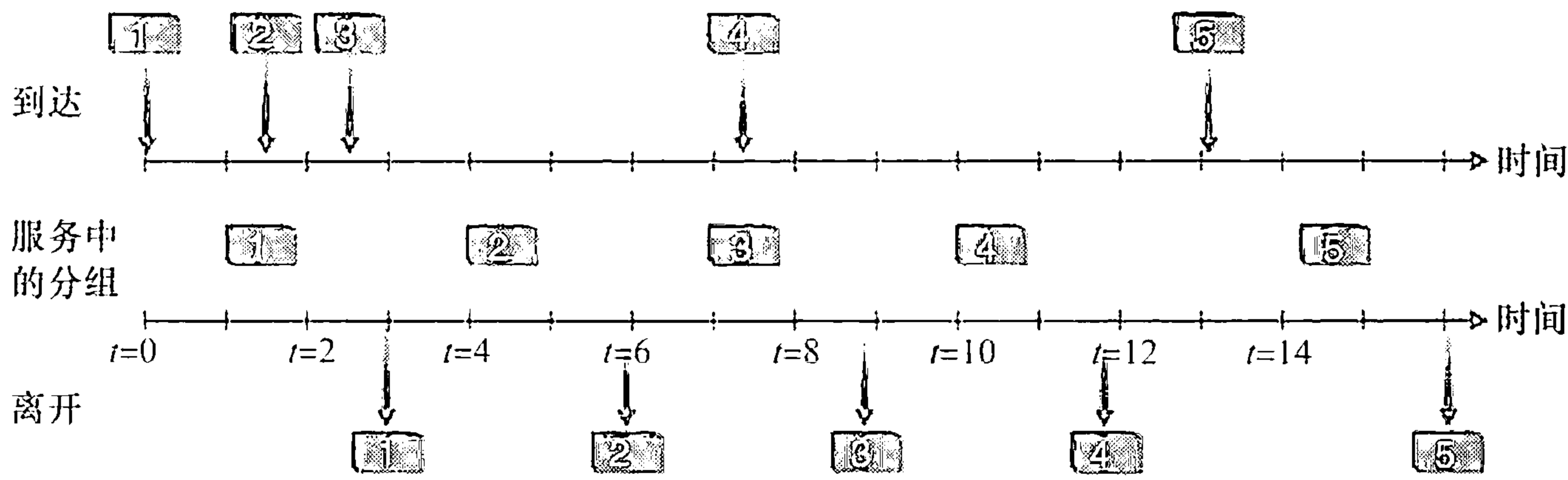


图 7-18 操作中的 FIFO 队列

(2) 优先级排队

使用优先级排队（priority queuing）规则，到达输出链路的分组被分类放入输出队列中的优先级类，如图 7-19 所示。如前一节所讨论，一个分组的优先级类可能取决于其分组首部携带的一个明显的标记（例如，在一个 IPv4 分组中 ToS 比特的值）、它的源或者目

的地址、它的目的端口号或者其他标准。每个优先级类通常都有自己的队列。当选择一个分组传输时，优先级排队规则将从队列为非空（也就是有分组等待传输）的最高优先级类中传输一个分组。在同一优先级类的分组之间的选择通常以 FIFO 方式完成。

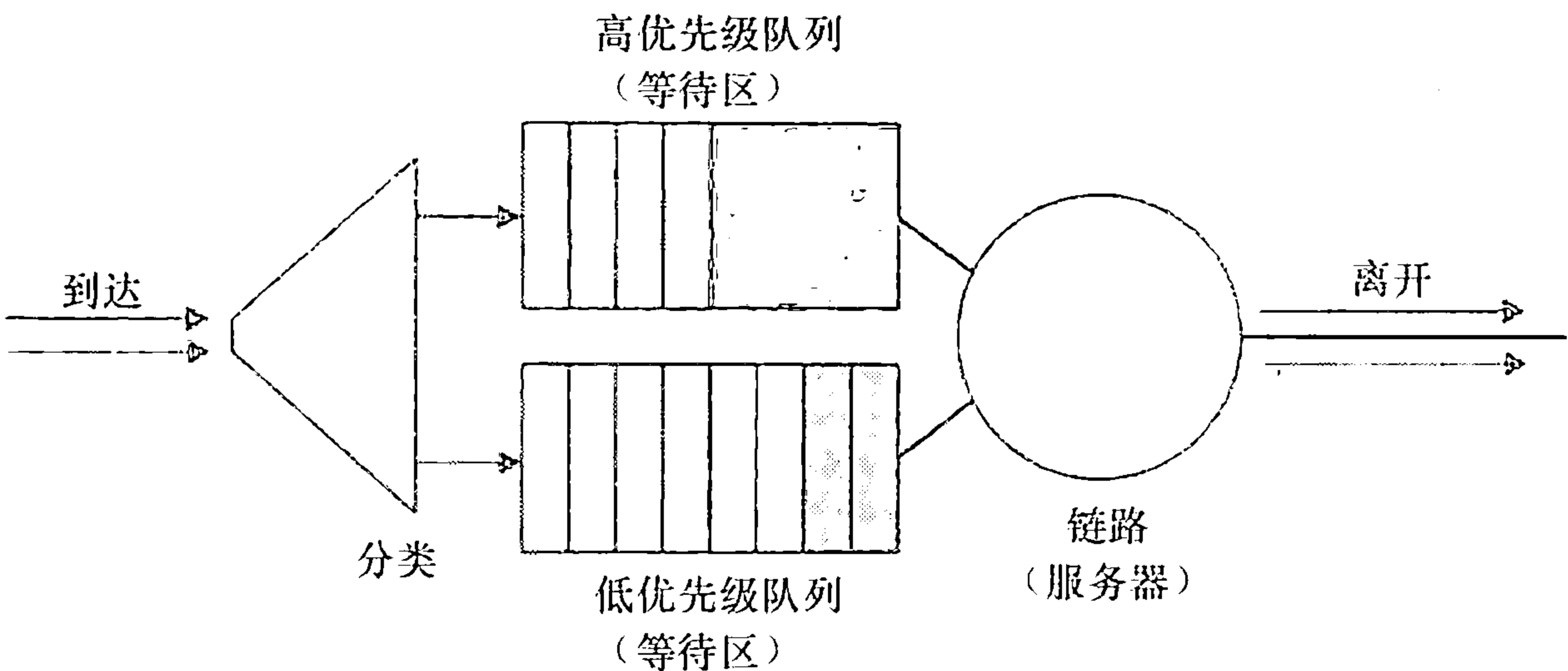


图 7-19 优先级排队模型

图 7-20 描述了有两个优先级类的一个优先级队列的操作。分组 1、3 和 4 属于高优先级类，分组 2 和 5 属于低优先级类。分组 1 到达并发现链路是空闲的，就开始传输。在分组 1 的传输过程中，分组 2 和 3 到达，并分别在低优先级和高优先级队列中排队。在传输完分组 1 后，分组 3（一个高优先级的分组）被选择在分组 2（尽管它到达得较早，但它是一个低优先级分组）之前传输。在分组 3 的传输结束后，分组 2 开始传输。分组 4（一个高优先级分组）在分组 2（一个低优先级分组）的传输过程中到达。在所谓的非抢占式优先级排队规则下，一旦分组开始传输，传输就不能打断。在这种情况下，分组 4 排队等待传输，并在分组 2 传输完成之后开始传输。

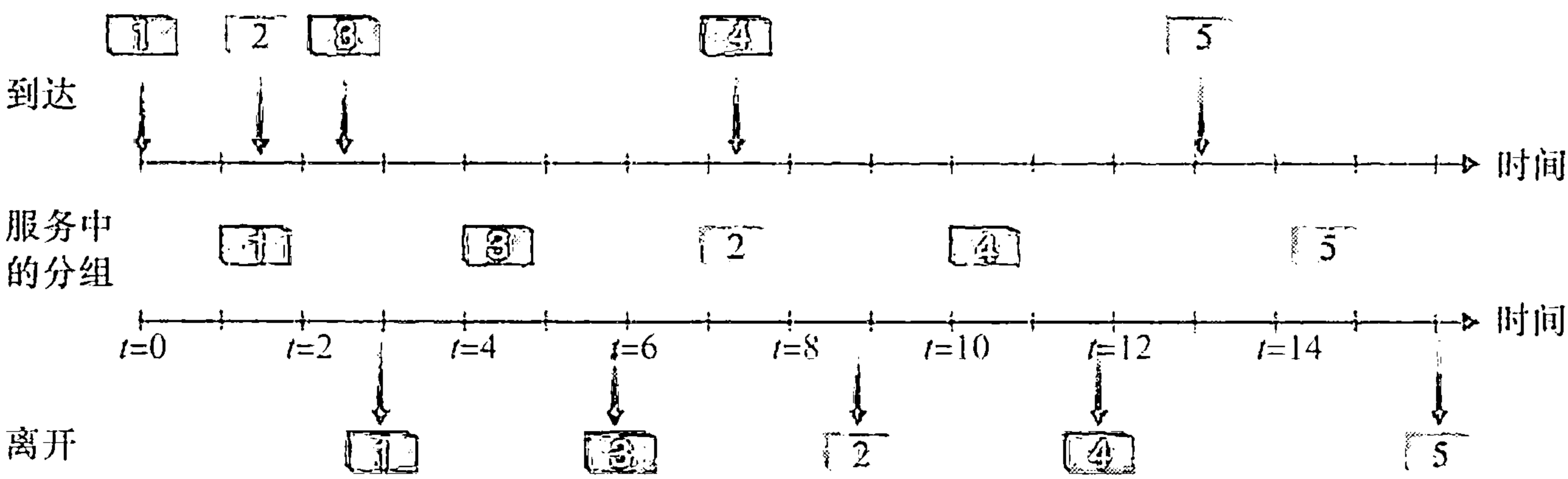


图 7-20 优先级队列的操作

(3) 循环和加权公平排队

使用循环排队规则（round robin queuing discipline），分组像使用优先级排队一样被分类。然而，在类之间不存在严格的服务优先级，循环调度器在这些类之间轮流提供服务。在最简单形式的循环调度中，类 1 的分组被传输，接着是类 2 的分组，接着又是类 1 的分组，再接着又是类 2 的分组等等。一个所谓的保持工作的排队规则，即在有（任何类的）分组排队等待传输时不允许链路保持空闲。当寻找给定类的分组但是没有找到时，保持工作的循环规则（work-conserving round robin discipline）将立即检查循环序列中的下一个类。

图 7-21 描述了一个两类循环队列的操作。在这个例子中，分组 1、2 和 4 属于第一类，分组 3 和 5 属于第二类。分组 1 一到达输出队列就立即开始传输。分组 2 和 3 在分组 1 的

传输过程中到达，因此排队等待传输。在分组 1 传输后，链路调度器寻找类 2 的分组，因此传输分组 3。在分组 3 传输完成后，调度器寻找类 1 的分组，因此传输分组 2。在分组 2 传输完成后，分组 4 是唯一的排队的分组，因此在分组 2 后立刻传输分组 4。

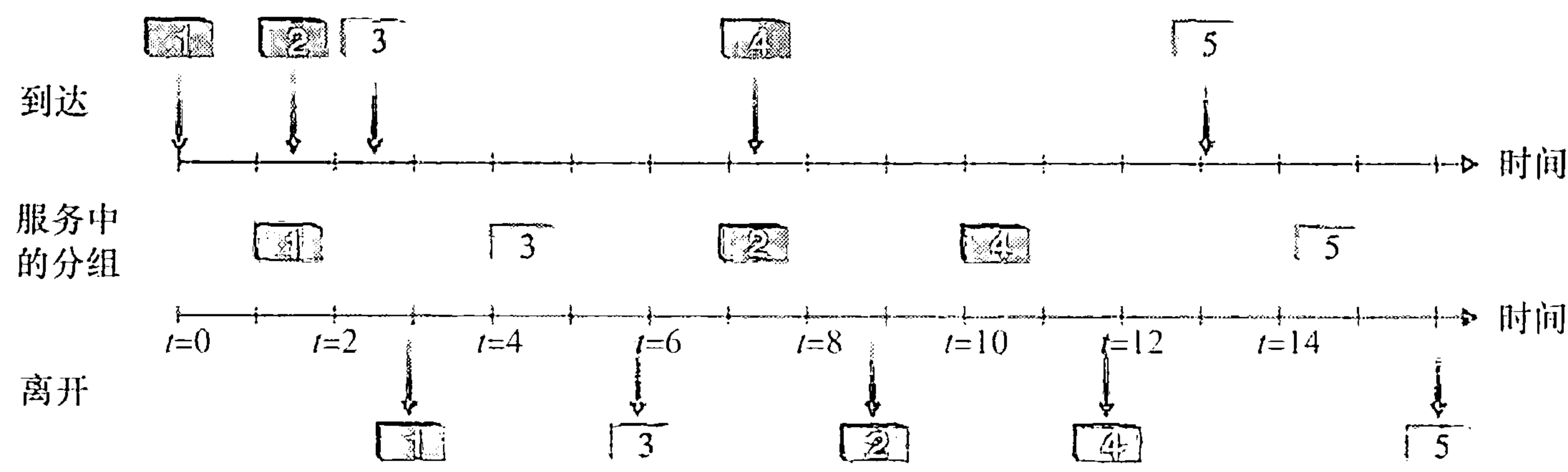


图 7-21 两类循环队列的操作

在 QoS 体系结构中已经得到大量使用的循环排队的一个通用抽象就是所谓的加权公平排队 (Weighted Fair Queuing, WFQ) 规则 [Demers 1990; Parekh 1993]。图 7-22 对 WFQ 进行了描述。到达的分组被分类并在合适的每个类的等待区域排队。与使用循环调度一样，WFQ 调度器也以循环的方式为各个类提供服务，即首先服务第 1 类，然后服务第 2 类，接着再服务第 3 类，然后（假设有 3 个类别）重复这种服务模式。WFQ 也是一个保持工作的排队规则，因此在发现一个空的类队列时，它立即移向服务序列中的下一个类。

WFQ 和循环排队不同之处在于，每个类在任何时间间隔内可能收到不同数量的服务。具体而言，每个类 i 被分配一个权 w_i 。使用 WFQ 方式，在类 i 有分组要发送的任何时间间隔中，第 i 类将确保接收到的服务部分等于 $w_i / (\sum w_j)$ ，式中分母中的和是计算所有有分组排队等待传输的类别得到的。在最坏的情况下，即使所有的类都有分组排队，第 i 类仍然保证分配到带宽的 $w_i / (\sum w_j)$ 部分。这样，对于一条传输速率为 R 的链路，第 i 类总能获得至少为 $R \cdot w_i / (\sum w_j)$ 的吞吐量。我们对 WFQ 的描述理想化了，因为我们没有考虑这样的事实：分组是离散的数据单元，并且不能打断一个分组的传输来开始传输另一个分组；[Demers 1990] 和 [Parekh 1993] 讨论了这个分组化问题。如我们将在下面几节所见，WFQ 在 QoS 体系结构中起了重要的作用。现在的路由器产品中也用到了 WFQ [Cisco QoS 2012]。

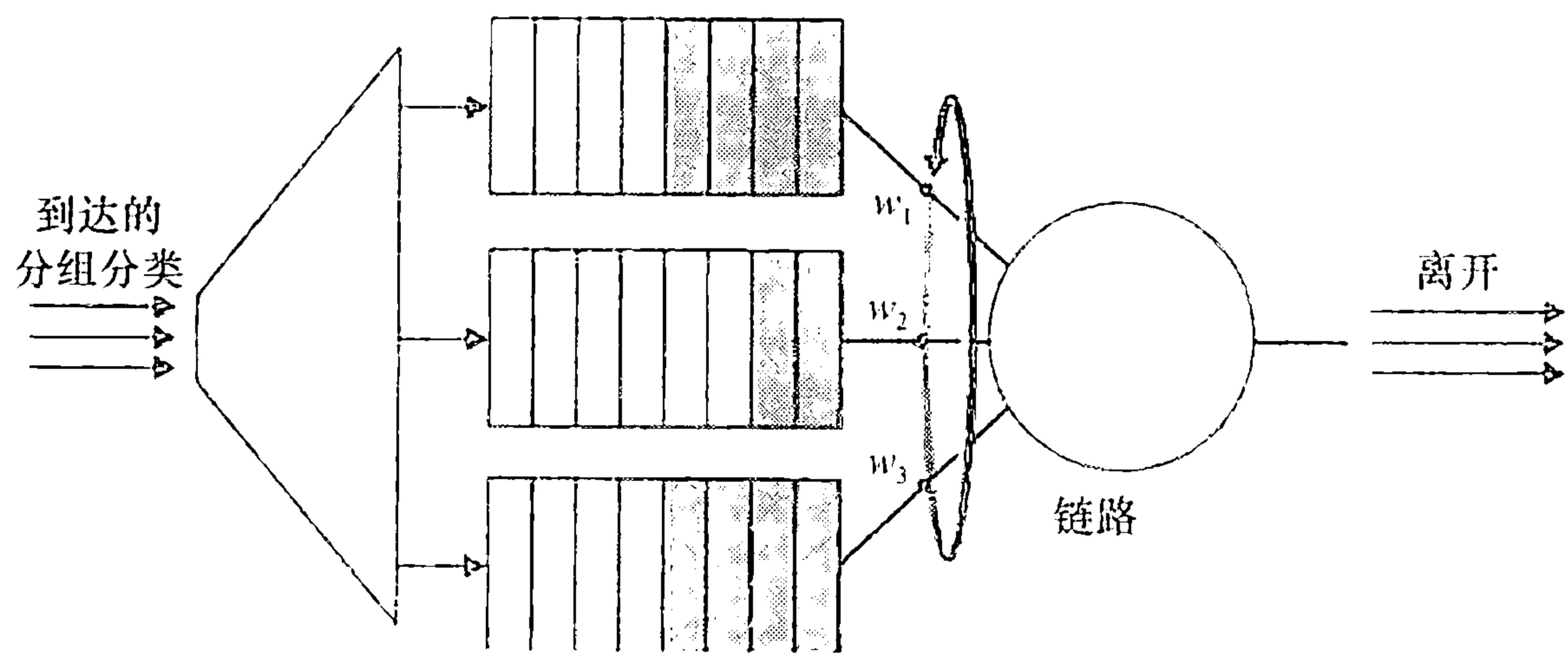


图 7-22 加权公平排队

3. 监管：漏桶

我们前面的见解之一是监管，速率的调节是一种重要的 QoS 机制，以该速率允许一类流量或流（在下面的讨论中，我们将假定监管的单位是流）向网络注入分组。但是应该对

一个流的分组速率的哪些方面进行监管呢？我们能够指出 3 个重要的监管准则，每个准则根据被监管分组流的时间范围而互不相同：

- 平均速率。网络可能希望限制一个流的分组能够发送到网络中的长期平均速率（每个时间间隔的分组数）。这里一个关键的问题是监管平均速率的时间间隔。一个平均速率被限制为每秒 100 个分组的流要比一个每分钟 6000 个分组的源受到的约束更严格，即使在一个足够长的时间间隔上它们有相同的平均速率。例如，后者的限制允许一个流在给定 1 秒钟长的时间间隔内发送 1000 个分组，而前者的限制不允许这种发送行为。
- 峰值速率。平均速率约束限制了一个相对长的时间内能够发送到网络中的流量，而峰值约束限制了一个较短时间内能够发送的最大分组数。使用我们上面的例子，网络能以每分钟 6000 个分组的平均速率来监管一个流，但是限制该流的峰值速率为每秒 1500 个分组。
- 突发长度。网络也许还希望限制极短的时间间隔内能够发送到网络中的最大分组数（分组的“突发（burst）”）。在这个限制中，因为时间间隔长度趋近于 0，该突发长度限制了能够瞬间发送到网络中的分组数量。尽管瞬间发送多个分组到网络中在物理上是不可能的（毕竟每条链路都有一个无法超越的物理传输速率！），但对最大突发长度的抽象也是有用的。

漏桶机制是一个能够用来表征这些监管限制的抽象。如图 7-23 所示，漏桶由一个能够容纳 b 个令牌的桶组成。令牌加进该桶的过程如下。可能潜在地加入桶中的新令牌总是以每秒 r 个令牌的速率产生。（为了简单化起见，我们这里假设时间单元是秒。）当产生一个令牌时，如果桶填充得少于 b 个令牌，新产生的令牌加入到该桶中；否则忽略该新产生的令牌，令牌桶保持具有 b 个令牌的满状态。

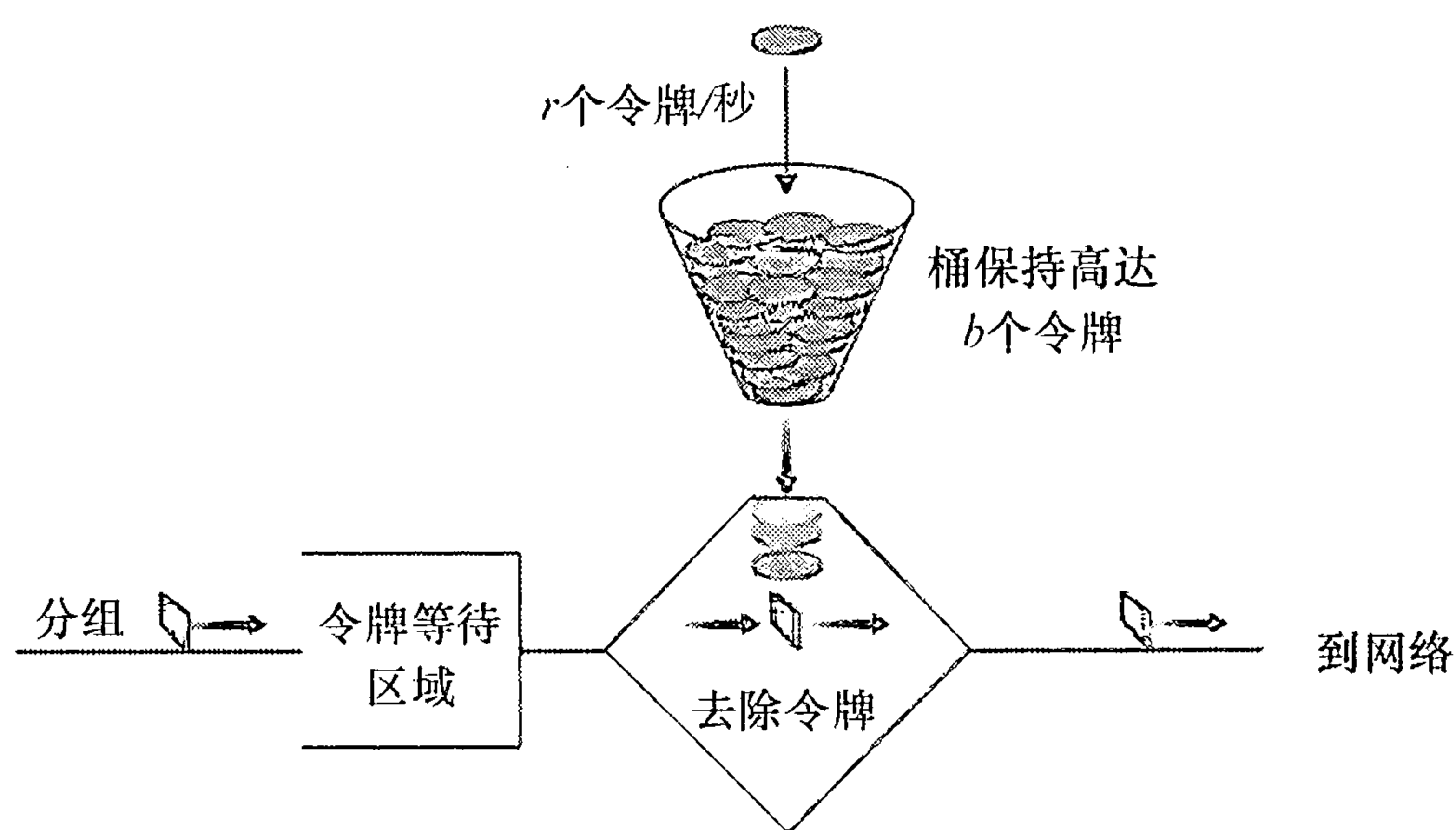


图 7-23 漏桶监管器

现在我们考虑如何用漏桶来监管分组流。假设在一个分组向网络传输之前，必须首先从令牌桶中去除一个令牌。如果令牌桶是空的，分组必须等待一个令牌。（另一种方法是丢弃该分组，尽管我们这里不讨论这种选择。）现在我们考虑这种行为是如何监管一个流量流的。因为在桶中最多能有 b 个令牌，所以漏桶监管的流的最大突发长度是 b 个分组。此外，因为令牌产生速率是 r ，所以在任何长度为 t 的时间间隔内能够进入网络中的最大分组数目为 $rt + b$ 。因此，令牌产生速率 r 用于限制分组能够进入网络的长期平均速率。除了监管长期平均速率之外，使用漏桶（具体来说，串联的两个漏桶）来监管流的峰值速率

也是有可能的；见本章末的习题。

漏桶 + 加权公平排队 = 队列中可证明的最大时延

通过显示如何将调度和监管两种机制结合起来，以提供通过路由器队列的时延界限，我们结束了相关讨论。我们考虑一台使用了 WFQ 调度的路由器，它的输出链路多路复用了 n 条流，其中每条流被一个参数为 b_i 和 r_i 的漏桶监管， $i = 1, \dots, n$ 。这里我们使用的术语“流”大致是指不能被调度器相互区别的分组集合。实践中，一条流可能是由单个端到端连接上的流量或者许多这种连接的集合流量组成的，参见图 7-24。

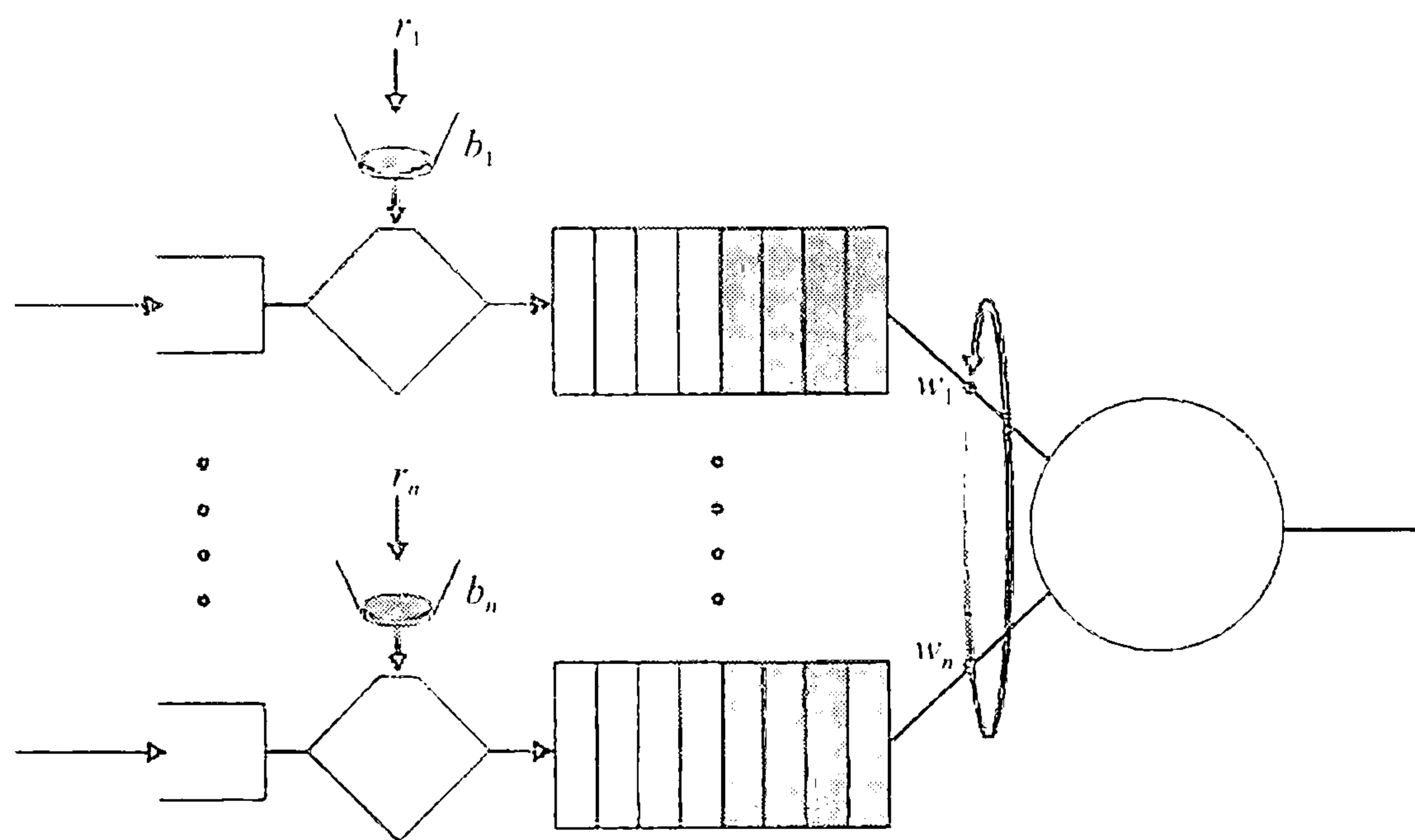


图 7-24 采用 WFQ 调度机制的 n 路复用的漏桶流

我们前面讨论 WFQ 时讲过，每个流 i 保证收到至少等于 $R \cdot w_i / (\sum w_j)$ 的共享链路带宽，其中 R 是以分组/秒为单位的链路传输速率。当以 WFQ 方式等待服务时（也就是通过漏桶传递之后），则分组经受的最大时延是什么？我们来关注流 1。假设流 1 的令牌桶最初是满的。然后 b_1 个分组的突发到达流 1 的漏桶监管器。这些分组去除了漏桶中所有的令牌（没有等待），然后加入了流 1 的 WFQ 等待区域。因为这 b_1 个分组以至少 $R \cdot w_1 / (\sum w_j)$ 分组/秒的速度得到服务，直到这些分组的最后一个传输完成，所以将有最大时延 $d_{\max}$ ，其中

$$d_{\max} = \frac{b_1}{R \cdot \frac{w_1}{\sum w_j}}$$

这个公式的基本原理是，如果在队列中有 b_1 个分组并且分组以至少每秒 $R \cdot w_1 / (\sum w_j)$ 个分组的速率从队列中接受服务（被去除），那么直到最后一个分组的最后一个比特被传送，时间量不会超过 $b_1 / (R \cdot w_1 / (\sum w_j))$ 。一道课后习题请你证明，只要 $r_1 < R \cdot w_1 / (\sum w_j)$ ，则 $d_{\max}$ 确实是流 1 中任何分组在 WFQ 队列中要经受的最大时延。

7.5.3 区分服务

在学习了提供多种类型服务的动机、见解和特定的机制后，我们将注意力集中于以举例方式学习提供多种类型服务的方法，即因特网区分服务（Diffserv）体系结构 [RFC 2475; Kilkki 1999]。Diffserv 提供服务区分，也就是在因特网中以一种可扩展性方式用不同的方法处理不同类型流量的能力。可扩展性的需求来源于这样的事实：在因特网的一个主干路由器上同时存在几十万个源到目的地的并行流。我们很快将看到，仅仅通过在网络

核心放置简单的功能，而在网络的“边缘”实现更复杂的控制操作，就可以满足该需要。

我们从图 7-25 中显示的简单网络开始讨论。这里我们将描述 Diffserv 的一种可能用法。如 RFC 2475 中所描述，其他的变化是可能的。区分服务体系结构由两个功能元素的集合所组成：

- 边界功能：分组分类和流量调节。在网络的人边缘（也就是，或者在产生流量的 Diffserv 使能的主机，或者在流量经过的第一个 Diffserv 使能的路由器），到达的分组被标记。更具体地说，在 IPv4 或者 IPv6 分组首部中的区分服务（DS）字段被设置为某个值 [RFC 3260]。该 DS 字段的定义意欲取代我们在第 4 章中讨论的 IPv4 服务类型字段和 IPv6 流量类型字段。例如，在图 7-25 中，从 H1 发送到 H3 的分组可能在 R1 被标记，而从 H2 发送到 H4 的分组可能在 R2 被标记。分组得到的标记标识了该分组所属的流量类型。然后不同类型的流量将在核心网络接受到不同的服务。
- 核心功能：转发。当一个 DS 标记的分组到达一个 Diffserv 使能的路由器时，根据与分组类型相关的所谓的每跳行为（Per-Hop Behavior, PHB），该分组被转发到它的下一跳。每跳行为影响在竞争的流量类型之间共享路由器缓存和链路带宽的方式。Diffserv 体系结构的一个关键原则是路由器的每跳行为只基于分组标记，即分组所属的流量类型。因此，如果图 7-25 中从 H1 发送到 H3 的分组和从 H2 发送到 H4 的分组收到同样的标记，网络路由器则将这些分组处理为一个聚合体，而不区别这些分组是源于 H1 还是源于 H2。例如，当这些分组向 R4 转发时，R3 不区分从 H1 和 H2 产生的分组。因此，Diffserv 体系结构消除了为各个“源到目的地”而对保留路由器状态的要求，这是使 Diffserv 具有可扩展性的一个重要考虑。

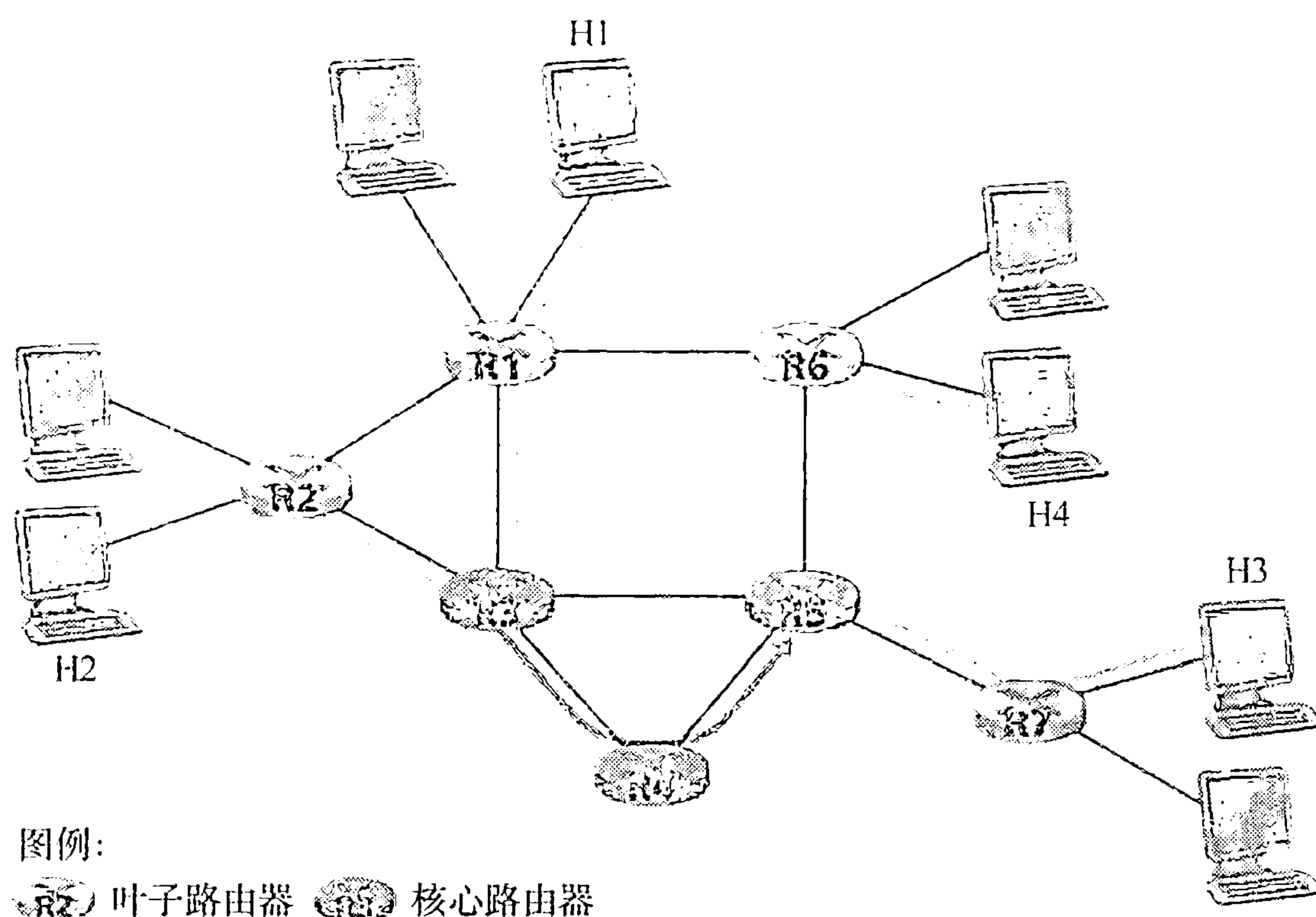


图 7-25 一个简单的 Diffserv 网络例子

这里有一个类比可能有助于理解相关概念。在很多大规模的社交活动中（例如，一场大型的公众招待会，一场大型的跳舞俱乐部活动或者迪斯科舞会，一场音乐会或者一场足球赛），参加活动的人收到某种类型的入场券。对于非常重要的人物有 VIP 入场券；对于 21 岁或者年龄更大的人有 21 岁以上的入场券（例如是否可以享受酒类饮料）；对于演唱人员有后台入场券；对于记者有新闻入场券；对于普通人有普通的入场券。这些入场券通

常在该活动的入口分发，也就是在活动的边缘进行分发。正是在边缘，进行着计算密集型操作，例如交入场费、检查适合的邀请类型以及对照检查邀请与证件。此外，对于允许进入一个活动的特定类型的人数可能有限制。如果有这种限制，人们可能在进入活动之前必须等待。一旦进入了活动，一个人的入场券使他在活动中的很多场所接受有区别的服务，如为 VIP 提供免费的饮料、较好的桌子、免费食物、单独的房间和殷勤的服务。与之相对照的是，一个普通人禁止进入某些区域，要为饮料支付费用，并且只受到基本服务。在这两种情况下，活动中得到的服务只依赖于入场券的类型。而且在同一类型的所有人得到相同的对待。

图 7-26 提供了在边缘路由器中分类和标记功能的逻辑视图。到达边缘路由器的分组首先被分类。分类器根据一个或多个分组首部字段的值（例如源地址、目的地址、源端口、目的端口和协议 ID）来选择分组，并引导该分组去做合适的标记功能。如前面所指出的，分组的标记携带在分组首部的 DS 字段中。

在某些情况下，端用户可能已经同意限制其分组发送速率以符合某个申报的流量配置文件（traffic profile）。该流量配置文件可能包含对峰值速率和分组流的突发度的限制，如我们前面在漏桶机制中所见。只要用户以符合协商的流量配置文件的方式向网络中发送分组，这些分组就会得到它们的优先级标记，并沿着到目的地的路径转发。另一方面，如果违反了该流量配置文件，那些超出流量配置文件的分组就可能被打上不同的标记，或被整形（例如，为了能够遵守最大速率限制而延时），或可能在网络边缘被丢弃。图 7-26 中所示的测定功能（metering function）的作用是比较进入的分组流和协商的流量配置文件，并确定某分组是否在协商的流量配置文件之内。有关是否立即重新标记、转发、延时或者丢弃一个分组的实际决定是由网络管理员决定的策略问题，而不是由 Diffserv 体系结构规定的。

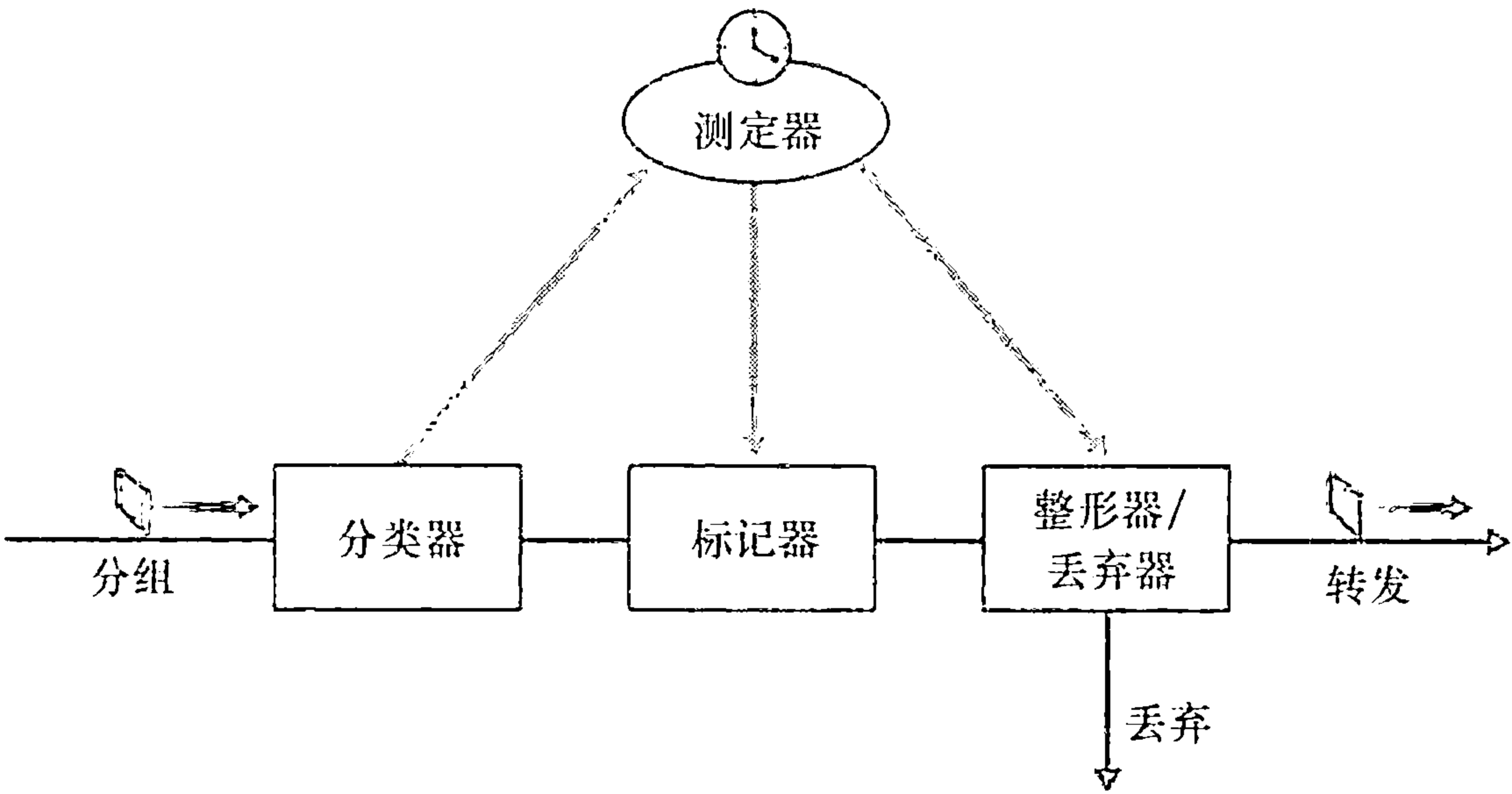


图 7-26 在端路由器上的分组分类与流量调节的逻辑视图

到现在为止，我们一直关注着 Diffserv 体系结构中的标记和监管功能。Diffserv 体系结构的第二个关键组件涉及由 Diffserv 使能路由器所执行的每跳行为（PHB）。该每跳行为有点神秘，但被仔细地定义为“Diffserv 结点的外部可观察的转发行为的描述，该结点应用了一个特定的 Diffserv 行为聚合”[RFC 2475]。稍微深入地钻研一下这个定义，我们可以看到它里面包含的几个重要的考虑：

- PHB 能够导致不同服务类型的流量接收到不同性能（即不同的外部可观察的转发行为）。

- 虽然 PHB 定义了在各种类型之间的性能（行为）差别，但它不强求为了获得这些行为的任何特定机制。只要外部可观察的性能准则得到满足，任何实现机制和任何缓存/带宽分配策略都可以使用。例如，一个 PHB 将不要求使用一个特定的分组排队规则（例如优先级队列、WFQ 队列或 FCFS 队列）以取得特定的行为。PHB 是最后的结果，资源分配和实现机制是达到 PHB 的手段。
- 性能的差别必须是可观察的，因而是可测量的。

已经定义了两种 PHB：一种是加速转发 PHB [RFC 3246]，另一种是确保转发 PHB [RFC 2597]。加速转发（Expedited Forwarding, EF）PHB 规定了一类流量离开路由器的速率必须等于或者大于某个已配置的速率。确保转发（Assured Forwarding, AF）PHB 将流量分为 4 类，其中每个 AF 类都确保提供某种最小数量的带宽和缓存。

我们通过对 Diffserv 的服务模型进行一些观察来结束对 Diffserv 的讨论。首先，我们隐含地假设 Diffserv 部署在单个管理域中，但典型情况是端到端的服务必须在跨越通信端系统之间的多个 ISP 来形成。为了提供端到端的 Diffserv 服务，端系统之间的所有 ISP 不仅必须提供这种服务，而且为了向端用户提供真正的端到端服务，多数 ISP 之间也要协作并作出安排。如果没有这种协作，直接向客户出售 Diffserv 的 ISP 会发现他们将重复地说：“是的，我们知道你支付了额外费用，但是我们与丢弃和延迟你的流量的 ISP 之间没有服务约定。我很抱歉在你的 VoIP 电话中有很多间隙！”第二，如果 Diffserv 实际存在并且该网络运行的负载不大，大多数时间尽力而为服务和 Diffserv 服务之间将没有可觉察的差异。的确，端到端时延通常是由接入速率和路由器跳数造成的，而不是由路由器中的排队时延造成的。想象一个支付了增值服务费用的不幸的 Diffserv 客户，发现为其他人提供的尽力而为服务几乎总是具有与增值服务相同的性能！

7.5.4 每连接服务质量保证：资源预约和呼叫准入

在前一节中，我们已经看到了分组标记和监管、流量隔离以及链路级调度能够为一类服务提供比另一类更好的性能。在某些调度规则下，如优先权调度，较低类型的流量基本对最高优先权类型的流量是“不可见”的。借助于适当的网络定制，最高类型的服务的确能够取得极低的丢包和时延，即基本上是类似于电路的性能。但是，这种网络能够确保一个在高优先权流量类型中的进行流，仅使用我们已经描述的这些机制就能在整个流期间持续得到这样的服务吗？答案是否定的。在本节中，我们将看到当对各个连接提供硬服务保证时，为什么还需要另外的网络机制和协议。

我们返回 7.5.2 节的场景中，考虑两个 1Mbps 的音频应用经 1.5Mbps 的链路传输它们的分组，如图 7-27 所示。这两条流的结合数据率（2Mbps）超过了这条链路的容量。即使使用分类和标记、流量隔离以及共享未使用的带宽（这里并没有未使用的带宽），这很明显还是一个与丢包有关的命题。只是因为没有足够的带宽同时满足两个应用的需求。如

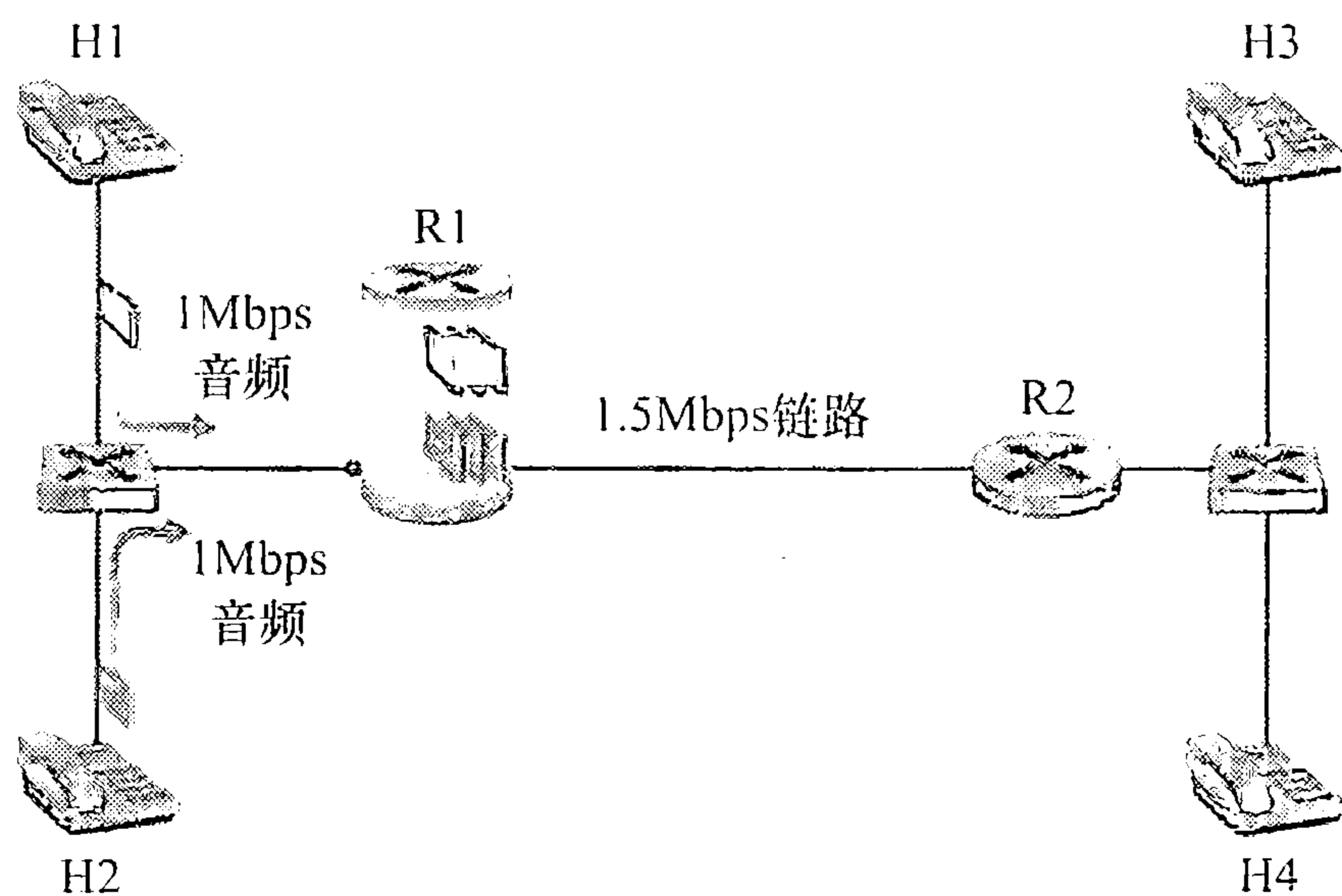


图 7-27 两个竞争的音频应用过载 R1 到 R2 的链路

果这两个应用平等地共享该带宽，每个应用将会丢失其 25% 的传输分组。这是一个无法接受的低 QoS，以至于两个音频应用完全不可用；甚至从一开始就没有必要传输任何音频分组。

在图 7-27 中两个应用不能同时满足的情况下，这个网络应当做些什么呢？如果允许两者以一种不可用的 QoS 继续在应用流上浪费网络资源，最终将无法向端用户提供任何效用。希望该答案是清晰的，即应用流中的一个应当被阻塞（即拒绝接入网络），而另一个应当允许继续进行，使该应用使用所需的全部 1Mbps。电话网是一个执行这种呼叫阻塞的网络例子，即如果要求的资源（在电话网的情况下是一个端到端的电路）不能分配给该呼叫，该呼叫就被阻塞了（阻止进入网络），并且返回给用户一个忙信号。在我们的例子中，如果一个流没有分配到足够的 QoS 来使自己可用，允许它进入网络没有任何好处。的确，接纳一个不能得到它需要的 QoS 的流是要付出代价的，因为网络资源被用于支持一个对端用户没有用的流。

通过基于流的资源要求以及已经准入流的资源要求明确地准入或阻塞流，网络能够保证准入流可得到它们所请求的 QoS。这隐含的意思是：需要为流提供有保证的 QoS 就需要流来申告它的 QoS 需求。让流申告它的 QoS 需求，然后让网络接受该流（以所要求的 QoS）或者阻塞该流的过程称为呼叫准入（call admission）过程。这则是我们的第 4 个见解（除了 7.5.2 节中前面 3 个见解外），该见解增加了提供 QoS 所需的机制。

见解 4：如果充分的资源不总是可用的，并且要确保 QoS，则需要一个呼叫准入过程，在这个过程中流申告其 QoS 需求，然后要么被网络准入（以所要求的 QoS），要么被网络阻塞（如果网络不能提供所要求的 QoS）。

如果一个呼叫（一个端到端流）一旦开始就要被确保给定的服务质量，则我们在图 7-27 中的启发性例子强调了对几种新的网络机制和协议的需求：

- 资源预留。为了满足一个呼叫所希望的 QoS，确保它具有所需的资源（链路带宽、缓存）的唯一方法是显式地为该呼叫分配这些资源，用网络的行话来讲，该过程称为资源预留（resource reservation）。一旦呼叫预约了资源，它在其整个过程中按需访问这些资源，而不管所有其他呼叫的需求。如果一个呼叫预约并得到链路带宽的 x Mbps 的保证，而且传输速率决不大于 x Mbps，则该呼叫将具有无丢包和无时延的性能。
- 呼叫准入。如果预留了资源，则该网络必须具有一种用于呼叫请求和预留资源的机制。由于资源不是无限的，如果请求的资源不可用，则进行呼叫准入请求的呼叫将被拒绝准入，即被阻塞。电话网执行的是这种呼叫准入机制，即当我们拨一个号码时，就请求了资源。如果完成该呼叫所需的电路（TDMA 时隙）是可用的，则分配电路并且完成了呼叫。如果电路不可用，则该呼叫被阻塞，我们得到了忙信号。为了得到网络的准入，被阻塞的呼叫能够一再尝试，但是直到它成功地完成呼叫准入过程，才被允许向网络发送流量。当然，分配链路带宽的路由器，其分配的带宽不应当超过链路可用的带宽。通常，一个呼叫仅能预约一条链路的带宽的一部分，因此一台路由器可以为多于一条呼叫分配链路带宽。然而，如果要提供硬服务质量保证的话，为所有呼叫分配的带宽总和应当小于该链路的容量。

- 呼叫建立信令。以上描述的呼叫准入过程要求一条呼叫沿着它的源到目的地路径在每台网络路由器中能够预约充足的资源，以确保满足它的端到端 QoS 请求。每台路由器必须决定会话所请求的本地资源，考虑已经承诺给其他进行中的会话的资源量，并决定它是否有足够的资源来满足在这台路由器上的该会话的每跳 QoS 要求，而不违反对所有已经准入的会话作出的本地 QoS 保证。需要一个信令协议来协调这些各种各样的活动，即本地资源的每跳分配，以及该呼叫是否能够在沿着端到端路径上的每台路由器中预约充分的资源。这是如图 7-28 所示的呼叫建立协议（call setup protocol）的任务。资源预留协议（resource ReSerVation Protocol, RSVP）[Zhang 1993, RFC 2210] 就是在因特网体系结构中为此目的所提出的，用于提供服务质量保证。在 ATM 网络中，Q2931b 协议 [Black 1995] 在 ATM 网络的交换机和端点之间承载了这种信息。

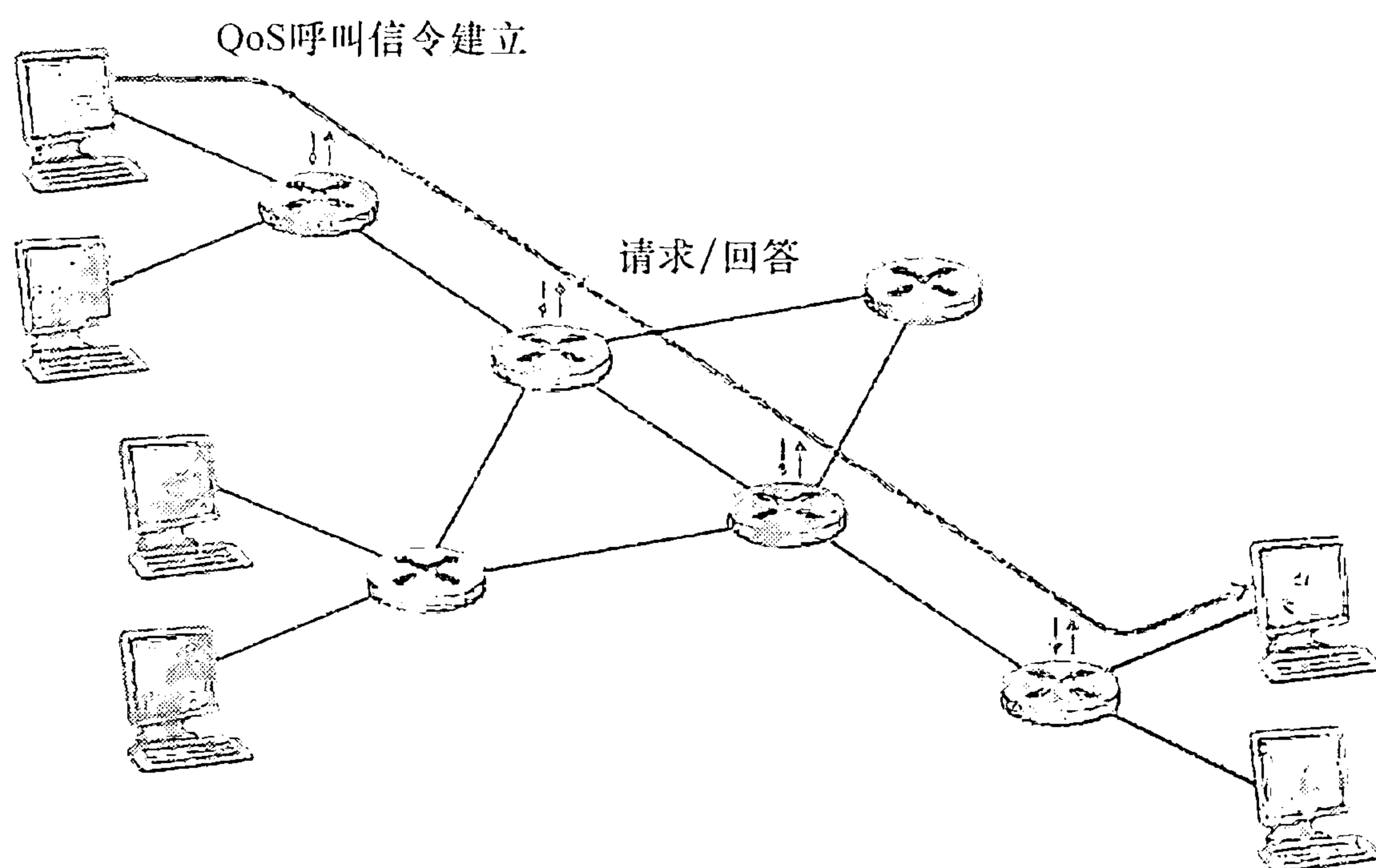


图 7-28 呼叫建立过程

尽管投入了巨大的研究和开发力量，并且甚至开发出了提供每连接服务质量保证的产品，但是几乎没有这种服务的扩展部署。有许多可能的原因。首当其冲的是，情况很可能是我们在 7.2 ~ 7.4 节学习的简单的应用级机制与适当的网络定制相结合（7.5.1 节），为多媒体应用提供了“足够好的”尽力而为的网络服务。此外，ISP 判断：提供每连接服务质量保证所需的附加复杂性以及部署和管理网络的成本，相对于该服务可预测的用户收入，前者可能过高。

7.6 小结

多媒体网络是当今因特网中最为激动人心的发展之一。全世界的人们在收音机和电视机面前花费的时间越来越少，而转向通过因特网来接收音频和视频传输，它们包含了实况转播和预先录制的节目。随着高速无线因特网接入变得越来越无所不在，该趋势无疑将继续下去。此外，借助于如 YouTube 这样的网站，用户除了是多媒体因特网内容的消费者，也成为了生产者。除了视频分发，因特网也正在用于传输电话。事实上，经过下一个 10 年，因特网连同无线因特网接入，可能致使传统的电路交换电话系统成为明日黄花。VoIP 不仅提供了廉价的电话服务，而且将提供大量的增值服务，例如视频会议、在线目录服

务、语音消息及融入如脸谱和 Google + 等社交网络。

在 7.1 节中，我们描述了视频和语音的内在特点，将多媒体应用分为 3 种类型：(i) 流式存储音频/视频；(ii) 会话式 IP 语音/视频；(iii) 流式实况音频/视频。

在 7.2 节中，我们更为深入地学习了流式存储视频。对于流式视频应用，预先录制的视频放置在服务器上，用户向这些服务器发送请求以按需观看这些视频。我们看到流式视频系统能够被分为三种类型：UDP 流、HTTP 流和适应性 HTTP 流。尽管所有这三类系统都在实践中得到了应用，但是今天的系统绝大多数应用了 HTTP 流和适应性 HTTP 流。我们观察到对于流式视频的最为重要的性能测量是平均吞吐量。在 7.2 节中我们也研究了 CDN，CDN 有助于在全世界向用户分发巨量的视频数据。我们也展望了 3 个大型因特网视频流公司所依赖的技术：Netflix、YouTube 和“看看”。

在 7.3 节中，我们研究了诸如 VoIP 等会话式多媒体应用能够设计运行在尽力而为网络上的方法。对于会话式多媒体，定时考虑是重要的，因为会话式应用是高度时延敏感的。在另一方面，会话式多媒体应用是丢包容忍的，即偶尔的丢包仅会在音频/视频播放中引起偶尔的干扰信号，并且这些丢包常常能被部分地或者全部地掩盖。我们看到将客户缓存、分组序号和时间戳结合起来的方法，极大地缓解了网络引入的时延抖动的影响。我们也展望了 Skype 所依赖的技术，Skype 是 IP 语音和 IP 视频界的领头公司之一。在 7.4 节中，我们学习了两种最为重要的用于 VoIP 的标准化协议，即 RTP 和 SIP。

在 7.5 节中，我们介绍了几种网络机制（链路级调度规则和流量监管）是如何用于在几类流量之间提供区分服务的。

课后习题和问题



复习题

7.1 节

- R1. 针对 Victor Video 正在观看 4Mbps 视频，Facebook Frank 每 20 秒钟观看一幅新的 100KB 图片，以及 Martha Music 正在听 200kbps 音频流的情况，重新构造 7-1 表。
- R2. 在视频中有两种类型的冗余。描述它们，并讨论如何能够利用它们进行有效压缩。
- R3. 假定一个模拟音频信号每秒抽样 16 000 次，并且每个样本量化为 1024 级之一，该 PCM 数字音频信号的比特率将是多少？
- R4. 多媒体应用能够分为三种类型。阐述它们的名称并对每种类型进行描述。

7.2 节

- R5. 流式多媒体系统能够分为三种类型。阐述它们的名称并对每种类型进行描述。
- R6. 列举 UDP 流的三种缺点。
- R7. 对于 HTTP 流，TCP 接收缓存和客户应用缓存是相同的東西吗？如果不是，它们是怎样交互的呢？
- R8. 考虑对于 HTTP 流的简单模型。假设服务器以 2Mbps 的恒定速率发送比特，并且当已经接收到 800 万比特时开始播放。初始缓存时延 t_p 是多少？
- R9. CDN 通常采用两种不同的服务器放置原则之一。阐述它们的名称并简要地描述这两种原则。
- R10. 在 7.2.4 节中描述了几种集群选择策略。这些策略中的哪一种找到了相对于客户的 LDNS 的一个好集群？这些策略中的哪一种找到了相对于客户自己的一个好集群？
- R11. 除了诸如时延、丢包和带宽性能等网络相关的考虑，设计一种集群选择策略时有许多附加的重要因素需要考虑。它们有哪些？

7.3 节

- R12. 端到端时延和分组时延抖动的区别是什么？分组时延抖动的原因是什么？
- R13. 为什么在预定的播放时间之后收到的分组被认为是丢失了？
- R14. 7.3 节描述了两种 FEC 方案，简要地总结它们。这两种方案通过增加开销而增加了流的传输速率。交织技术也会增加传输速率吗？

7.4 节

- R15. 接收方如何识别在不同会话中的不同 RTP 流？同一个会话中的不同流是怎样识别的？
- R16. SIP 注册器的作用是什么？一个 SIP 注册器的作用与移动 IP 中的归属代理的作用有怎样的差异？

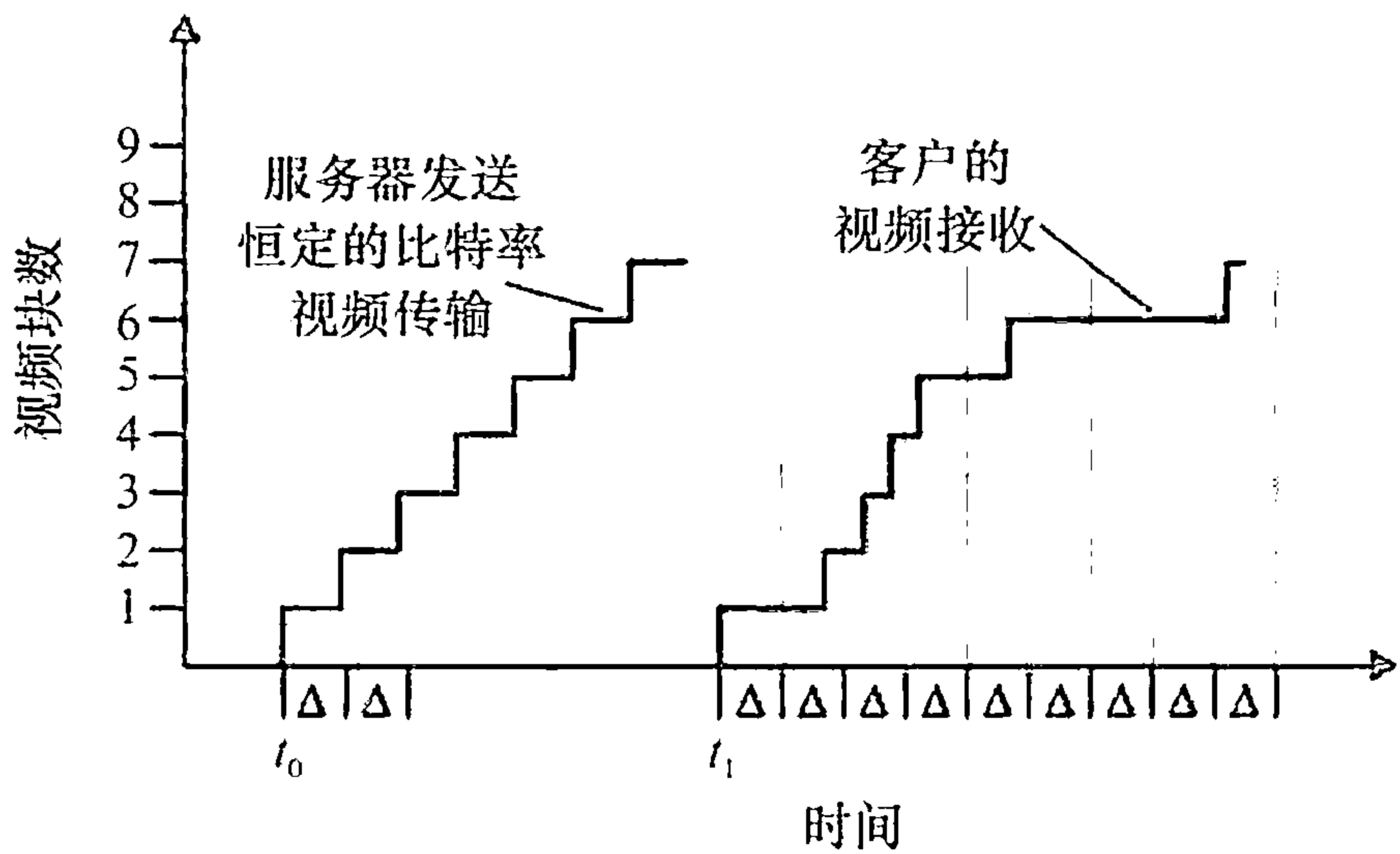
7.5 节

- R17. 在 7.5 节，我们讨论了非抢占优先级排队。抢占优先级排队是什么？抢占优先级排队对于计算机网络有意义吗？
- R18. 举一个非保持工作的调度规则的例子。
- R19. 举例说明你的日常生活中所经历的 FIFO、优先权、RR 和 WFQ 的排队。



习题

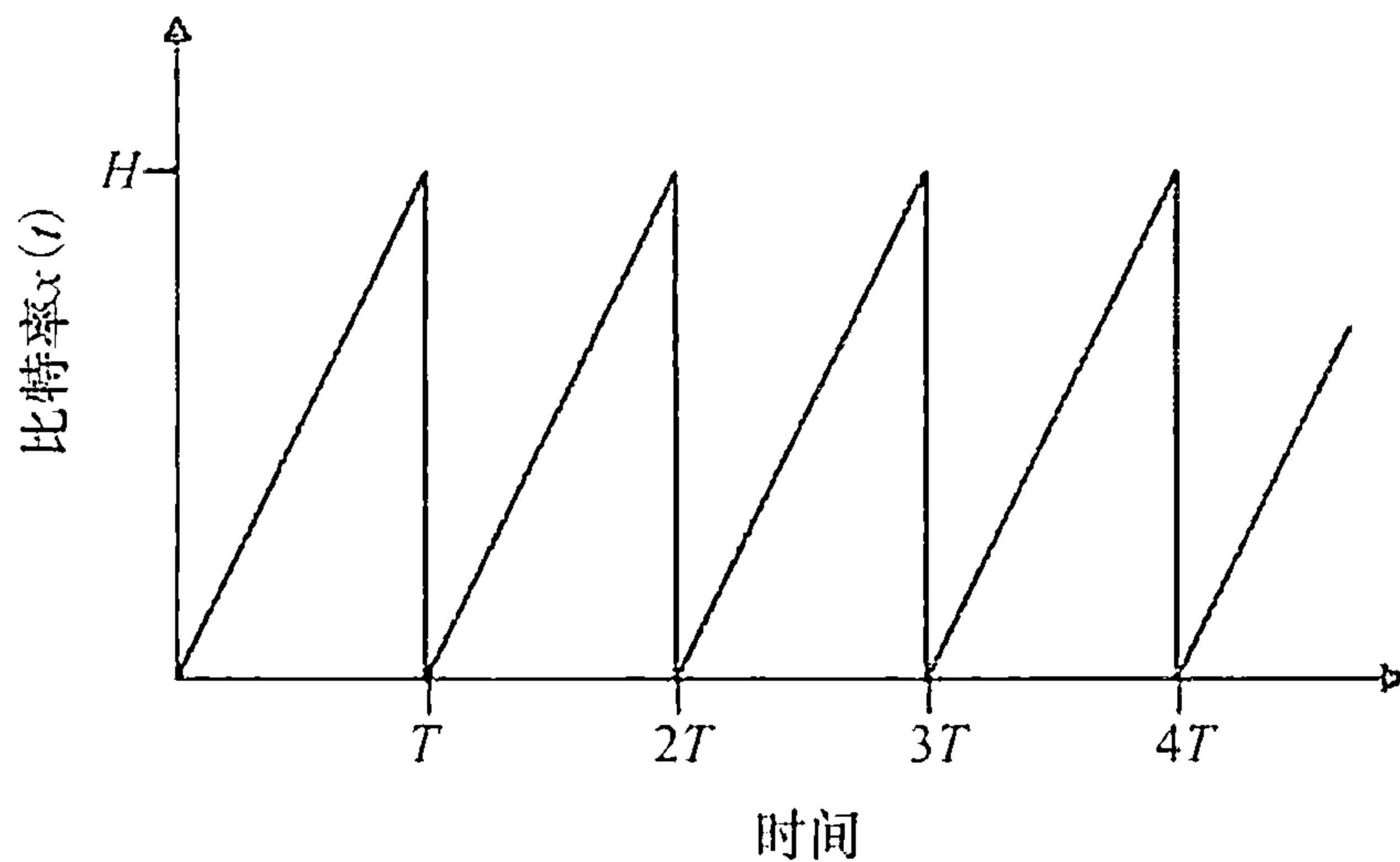
- P1. 考虑下图。类似于我们对图 7-1 的讨论，假设视频以固定比特率进行编码，因此每个视频块包含了在相同的固定时间量 Δ 播放的视频帧。服务器在 t_0 时刻传输第一个视频块，在 $t_0 + \Delta$ 时刻传输第二块，在 $t_0 + 2\Delta$ 时刻传输第三块，等等。一旦客户开始播放，每个块应当在前一块后播放 Δ 时间单元。
- a. 假设第一块在 t_1 时刻一到达，客户就开始播放。在下图中，多少视频块（包括第一块）将及时到达客户供其播放？解释你怎样得到该答案的。
 - b. 假设该客户现在在 $t_1 + \Delta$ 时刻开始播放。多少视频块（包括第一块）将及时到达客户供其播放？解释你怎样得到该答案的。
 - c. 在如上面（b）的相同场景中，存储在客户缓存中等待播放的块的最大数量有多少？解释你怎样得到该答案的。
 - d. 在客户中为使每个视频块及时到达供其播放，最小的播放时延是多少？解释你怎样得到该答案的。



- P2. 回想在图 7-3 中显示的 HTTP 流的简单模型。前面讲过 B 表示客户应用缓存的长度， Q 表示在客户应用缓存开始播放之前必须被缓存的比特数量。还有 r 表示视频消耗速率。假设无论何时客户缓存不满，服务器都以恒定速率 x 发送比特。
- a. 假设 $x < r$ 。如在本书所讨论，在此场合播放将在连续播放期和停滞期之间变动。作为 Q 、 r 和 x 的函数，确定每个连续播放和停滞期的长度。
 - b. 现在假设 $x > r$ 。求客户应用缓存变满的时刻 $t = t_f$ ？
- P3. 回想在图 7-3 中显示的 HTTP 流的简单模型。假设缓存长度是有限的，并服务器以可变的速率 $x(t)$

发送比特。具体而言，假设 $x(t)$ 具有下列锯齿形状。其速率在时刻 $t=0$ 初始为零，在时刻 $t=T$ 线性爬升到 H 。然后反复重复这种模式，如下图中显示的那样。

- 服务器的平均发送速率是什么？
- 假设 $Q=0$ ，客户一接收到一个视频块就开始播放。将发生什么现象？
- 现在假设 $Q>0$ 。作为 Q 、 H 和 T 的函数，确定首先开始播放的时间。
- 假设 $H>2r$ 和 $Q=HT/2$ 。证明在初始播放时延之后将不会有停滞。
- 假设 $H>2r$ 。求出最小的 Q 值，使在初始播放时延之后将不会有停滞。
- 现在假定缓存长度 B 是有限的。假定 $H>2r$ 。作为 Q 、 B 、 T 和 H 的函数，确定当客户应用缓存首次变满的时刻 $t=t_f$ 。



- P4. 回想在图 7-3 中显示的 HTTP 流的简单模型。假设客户应用缓存长度是有限的，服务器以恒定的速率 x 发送，并且视频的压缩率是 r ，其中 $r < x$ 。还假设播放立即开始。假设用户早在时刻 $t=E$ 中止视频。在中止的时刻，服务器停止发送比特（如果它还没有发送视频中所有比特的话）。
- 假设视频为有限长。问浪费了多少比特（即发送但未观看）？
 - 假设视频为 T 秒长，其中 $T>E$ 。问浪费了多少比特（即发送但未观看）？
- P5. 考虑一个 DASH 系统，其中有 N 个视频版本（有 N 种不同的速率和质量）和 N 种音频版本（有 N 种不同的速率和质量）。假设我们要允许播放者在任何时间对任何 N 种视频版本和任何 N 种音频版本进行选择。
- 如果我们生成文件，使音频与视频混合起来，这样服务器在给定的时间仅发送一种媒体流，该服务器将需要存储多少文件（每个文件有一种不同的 URL）？
 - 如果服务器单独发送音频流和视频流并且让客户同步这些流的话，该服务器将需要存储多少文件？
- P6. 在 7.3 节的 VoIP 例子中，令 h 为加到每个块中的首部字节总数，包括 UDP 和 IP 的首部。
- 假设每 20 秒钟发射一个 IP 数据报，求出由该应用一侧产生的数据报的每秒传输速率（以比特计）。
 - 当使用 RTP 时， h 的典型值是什么？
- P7. 考虑在 7.3 节中描述估计平均时延 d_i 的过程。假设 $u=0.1$ 。令 r_1-t_1 是最近的采样时延，令 r_2-t_2 是下一个最近的采样时延，等等。
- 对于一个特定的音频应用，假设四个分组到达接收方的采样时延为 r_4-t_4 、 r_3-t_3 、 r_2-t_2 和 r_1-t_1 。根据这 4 个采样来表示时延 d 的估计值。
 - 对于 n 个采样时延归纳出公式。
 - 对于 b 小题的公式，令 n 趋于无穷，给出最后的公式。评论一下为什么这个平均过程被称为一个指数移动平均数。
- P8. 重复习题 P7 中的 (a) 和 (b)，求出平均时延偏差的估计值。
- P9. 对于 7.3 节中的 VoIP 例子，我们引入了一个估计时延的在线过程（指数移动平均数）。在本习题中我们将研究另一种过程。设 t_i 是接收到的第 i 个分组的时间戳；设 r_i 是收到第 i 个分组的时间。令 d_n 表示在收到第 n 个分组后我们对平均时延的估计。在收到第一个分组后，我们设置时延估计值等于

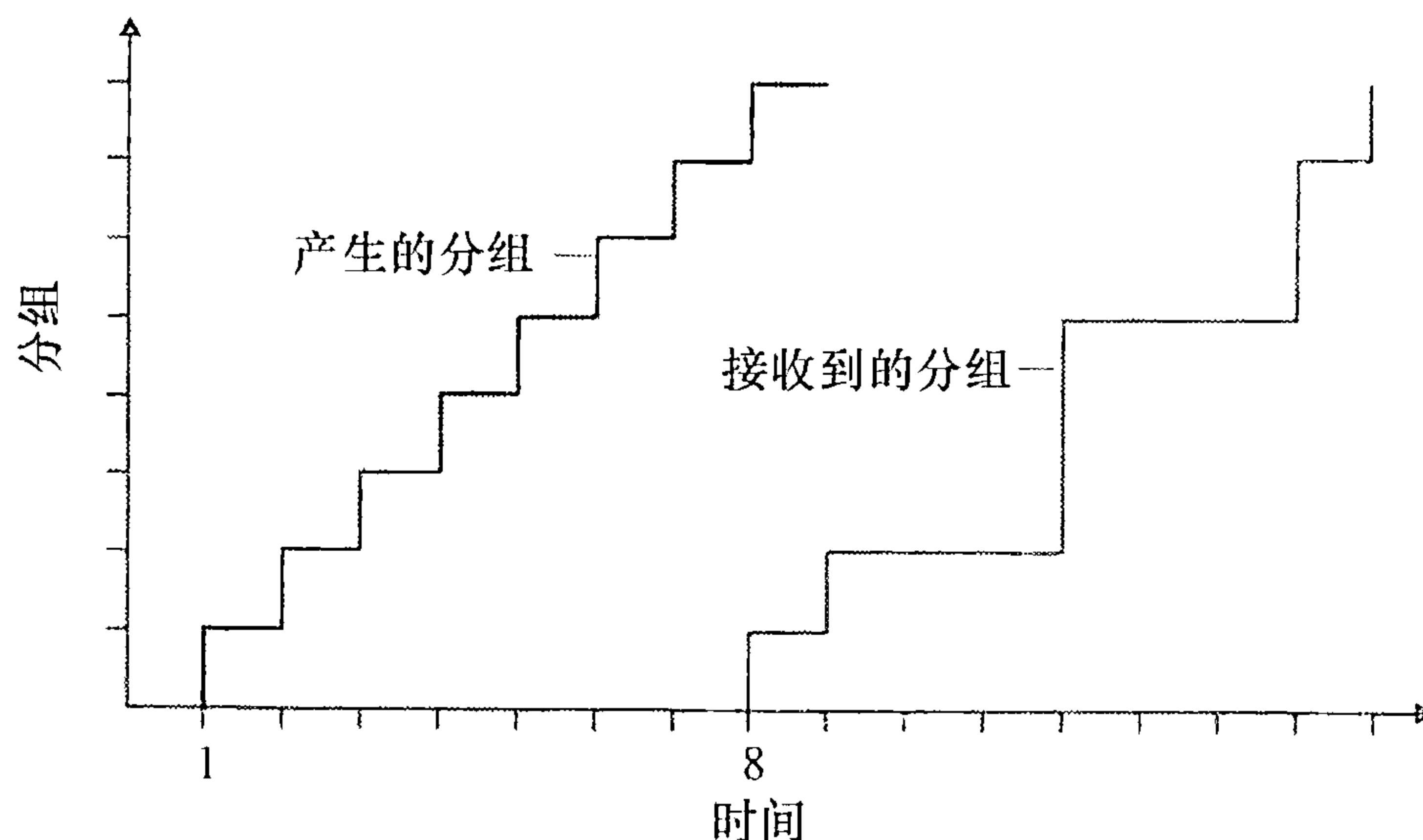
$$d_1 = r_1 - t_1。$$

a. 假设我们希望对于所有的 n , 有 $d_n = (r_1 - t_1 + r_2 - t_2 + \cdots + r_n - t_n) / n$ 。根据 d_{n-1} 、 r_n 和 t_n , 给出 d_n 的递归公式。

b. 描述一下对于 VoIP, 为什么在 7.3 节中描述的时延估计比 (a) 部分叙述的时延估计更合适。

P10. 比较 7.3 节描述的估计平均时延的过程和 3.5 节估计往返时间的过程。这些过程有什么相同? 有什么不同?

P11. 考虑下图 (它类似于图 7-7)。某发送方在 $t=1$ 时开始周期性地发送分组化的音频。在 $t=8$, 第一个分组到达接收方。



a. 分组 2 到分组 8 的时延 (从发送方到接收方, 忽略任何播放时延) 是什么? 注意到在该图上每个垂直和水平线段具有 1、2 或 3 个时间单元的长度。

b. 如果第一个分组在 $t=8$ 时刻一到达接收方, 音频就开始播放, 发送的前 8 个分组中的哪些将不能按时到达进行播放?

c. 如果在 $t=9$ 时刻音频开始播放, 发送的前 8 个分组中的哪些将不能按时到达进行播放?

d. 在接收方, 导致所有前 8 个分组按时到达进行播放的最小播放时延是什么?

P12. 再次考虑习题 P11 中的图, 显示分组音频传输和接收时间。

a. 计算分组 2 到分组 8 的估计时延, 使用 7.3.2 节中的对于 d_i 的公式。使用 $u=0.1$ 的值。

b. 使用 7.3.2 节中对于 v_i 的公式, 从对分组 2 到分组 8 的估计平均时延计算其估计的偏差。

P13. 在 7.3 节中讲过 VoIP 的两种 FEC 方案。假设第一种方案为每 4 个初始块产生一个冗余块。假设第二种方案使用传输速率为标称流传输速率的 25% 的低比特率编码。

a. 每种方案需要多少额外带宽? 每种方案增加多少播放时延?

b. 如果在每组的 5 个分组中第一个分组丢失了, 这两种方案如何执行? 哪一种方案有更好的音频质量?

c. 如果在每组的 2 个分组中第一个分组丢失了, 这两种方案如何执行? 哪一种方案有更好的音频质量?

P14. a. 考虑 Skype 中的一个音频会议具有 $N > 2$ 个与会者。假定每个与会者生成一个速率 r bps 的恒定流。呼叫的发起方将需要每秒发送多少比特? 其他 $N-1$ 个与会者每个都将需要每秒发送多少比特? 聚合所有与会者, 总的发送速率是多少?

b. 对于使用一台中心服务器的一个 Skype 视频会议, 重复 (a)。

c. 重复 (b), 只是现在情况为每个对等方发送其视频流副本到其他 $N-1$ 个对等方。

P15. a. 假设我们向因特网发送两个 IP 数据报, 每个数据报携带不同的 UDP 段。第一个数据报的源 IP 地址为 A1, 目的 IP 地址为 B, 源端口为 P1, 目的端口为 T。第二个数据报的源 IP 地址为 A2, 目的 IP 地址为 B, 源端口为 P2, 目的端口为 T。假设 A1 和 A2 不同, P1 和 P2 不同。假设这两个数据报都到达它们的目的地址, 这两个 UDP 数据报会被同一个套接字接收吗? 为什么?

b. 假设 Alice、Bob 和 Claire 要使用 SIP 和 RTP 来进行音频会议呼叫。Alice 与 Bob 和 Claire 之间发送

和接收 RTP 分组，只有一个 UDP 套接字足够吗（SIP 报文所需的套接字除外）？如果够，那么 Alice 的 SIP 客户如何区分 RTP 分组是来自 Bob 还是来自 Claire？

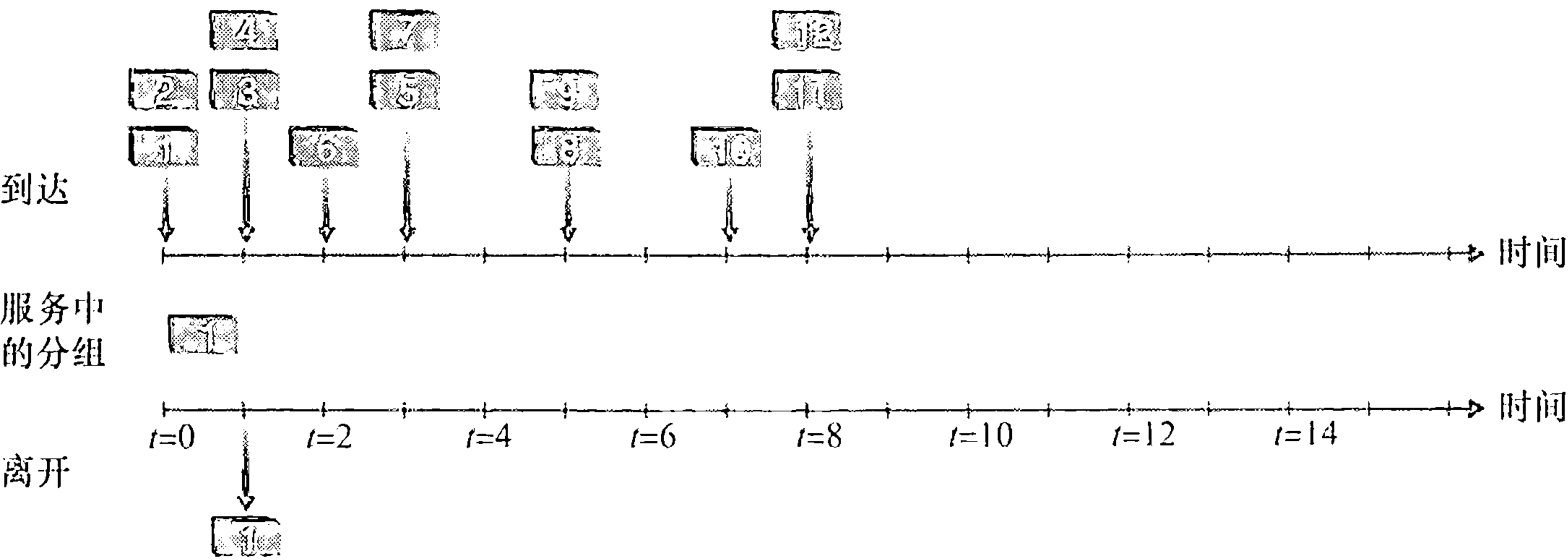
P16. 是非判断题：

- a. 如果存储视频直接从 Web 服务器流式传输到媒体播放器，这个应用则正在使用 TCP 作为底层的传输协议。
- b. 当使用 RTP 时，发送方有可能在会话的中间改变编码。
- c. 所有使用 RTP 的应用必须使用端口 87。
- d. 假设一个 RTP 会话对每个发送方有独立的音频和视频流，则这些音频和视频流使用同样的 SSRC。
- e. 在区分服务中，尽管每跳行为定义了各类型之间的性能差别，但它没有强制要求为了获得这些性能而使用任何特定机制。
- f. 假设 Alice 要和 Bob 建立一个 SIP 会话。在她的 INVITE 报文中包括了这样的行：m = audio 48753 RTP/AVP 3（AVP 3 指示 GSM 音频）。因此 Alice 在该报文中指示她要发送 GSM 音频。
- g. 参考前一句说法，Alice 在她的 INVITE 报文中指示了她将把音频发送到端口 48753。
- h. SIP 报文在 SIP 实体之间通常使用一个默认的 SIP 端口号发送。
- i. 为了维护注册，SIP 客户必须周期地发送 REGISTER 报文。
- j. SIP 强制所有的 SIP 客户支持 G.711 音频编码。

P17. 假设对一个支持 3 种流量类型的缓存运用 WFQ 调度策略，并假设这 3 种类型的权重分别为 0.5、0.25 和 0.25。

- a. 假设每种类型在缓存里都有大量的分组。为了获得这些 WFQ 权重，这 3 种类型可能以什么顺序接受服务？（对于循环调度，一种自然的序列为 123123123...）。
- b. 假设类型 1 和类型 2 在缓存中有大量的分组，缓存中没有类型 3 的分组。为了获得这些 WFQ 权重，这 3 种类型可能以什么顺序接受服务？

P18. 考虑下图。回答下列问题：



- a. 假设 FIFO 服务，指出分组 2 ~ 12 每个离开队列的时间。对每个分组，它到达和传输开始的时隙之间的时延是什么？所有 12 个分组的平均时延是什么？
- b. 现在假设优先权服务，并假设奇数编号的分组是高优先权，偶数编号的分组是低优先权。指出分组 2 ~ 12 每个离开队列的时间。对每个分组，它到达和传输开始的时隙之间的时延是什么？所有 12 个分组的平均时延是什么？
- c. 现在假设循环服务。假设分组 1、2、3、6、11 和 12 属于类型 1，分组 4、5、7、8、9 和 10 属于类型 2。指出分组 2 ~ 12 每个离开队列的时间。对每个分组，它到达和传输开始的时隙之间的时延是什么？所有 12 个分组的平均时延是什么？
- d. 现在假定加权公平排队（WFQ）服务。假设奇数编号的分组属于类型 1，偶数编号的分组属于类型 2。注意到也许不可能取得像本书中描述的那种理想的 WFQ 调度，故指出在每个时隙为什么你选择了特定分组进入服务。对每个分组，它到达和传输开始的时隙之间的时延是什么？所有

12 个分组的平均时延是什么？

e. 在所有四类（FCFS、RR、优先权和 WFQ）的平均时延中你注意到什么？

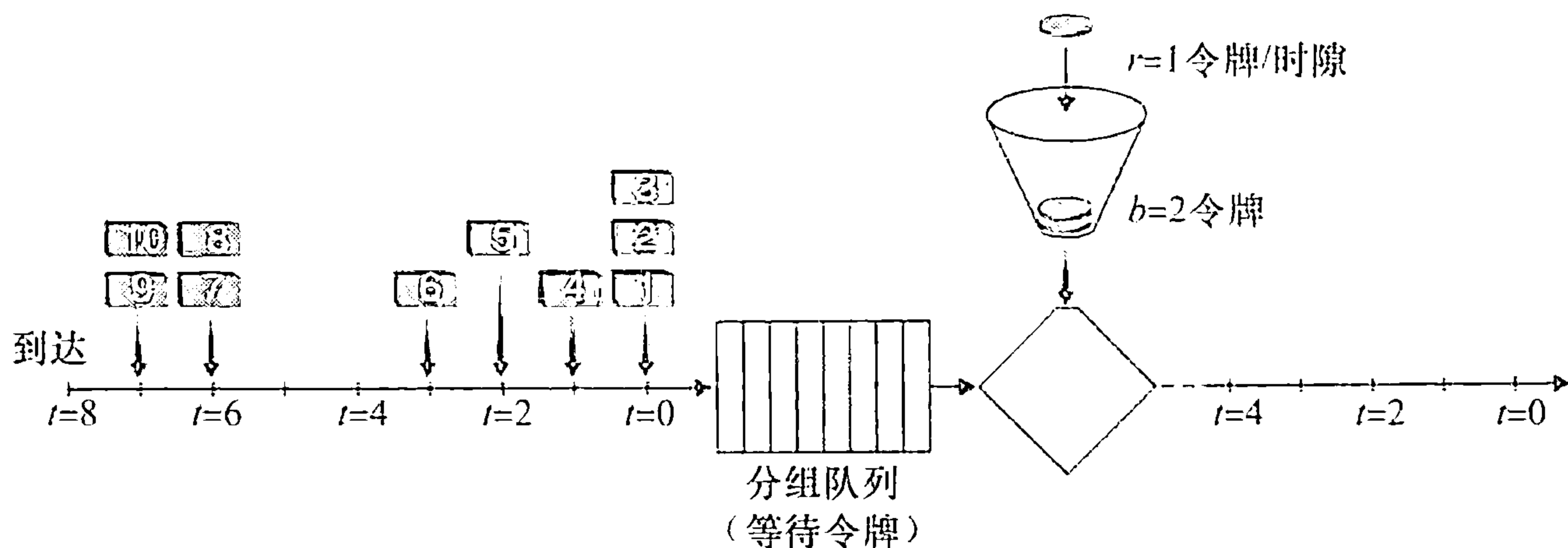
P19. 再次考虑习题 P18 中的图。

a. 假设优先权服务，分组 1、4、5、6 和 11 为高优先权分组。余下的分组是低优先权。指出分组 2 ~ 12 每个离开队列的时隙。

b. 现在假设使用循环服务，分组 1、4、5、6 和 11 属于一类流量，余下的分组属于第二类流量。指出分组 2 ~ 12 每个离开队列的时隙。

c. 现在假设使用 WFQ 服务，分组 1、4、5、6 和 11 属于一类流量，余下的分组属于第二类流量。类型 1 具有的 WFQ 权重为 1，而类型 2 具有的 WFQ 权重为 2（注意到这些权重与前面问题中的不同）。指出分组 2 ~ 12 每个离开队列的时隙。也要注意本问题中有关 WFQ 服务的提醒。

P20. 考虑下图，图中显示了一个由分组流提供输入的漏桶监管器。令牌桶能够保持最多两个令牌，并最初在 $t=0$ 时刻是满的。新的令牌以每时隙 1 个令牌的速率到达。输出链路速率采用下列规则：如果两个分组在一个时隙的开始获得令牌，它们能够在相同的时隙中进入输出链路。该系统的定时细节如下：



1. 分组在时隙的开始到达（如果有的话）。因此在该图中，分组 1、2 和 3 在时隙 0 到达。如果在队列中已经有分组，则到达的分组加入该队列的尾部。分组以 FIFO 方式朝着该队列的前方行进。
2. 当到达分组增加进队列之后，如果有任何排队分组，那些分组中的一个或两个（取决于可用令牌的数量）将每个从令牌桶中去除一个令牌，并在那个时隙去往输出链路。因此，分组 1 和 2 从桶中去除一个令牌（因为最初有两个令牌）并在时隙 0 期间去往输出链路。
3. 如果令牌桶未满载则新令牌加入，因为令牌产生速率是 $r=1$ 令牌/时隙。
4. 然后时间前进到下一个时隙，并且重复这些步骤。

回答下列问题：

- a. 对每个时隙，在到达分组处理后（上述步骤 1）但在任何分组通过队列传输并去除一个令牌之前的那个时刻，指出位于队列中的分组和位于桶中的令牌数量。因此，对于在上述例子中的 $t=0$ 时刻的时隙，分组 1、2 和 3 位于队列中，桶中有两个令牌。
- b. 对每个时隙，指出在令牌从队列中去除后哪个分组出现在输出链路。这样，对于上述例子中的 $t=0$ 时隙，在时隙 0 期间，分组 1 和 2 从漏桶出现在输出链路上。

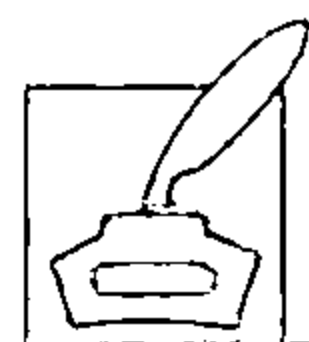
P21. 重复习题 P20，但假设 $r=2$ 。再次假设初始时桶是满的。

P22. 考虑习题 P21 并假设现在 $r=3$ 且 $b=2$ 与以前一样。你对上述问题的回答有变化吗？

P23. 考虑一下监管分组流的平均速率和突发长度的漏桶监管器。我们现在也要监管峰值速率 p 。说明这个漏桶监管器的输出如何能够提供给第二个漏桶监管器，使得这两个串行的漏桶能够监管平均速率、峰值速率和突发长度。要给出第二个监管器的桶长度和令牌产生速率。

P24. 如果对于任何 t 来说，在每个时间间隔长度 t 内到达漏桶的分组数小于 $rt+b$ 个，那么就说分组流符合一个突发长度 b 和平均速率 r 的漏桶规范 (r, b) 。一个符合漏桶规范 (r, b) 的分组流必须在参数 r 和 b 的漏桶监管器那里等待吗？评估你的答案。

P25. 说明只要 $r_1 < R w_1 / (\sum w_j)$ ，那么 $d_{\max}$ 实际上是流 1 中任何分组在 WFQ 队列中要经受的最大时延。



编程作业

在这个实验中，将实现流式视频服务器和客户。该客户使用实时流协议（RTSP）来控制服务器的动作。服务器使用实时协议（RTP）来分组化视频以便在 UDP 上传输。将为你提供在客户和服务器中部分实现了 RTSP 和 RTP 的 Python 代码，你的工作是完成客户和服务器代码。当完成时，你将已经创建了一个进行下面工作的客户-服务器应用：

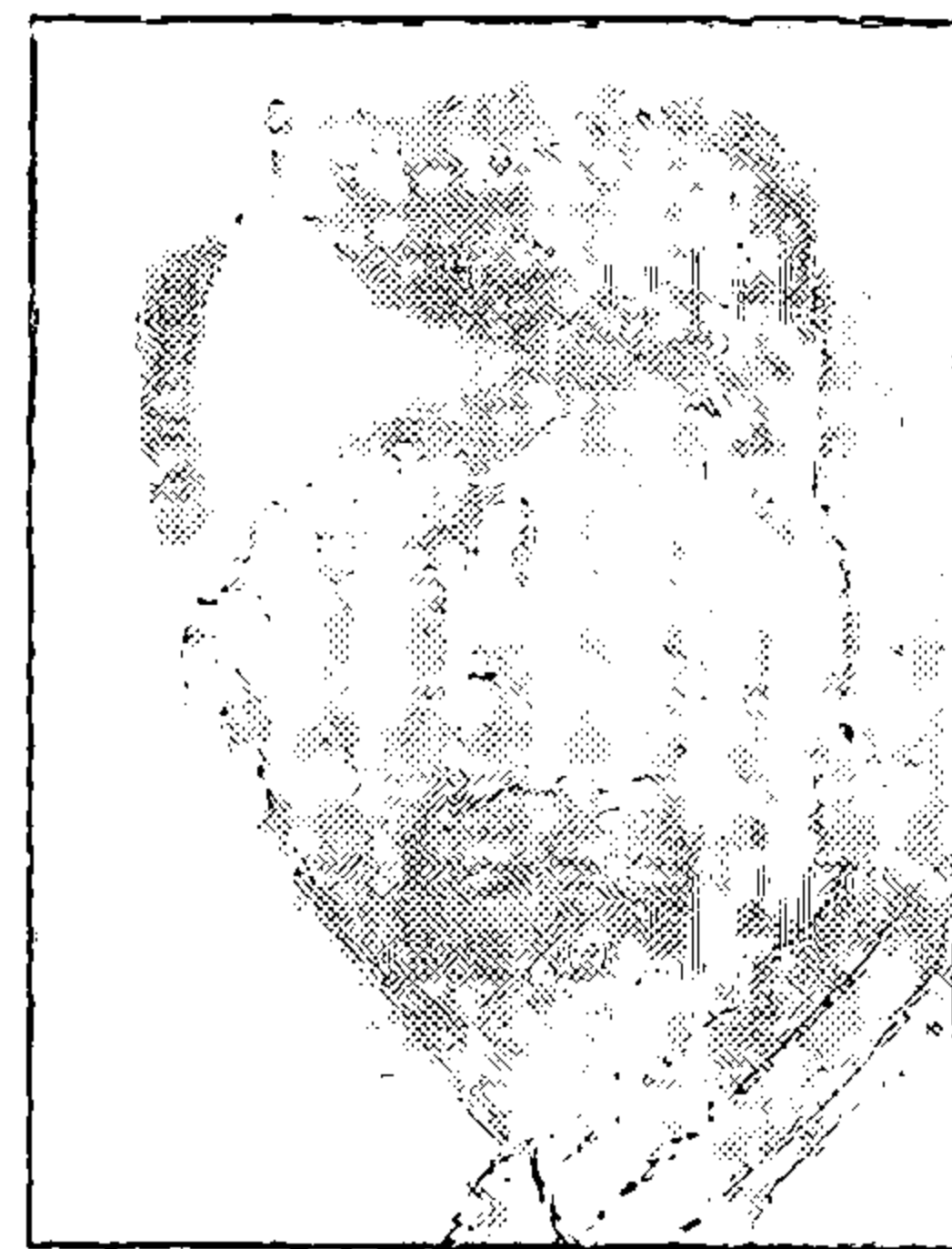
- 客户发送 SETUP、PLAY、PAUSE 和 TEARDOWN 等 RTSP 命令，并且服务器应答这些命令。
- 当服务器处于播放状态，它周期地抓取存储的 JPEG 帧，用 RTP 对该帧分组化，并将该 RTP 分组发送到一个 UDP 套接字中。
- 客户接收该 RTP 分组，去除 JPEG 帧，解压缩该帧，并在客户的监管器上再现该帧。

为你提供的代码在服务器实现 RTSP 协议，在客户实现对 RTP 解分组化。这个代码也考虑传输视频的显示。你需要在客户实现 RTSP 和 RTP 服务器。该编程作业将极大地增强学生对 RTP、RTSP 和流式视频的理解。我们极力推荐它。该作业也建议进行大量可选的练习，包括在客户和服务器实现 RTSP 的 DESCRIBE 命令。你能够在 Web 站点 <http://www.awl.com/kurose-ross> 找到该作业的全部细节以及 RTSP 协议的概述。



人物专访

Henning Schulzrinne 是哥伦比亚大学的教授、计算机科学系的主任和因特网实时实验室的负责人。他是 RTP、RTSP、SIP 和 GIST 这些经因特网进行音频和视频通信的关键协议的作者之一。Henning 在德国达姆施塔特工业大学（TU Darmstadt）获得了电子和工业工程学士学位，在辛辛那提大学获得了电子和计算机工程硕士学位，以及在马萨诸塞大学阿默斯特学院获得了电子工程博士学位。



Henning Schulzrinne

- 是什么使得您决定致力于多媒体网络？

这几乎是巧合。作为一个博士生，我从事 DARTnet 方面的工作，DARTnet 是一个用 T1 线路跨越美国的实验网络。DARTnet 用于为多播和因特网实时工具提供场所。这促使我写了我的第一个音频工具 NeVoT。通过一些 DARTnet 的参与者，我开始参与 IETF 和那时新成立的音频视频传输工作组的工作。这个工作组后来完成了 RTP 的标准化。

- 您在计算机行业中的第一份工作是什么？它给了您什么收获？

我在计算机行业的第一份工作是在加利福尼亚的 Livermore 读高中时焊接一个牵牛星（Altair）计算机工具包。回到德国，我开了一个小咨询公司来给旅行社设计一个地址管理程序，为我们的 TRS-80 开发了将数据存储到磁带上的功能，并通过一个自己制作的硬件接口把 IBM 的电动打字机作为打印机使用。

我第一份真正的工作是在 AT&T 的贝尔实验室，为在实验室环境下构建实验网络而研发的一个网络仿真器。

- 因特网实时实验室的目的是什么？

我们的目的是为因特网作为单一的未来通信基础设施提供组件和构件模块。这包括开发新协议，如 GIST（用于网络层信令）和 LoST（用于由位置寻找资源），或通过丰富呈现、对等系统、下一代紧急情况呼叫和服务产生工具等方面的工作加强我们以前从事的协议，如 SIP。最近，我们也大规模地研究了用于 VoIP 的无线系统，因为 802.11b 和 802.11n 网络以及也许 WiMAX 网络有可能成为用于电话的重要的最后 1 英里技术。我们也试图使用一种称为 DYSWIS（Do You See What I See，你所见即我所见）的对等方到对等方故障诊断系统，大大改进用户在面对提供商和设备的复杂、混乱问题时的故障诊断能力。

通过构建原型和开放源码系统、测量实际系统的性能，以及对 IETF 标准作出贡献，我们试图做些实践性相关工作。

- 您对多媒体网络的未来展望是什么？

我们现在正处于一种过渡阶段，距离 IP 成为从 IPTV 到 VoIP 等多媒体服务的通用平台只有几年之遥了。我们期望着收音机、电话和电视即使在暴风雨和地震中都能工作，所以当因特网接替了这些专用网络的职责时，用户将期待有同样级别的可靠性。

我们将不得不学会为一个生态系统设计网络技术，该生态系统包括竞争的电信公司、服务和内容提供商，服务于大量技术上未受训练的用户，并保护他们免受少数但具破坏性的一批恶意和犯罪用户的侵害。改变协议变得日益艰难。协议也变得更加复杂，因为它们需要考虑竞争的商业利益、安全性、隐私以及由防火墙和网络地址转换引起的网络透明性的欠缺。

因为多媒体网络正在成为几乎所有消费者娱乐的基础，因此以低成本管理非常大的网络将成为重点。用户将期待易于使用的网络，例如可以在所有的设备上找到相同的内容。

- 为什么 SIP 的未来很有前途？

随着现在的无线网络正在向 3G 网络升级，希望单个多媒体信令机制能够跨越所有类型的网络，包括从电缆调制解调器到企业电话网和公用无线网络。这连同软件无线电一道，在将来使下列东西成为可能：单一设备能被用于家庭网络中，能被作为无绳蓝牙电话，用于经 802.11 的企业网中和经 3G 网络的广域网中。即使在我们有这样一个通用的无线设备之前，个人移动机制使得隐藏网络之间差别成为可能。一个标识成为找到一个人的通用方法，而不必记住或者分发一堆特定技术或特定位置的电话号码。

SIP 还将提供语音（比特）传输和语音服务分离。现在打破本地电话垄断在技术上已成为可能，即一个公司提供中性的比特传输，其他的公司提供 IP “拨号音”和常用的电话服务，例如网关、呼叫转移和主叫者 ID。

除了多媒体信令，SIP 提供在因特网中缺少的一种新服务：事件通知。我们已经有了这种具有 HTTP 不完善系统和电子邮件功能的近似服务，但是这决不会令人非常满意。因为事件是对分布式系统的通用抽象，这样可能简化新服务的构建。

- 您对进入到网络领域的学生有什么建议吗？

网络沟通了多个学科。它源于电子工程和计算机科学、运筹学、经济学和其他学科的所有方面。因此，网络研究者必须熟悉除了协议和路由选择算法以外的主题。

既然网络已经成为日常生活中如此重要的一部分，要在该领域标新立异的学生们应当思考网络中新的资源限制：人的时间及努力，而不只是带宽或存储。

从事网络研究工作能够给人以极大的满足，因为它使得人们能够相互通信和交换思想，这是人类所必需的。因特网服务提供商已经成为第三大全球性基础设施，接近于运输系统和能源分配。经济领域几乎没有哪个部分离开高性能网络还能正常运转，由此对可预测的将来应当有大量机会。

计算机网络中的安全

早在 1.6 节我们就描述了某些非常盛行和危险的因特网攻击，包括恶意软件攻击、拒绝服务、嗅探、源伪装以及报文修改和删除。尽管我们已经学习了有关计算机网络的大量知识，但我们仍然没有考察如何使网络安全，使其免受那些攻击的威胁。在获得了新的计算机网络和因特网协议的专业知识后，我们现在将深入学习安全通信，特别是计算机网络能够防御那些令人厌恶的坏家伙的原理。

我们首先介绍一下 Alice 和 Bob，这两人要进行通信，并希望该通信过程是“安全”的。由于本书是一本网络教科书，因此 Alice 和 Bob 可以是两台需要安全地交换路由选择表的路由器，也可以是希望建立一个安全传输连接的客户和服务端，或者是两个交换安全电子邮件的电子邮件应用程序，所有这些学习案例都是在本章后面我们要考虑的。总之，Alice 和 Bob 是安全领域中的两个众所周知的固定设备，也许因为使用 Alice 和 Bob 更为有趣，这与命名为“A”的普通实体需要安全地与命名为“B”的普通实体进行通信的作用是一样的。需要安全通信的例子通常包括不正当的情人关系、战时通信和商业事务往来；我们宁愿用第一个例子而不用后两个例子，并乐于使用 Alice 和 Bob 作为我们的发送方和接收方，并以第一种情况为背景来讨论问题。

我们说过 Alice 和 Bob 要进行通信并希望做到“安全”，那么此处的安全其确切含义是什么呢？如我们将看到的那样，安全性（像爱一样）是多姿多彩的东西；也就是说，安全性有许多方面。毫无疑问，Alice 和 Bob 希望他们之间的通信内容对于窃听者是保密的。他们可能也想要确保当他们需要进行通信时，确实是在和对方在通信，还希望如果他们之间的通信被窃听者篡改时，他们能够检测到该通信已被这种篡改破坏。在本章的第一部分，我们将讨论能够加密通信的密码技术，鉴别正在与他通信的对方并确保报文完整性。

在本章的第二部分，我们将研究基本的密码学原则怎样能够被用于生成安全的网络协议。我们再次采用自顶向下方法，从应用层开始，将逐层（上面四层）研究安全协议。我们将研究如何加密电子邮件，如何加密一条 TCP 连接，如何在网络层提供覆盖式安全性，以及如何使无线 LAN 安全。在本章的第三部分，我们将考虑运行的安全性，这与保护机构网络免受攻击有关。特别是，我们将仔细观察防火墙和入侵检测系统是怎样加强机构网络的安全性的。

8.1 什么是网络安全

我们还是以要进行“安全”通信的情人 Alice 和 Bob 为例，开始我们的网络安全的研究。这确切地意味着什么呢？显然，Alice 希望即使他们在一个不安全的媒体上进行通信，也只有 Bob 能够理解她所发送的报文，其中入侵者（入侵者名叫 Trudy）能够在该媒体上截获从 Alice 向 Bob 传输的报文。Bob 也需要确保从 Alice 接收到的报文确实是由 Alice 所发送，并且 Alice 要确保和她进行通信的人的确就是 Bob。Alice 和 Bob 还要确保他们报文的内容在传输过程中没有被篡改。他们首先也要确信他们能够通信（即无人能够拒绝他们

接入通信所需的资源)。考虑了这些问题后,我们能够指出安全通信 (secure communication) 具有下列所需要的特性。

- 机密性 (confidentiality)。仅有发送方和希望的接收方能够理解传输报文的内容。因为窃听者可以截获报文,这必须要求报文在一定程度上进行加密 (encrypted),使截取的报文无法被截获者所理解。机密性的这个方面大概就是通常意义上对于术语安全通信的理解。我们将在 8.2 节中学习数据加密和解密的密码学技术。
- 报文完整性 (message integrity)。Alice 和 Bob 希望确保其通信的内容在传输过程中未被改变——或者恶意篡改或者意外改动。我们在可靠传输和数据链路协议中遇到的检验和技术在扩展后能够用于提供这种报文完整性,我们将在 8.3 节中研究该主题。
- 端点鉴别 (end-point authentication)。发送方和接收方都应该能证实通信过程所涉及的另一方,以确信通信的另一方确实具有他们所声称的身份。人类的面对面通信可以通过视觉识别轻易地解决这个问题。当通信实体在不能看到对方的媒体上交换报文时,鉴别就不是那么简单了。当某用户要访问一个邮箱,邮件服务器如何证实该用户就是他或她所声称的那个人呢?我们将在 8.4 节中学习端点鉴别技术。
- 运行安全性 (operational security)。几乎所有的机构(公司、大学等)今天都有了与公共因特网相连接的网络。这些网络都因此潜在地能够被危及安全。攻击者能够试图在网络主机中安放蠕虫,获取公司秘密,勘察内部网络配置并发起 DoS 攻击。我们将在 8.9 节中看到诸如防火墙和入侵检测系统等运行设备正被用于反制对机构网络的攻击。防火墙位于机构网络和公共网络之间,控制接入和来自网络的分组。入侵检测系统执行“深度分组检查”任务,向网络管理员发出有关可疑活动的警告。

明确了我们所指的网络安全的具体含义后,我们接下来考虑入侵者可能要访问的到底是哪些信息,以及入侵者可能采取哪些行动。图 8-1 阐述了一种情况。Alice (发送方) 想要发送数据给 Bob (接收方)。为了安全地交换数据,即在满足机密性、端点鉴别和报文完整性要求的情况下, Alice 和 Bob 将交换控制报文和数据报文 (以非常类似于 TCP 发送方和接收方双方交换控制报文和数据报文的方式进行)。通常将这些报文全部或部分加密。如在 1.6 节所讨论的那样,入侵者能够潜在地执行下列行为:

- 窃听——监听并记录信道上传输的控制报文和数据报文。
- 修改、插入或删除报文或报文内容。

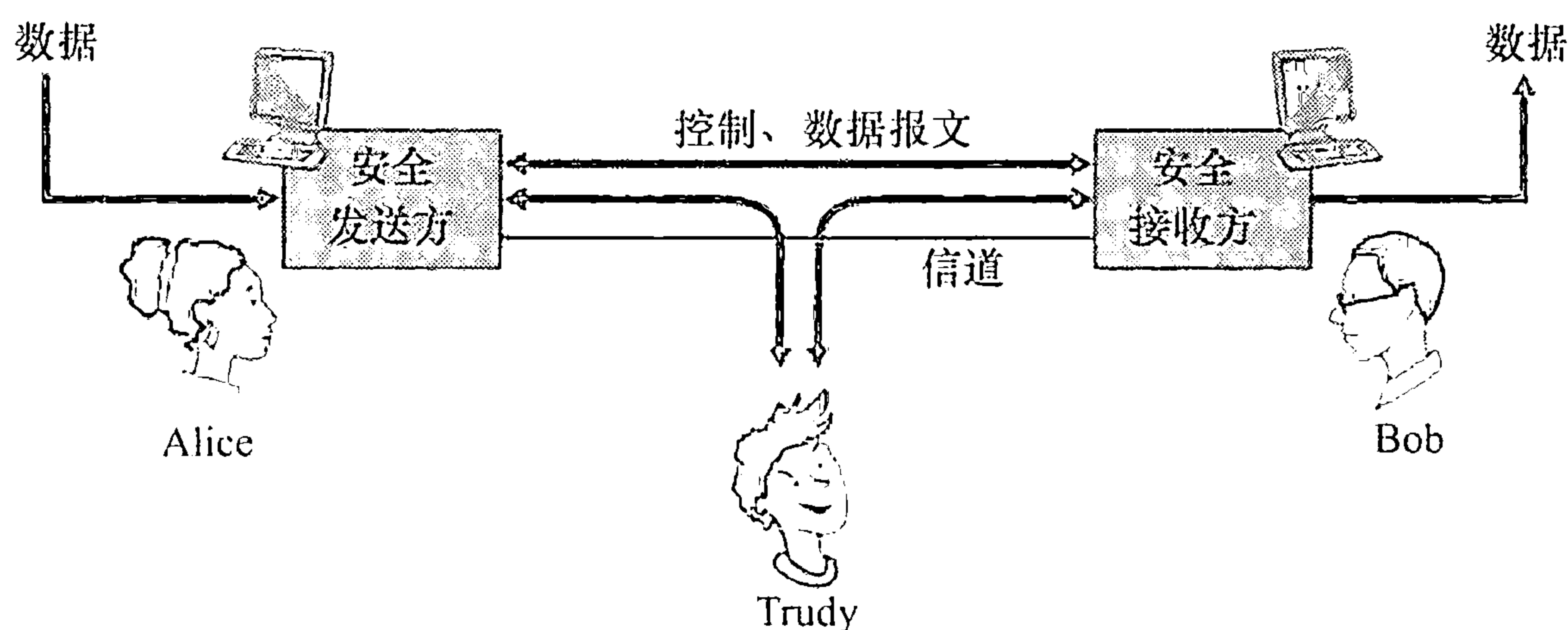


图 8-1 发送方、接收方和入侵者 (Alice、Bob 和 Trudy)

如我们将看到的那样，除非采取适当的措施，否则上述能力使入侵者可以用多种方式发动各种各样的安全攻击：窃听通信内容（可能窃取口令和数据），假冒另一个实体，“劫持”一个正在进行的会话，通过使系统资源过载拒绝合法网络用户的服务请求等等。CERT 协调中心对已报道的攻击进行了总结 [CERT 2012]。

已经知道在因特网中某处的确存在真实的威胁，则 Alice 和 Bob（我们的两个需要安全通信的朋友）在因特网上的对应实体是什么呢？当然，Alice 和 Bob 可以是位于两个端系统的人类用户，例如，真实的 Alice 和真实的 Bob 真的需要交换安全电子邮件。他们也可以参与电子商务事务。例如，一个真实的 Bob 希望安全地向一台 Web 服务器传输他的信用卡号码，以在线购买商品。类似地，真实的 Alice 要与银行在线交互。需要安全通信的各方自身也可能是网络基础设施的一部分。前面讲过，域名系统（DNS，参见 2.5 节）或交换路由选择信息的路由选择守护程序（参见 4.6 节）需要在两方之间安全通信。对于网络管理应用也有相同的情况，网络管理是第 9 章学习的主题。能主动干扰 DNS 查找和更新（如在 2.5 节中讨论的那样）、路由选择计算 [RFC 4272] 或网络管理功能 [RFC 3414] 的入侵者能够给因特网造成不可估量的破坏。

建立了上述框架，明确了一些重要定义以及网络安全需求之后，我们将深入学习密码学。应用密码学来提供机密性是不言而喻的，同时我们很快将看到它对于提供端点鉴别、报文完整性也起到了核心作用，这使得密码学成为网络安全的基石。

8.2 密码学的原则

尽管密码学的漫长历史可以追溯到朱利叶斯·凯撒（Julius Caesar）时代，但现代密码技术（包括正在今天的因特网中应用的许多技术）基于的是过去 30 年所取得的进展。Kahn 的著作《破译者（The Codebreakers）》（[Kahn 1967]）和 Singh 的著作《编码技术：保密的科学——从古埃及到量子密码（The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography）》[Singh 1999] 回顾了引人入胜的密码学的悠久历史。对密码学全面的讨论需要一本完整的书 [Kaufman 1995; Schneier 1995]，所以我们只能初步了解密码学的基本方面，特别是因为这些东西正在今天的因特网上发挥作用。我们也注意到尽管本节的重点是密码学在机密性方面的应用，但我们将很快看到密码学技术与鉴别、报文完整性和不可否认性等是紧密相关的。

密码技术使得发送方可以伪装数据，使入侵者不能从截取到的数据中获得任何信息。当然，接收方必须能够从伪装的数据中恢复出初始数据。图 8-2 说明了一些重要的术语。

现在假设 Alice 要向 Bob 发送一个报文。Alice 报文的最初形式（例如，“Bob, I love you. Alice”）被称为明文（plaintext, cleartext）。Alice 使用加密算法（encryption algorithm）加密其明文报文，生成的加密报文被称为密文（ciphertext），该密文对任何入侵者看起来是不可懂的。有趣的是在许多现代密码系统中，包括因特网上所使用的那些，加密技术本身是已知的，即公开发行的、标准化的和任何人都可使用的（例如 [RFC 1321; RFC 3447; RFC 2420; NIST 2001]），即使对潜在的入侵者也是如此！显然，如果任何人都知道数据编码的方法，则一定有一些秘密信息可以阻止入侵者解密被传输的数据。这些秘密信息就是密钥。

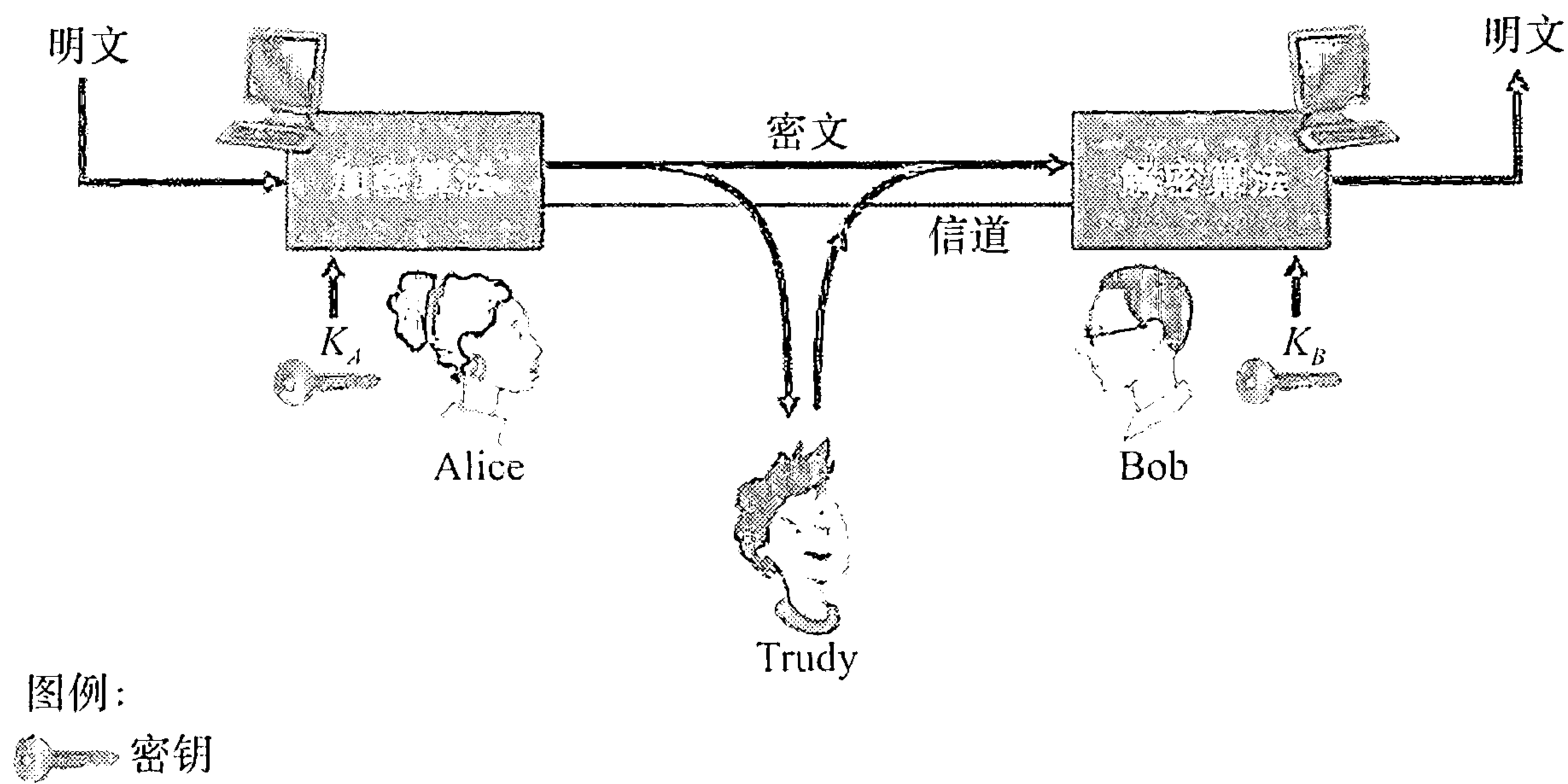


图 8-2 密码学组成部分

在图 8-2 中，Alice 提供了一个**密钥**（key） K_A ，它是一串数字或字符，作为加密算法的输入。加密算法以密钥和明文报文 m 为输入，生成的密文作为输出。用符号 $K_A(m)$ 表示（使用密钥 K_A 加密的）明文报文 m 的密文形式。使用密钥 K_A 的实际加密算法显然与上下文有关。类似地，Bob 将为**解密算法**（decryption algorithm）提供密钥 K_B ，将密文和 Bob 的密钥作为输入，输出初始明文。也就是说，如果 Bob 接收到一个加密的报文 $K_A(m)$ ，他可通过计算 $K_B(K_A(m)) = m$ 进行解密。在**对称密钥系统**（symmetric key system）中，Alice 和 Bob 的密钥是相同的并且是秘密的。在**公开密钥系统**（public key system）中，使用一对密钥：一个密钥为 Bob 和 Alice 俩人所知（实际上为全世界所知），另一个密钥只有 Bob 或 Alice 知道（而不是双方都知道）。在下面两小节中，我们将更为详细地考虑对称密钥系统和公钥系统。

8.2.1 对称密钥密码体制

所有密码算法都涉及用一种东西替换另一种东西的思想，例如，取明文的一部分进行计算，替换适当的密文以生成加密的报文。在分析现代基于密钥的密码系统之前，我们首先学习一下**凯撒密码**（Caesar cipher）找找感觉，这是一种加密数据的方法。这种非常古老而简单的对称密钥算法由 Julius Caesar 发明。

凯撒密码用于英语文本时，将明文报文中的每个字母用字母表中该字母后第 k 个字母进行替换（允许回绕，即把字母“a”排在字母“z”之后）。例如，如果 $k=3$ ，则明文中的字母“a”变成密文中的字母“d”；明文中的字母“b”变成密文中的字母“e”，依此类推。因此， k 的值就作为密钥。举一个例子，明文报文“bob, i love you. alice”在密文中变成“ere, l oryh brx. dolfh”。尽管密文看起来像乱码，但如果你知道使用了凯撒密码加密时，因为密钥值只有 25 个，所以用不了多久就可以破解它。

凯撒密码的一种改进方法是**单码代替密码**（monoalphabetic cipher），也是使用字母表中的一个字母替换该字母表中的另一个字母。然而，并非按照规则的模式进行替换（例如，明文中的所有字母都用偏移量为 k 的字母进行替换），只要每个字母都有一个唯一的替换字母，任一字母都可用另一字母替换，反之亦然。图 8-3 为加密明文的一种可行替换规则。

明文字母:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
密文字母:	m	n	b	v	c	x	z	a	s	d	f	g	h	j	k	l	p	o	i	u	y	t	r	e	w	q

图 8-3 单码代替密码

明文报文“bob, i love you. alice”变成“nkn, s gktc wky. mgsbc”。因此, 与用凯撒密码情况一样, 这看起来像乱码。单码代替密码的性能看来要比凯撒密码的好得多, 可能的字母配对为 $26!$ (为 10^{26} 数量级), 而不是 25 种可能的配对。尝试所有的 10^{26} 种可能配对的蛮力法, 其要求的工作量太大, 不是一种破解加密算法和解密报文的可行方式。但是, 通过对明文语言进行统计分析, 例如, 在典型的英语文本中, 由于已知字母“e”和字母“t”出现的频率最高 (这些字母出现的频率分别为 13% 和 9%), 还可知常见的二三个字母的组合通常一起出现 (例如, “in”、“it”、“the”、“ion”、“ing”等等), 这就使得破解该密文变得相对容易。如果入侵者具有某些该报文的可能内容的知识, 则破解该密码就会更为容易。例如, 如果入侵者 Trudy 是 Bob 的妻子, 怀疑 Bob 和 Alice 有暧昧关系, 则她可能猜想“bob”和“alice”这些名字可能会出现在密文中。如果 Trudy 确信这两个名字出现在密文中, 并有了上述报文的密文副本, 她则能够立即决定这 26 个字母配对中的 7 个, 比蛮力法少检查 10^9 种可能性。如果 Trudy 的确怀疑 Bob 有不正当的男女关系, 她可能也非常期待从该报文中找到某些其他选择的词汇。

当考虑 Trudy 破解 Bob 和 Alice 之间加密方案的难易程度时, 可以根据入侵者所拥有的信息区分三种不同的情况。

- 唯密文攻击。有些情况下, 入侵者只能得到截取的密文, 也不了解明文报文的内容。我们已经看到, 统计分析有助于对加密方案的唯密文攻击 (ciphertext-only attack)。
- 已知明文攻击。前面已经看到, 如果 Trudy 以某种方式确信在密文报文中会出现“bob”和“alice”, 她就可以确定字母 a、l、i、c、e、b 和 o 的 (明文, 密文) 匹配关系。Trudy 也可能会幸运地记录到传输的所有密文, 然后在一张纸上找到 Bob 写下的已解密的明文。当入侵者知道 (明文, 密文) 的一些匹配时, 我们将其称之为对加密方案的已知明文攻击 (known-plaintext attack)。
- 选择明文攻击。在选择明文攻击 (chosen-plaintext attack) 中, 入侵者能够选择某一明文报文并得到该明文报文对应的密文形式。对于我们前面所说的简单加密算法来说, 如果 Trudy 能让 Alice 发送报文“The quick brown fox jumps over the lazy dog,”, 则 Trudy 就能够完全破解 Alice 和 Bob 所使用的加密方案。但是随后我们将看到, 对于更为复杂的加密技术来说, 使用选择明文攻击不一定意味着能够攻破该加密机制。

500 年前, 发明了多码代替密码 (polyalphabetic encryption), 这种技术是对单码代替密码的改进。多码代替密码的基本思想是使用多个单码代替密码, 一个单码代替密码用于加密某明文报文中一个特定位置的一个字母。因此, 在某明文报文中不同位置出现的相同字母可能以不同的方式编码。图 8-4 中显示了多码代替密码机制的一个例子。它使用两个凯撒密码 (其中 $k=5$ 和 $k=19$), 如图中不同的行所示。我们可以选择使用这两个凯撒密码 C_1 和 C_2 , 加密时采用以 C_1, C_2, C_2, C_1, C_2 的次序循环的模式即明文的第一个字母用 C_1 加密, 第二和第三个字母用 C_2 编码, 第四个字母使用 C_1 , 第五个字母用 C_2 , 然后循环重复该模式, 即第六个字母用 C_1 加密, 第七个字母用 C_2 加密, 依此类推。这样一来, 明文报文“bob, i love you.”加密后成为“ghu, n etox dhz.”。注意到明文报文中的第一个“b”用 C_1 加密为“g”, 第二个“b”用 C_2 加密为“u”。在这个例子中, 加密和解密“密钥”是两个凯撒密码密钥 ($k=5$ 和 $k=19$) 和 C_1, C_2, C_2, C_1, C_2 的次序模式的知识。

明文字母: a b c d e f g h i j k l m n o p q r s t u v w x y z
C₁(k=5): f g h i j k l m n o p q r s t u v w x y z a b c d e
C₂(k=19): t u v w x y z a b c d e f g h i j k l m n o p q r s

图 8-4 使用两个凯撒密码的多码代替密码

1. 块密码

我们现在跳回到现代社会中来，考察对称密钥加密今天的工作方式。对称加密技术有两种宽泛的类型：**流密码**（stream ciphers）和**块密码**（block ciphers）。当我们研究无线 LAN 的安全性时，将在 8.7 节中简要地研究流密码。在本节中，我们关注块密码，该密码用于多种因特网协议的加密中，包括 PGP（用于安全电子邮件）、SSL（用于使 TCP 连接更安全）和 IPsec（用于使网络层传输更安全）。

在块密码中，要加密的报文被处理为 k 比特的块。例如，如果 $k = 64$ ，则报文被划分为 64 比特的块，每块被独立加密。为了加密一个块，该密码采用了一对一映射，将 k 比特块的明文映射为 k 比特块的密文。我们来看一个例子。假设 $k = 3$ ，因此块密码将 3 比特输入（明文）映射为 3 比特输出（密文）。表 8-1 给出了一种可能的映射。注意到这是一个一对一的映射，即对每种输入有不同的输出。这种块密码将报文划分成 3 比特的块并根据映射关系进行加密。可以验证，报文 010110001111 被加密成了 101000111001。

表 8-1 一种特定的 3 比特块密码

输入	输出	输入	输出
000	110	100	011
001	111	101	010
010	101	110	000
011	100	111	001

继续这个 3 比特块的例子，注意到上述映射只是许多可能映射中的一种。有多少种可能的映射呢？要回答这个问题，观察到一个映射只不过是所有可能输入的排列。共有 $2^3 (=8)$ 种可能的输入（排列在“输入”栏中）。这 8 种输入能够排列为 $8! = 40\,320$ 种不同方式。因为这些排列的每种都定义了一种映射，共有 40 320 种可能的映射。我们能够将这些映射的每种视为一个密钥，即如果 Alice 和 Bob 都知道该映射（密钥），他们能够加密和解密在他们之间发送的报文。

对这种密码的蛮力攻击即通过使用所有映射来尝试解密密文。仅使用 40 320 种映射（当 $k = 3$ ），这能够在一台桌面 PC 上迅速完成。为了挫败蛮力攻击，块密码通常使用大得多的块，由 64 比特甚至更多比特组成。注意到对于通常的 k 比特块密码，可能映射数量是 $2^k!$ ，对于即使不大的 k 值（如 $k = 64$ ），这也是一个天文数字。

如刚才所述，尽管全表块密码对于不大的 k 值能够产生健壮的对称密钥加密方案，但不幸的是它们难以实现。对于 $k = 64$ 和给定的映射，将要求 Alice 和 Bob 维护一张具有 2^{64} 个输入值的表，这是一个难以实现的任务。此外，如果 Alice 和 Bob 要改变密钥，他们将不得不每人重新生成该表。因此，全表块密码在所有输入和输出之间提供了预先决定的映射（如在上述例子中那样），这简直是不可能实现的事。

取而代之的是，块密码通常使用函数模拟随机排列表。在图 8-5 中显示了当 $k = 64$ 时这种函数的一个例子（引自 [Kaufman 1995]）。该函数首先将 64 比特块划分为 8 个块，

每个块由 8 比特组成。每个 8 比特块由一个“8 比特到 8 比特”表处理，这是个可管理的长度。例如，第一个块由标志为 T_1 的表来处理。接下来，这 8 个输出块被重新装配成一个 64 比特的块。该输出被回馈到 64 比特的输入，开始了第二次循环。经 n 次这样的循环后，该函数提供了一个 64 比特的密文块。这种循环的目的是使得每个输入比特影响最后输出比特的大部分（即使不是全部）。（如果仅使用一次循环，一个给定的输入比特将仅影响 64 输出比特中的 8 比特。）这种块密码算法的密钥将是 8 张排列表（假定置乱函数是公共已知的）。

目前有一些流行的块密码，包括 DES（Data Encryption Standard，数据加密标准）、3DES 和 AES（Advanced Encryption Standard，高级加密标准）。这些标准中的每种都使用了函数（而不是预先决定的表），连同图 8-5 的线（虽然对每种密码来说更为复杂和具体）。这些算法中的每种也使用了比特串作为密钥。例如，DES 使用了具有 56 比特密钥的 64 比特块。AES 使用 128 比特块，能够使用 128、192 和 256 比特长的密钥进行操作。一个算法的密钥决定了特定“小型表”映射和该算法内部的排列。对这些密码中的每种进行蛮力攻击要循环通过所有密钥，用每个密钥应用解密算法。观察到采用长度为 n 的密钥，有 2^n 种可能的密钥。NIST [NIST 2001] 估计，如果用 1 秒钟破解 56 比特 DES 的计算机（就是说，每秒钟尝试所有 2^{56} 个密钥）来破解一个 128 比特的 AES 密钥，要用大约 149 万亿年的时间才有可能成功。

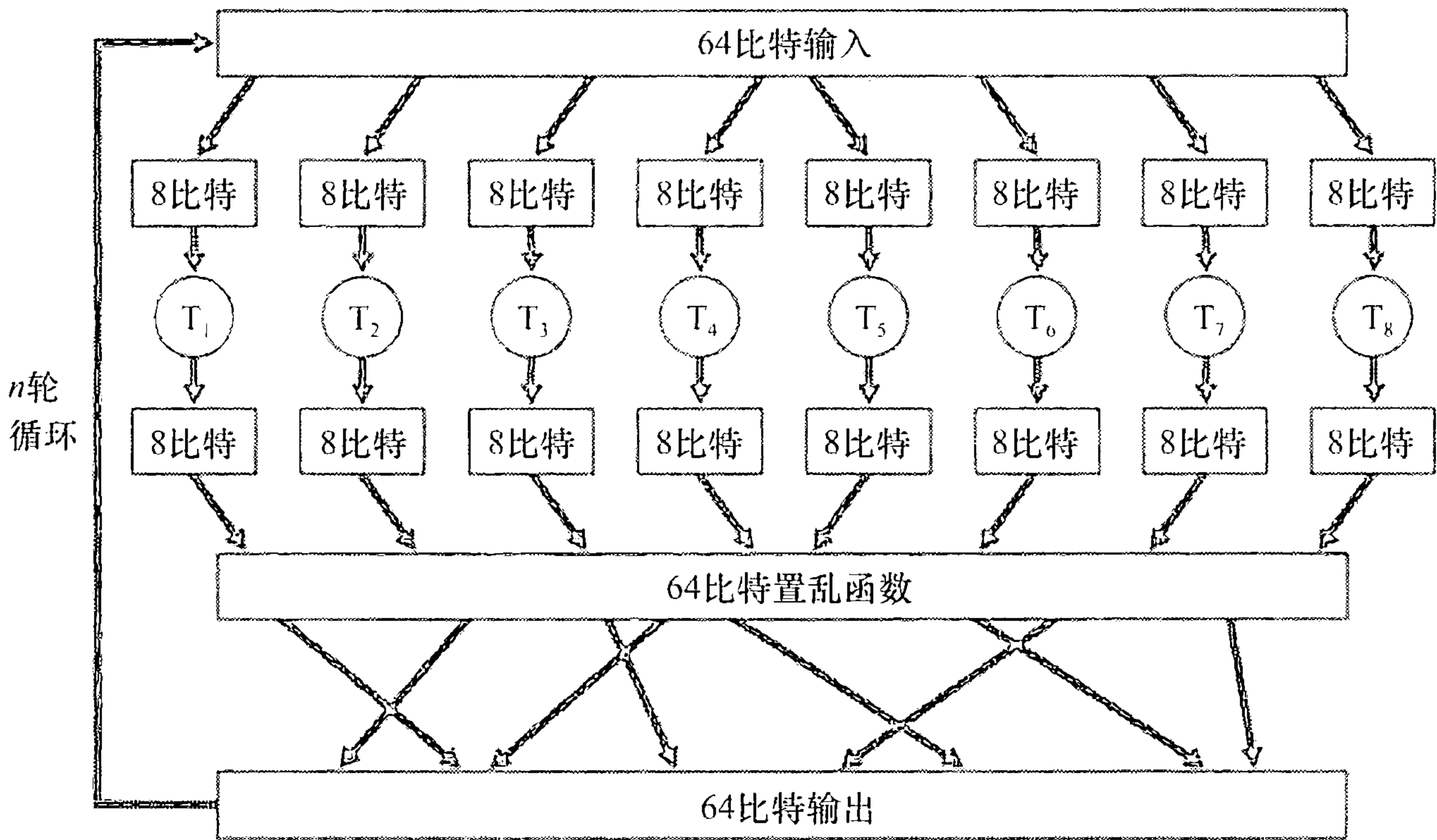


图 8-5 一个块密码的例子

2. 密码块链接

在计算机网络应用中，我们通常需要加密长报文（或长数据流）。如果我们使用前面描述的块密码，通过直接将报文切割成 k 比特块并独立地加密每块，将出现一个微妙而重要的问题。为了理解这个问题，注意到两个或更多个明文块可能是相同的。例如，两个或更多块中的明文可能是“HTTP/1.1”。对于这些相同的块，块密码当然将产生相同的密文。当攻击者看到相同的密文块时，它可能潜在地猜出其明文，并且通过识别相同的密文块和利用支撑协议结构的知识，甚至能够解密整个报文 [Kaufman 1995]。

为了解决这个问题，我们能够在密文中混合某些随机性，使得相同的明文块产生不同

的密文块。为了解释这个想法，令 $m(i)$ 表示第 i 个明文块， $c(i)$ 表示第 i 个密文块，并且 $a \oplus b$ 表示两个比特串 a 和 b 的异或（XOR）。（前面讲过 $0 \oplus 0 = 1 \oplus 1 = 0$ 和 $0 \oplus 1 = 1 \oplus 0 = 1$ ，并且两个比特串的异或是逐位进行的。因此有，例如 $10101010 \oplus 11110000 = 01011010$ 。）另外，将具有密钥 S 的块密码加密算法表示为 K_S 。其基本思想如下：发送方为第 i 块生成一个随机的 k 比特数 $r(i)$ ，并且计算 $c(i) = K_S(m(i) \oplus r(i))$ 。注意到每块选择一个新的 k 比特随机数。则发送方发送 $c(1)$ 、 $r(1)$ 、 $c(2)$ 、 $r(2)$ 、 $c(3)$ 和 $r(3)$ 等等。因为接收方接收到 $c(i)$ 和 $r(i)$ ，它能够通过计算 $m(i) = K_S(c(i) \oplus r(i))$ 而恢复每个明文块。重要的是注意到下列事实：尽管 $r(i)$ 是以明文发送的，并且因此能被 Trudy 嗅探到，但她无法获得明文 $m(i)$ ，因为她不知道密钥 K_S 。同时注意到如果两个明文块 $m(i)$ 和 $m(j)$ 是相同的，对应的密文块 $c(i)$ 和 $c(j)$ 将是不同的（只要随机数 $r(i)$ 和 $r(i)$ 不同，这种情况出现的概率将很高）。

举例来说，考虑在表 8-1 中的 3 比特块密码。假设明文是 010010010。如果 Alice 直接对此加密，没有包括随机性，得到的密文变为 101101101。如果 Trudy 嗅探到该密文，因为这三个密文块的每个都是相同的，她能够正确地推断出这三个明文块的每个都是相同的。现在假设 Alice 产生了随机块 $r(1) = 001$ 、 $r(2) = 111$ 和 $r(3) = 100$ ，并且使用了上述技术来生成密文 $c(1) = 100$ 、 $c(2) = 010$ 和 $c(3) = 000$ 。注意到即使明文块相同，三个密文块也是不同的。Alice 则发送 $c(1)$ 、 $r(1)$ 、 $c(2)$ 和 $r(2)$ 。读者可证实 Bob 能够使用共享的密钥 K_S 获得初始的明文。

机敏的读者将注意到，引入随机性解决了一个问题而产生了另一个问题：Alice 必须传输是以前两倍的比特。实际上，对每个加密比特，她现在必须再发送一个随机比特，使需要的带宽加倍。为了有效利用该技术，块密码通常使用了一种称为密码块链接（Cipher Block Chaining, CBC）的技术。其基本思想是仅随第一个报文发送一个随机值，然后让发送方和接收方使用计算的编码块代替后继的随机数。具体而言，CBC 运行过程如下：

1) 在加密报文（或数据流）之前，发送方生成一个随机的 k 比特串，称为初始向量（Initialization Vector, IV）。将该初始向量表示为 $c(0)$ 。发送方以明文方式将 IV 发送给接收方。

2) 对第一个块，发送方计算 $m(1) \oplus c(0)$ ，即计算第一块明文与 IV 的异或。然后通过块密码算法运行得到的结果以得到对应的密文块，即 $c(1) = K_S(m(1) \oplus c(0))$ 。发送方向接收方发送加密块 $c(1)$ 。

3) 对于第 i 个块，发送方根据 $c(i) = K_S(m(i) \oplus c(i-1))$ 生成第 i 个密文块。

我们现在来考察这种方法的某些后果。首先，接收方将仍能够恢复初始报文。毫无疑问，当接收方接收到 $c(i)$ ，它用 K_S 解密之以获得 $s(i) = m(i) \oplus c(i-1)$ ；因为接收方已经知道 $c(i-1)$ ，它则从 $m(i) = s(i) \oplus c(i-1)$ 获得明文块。第二，即使两个明文块是相同的，相应的密文块将（几乎）总是不同的。第三，虽然发送方以明文发送 IV，入侵者将仍不能解密密文块，因为该入侵者不知道秘密密钥 S 。最后，发送方仅发送一个最前面的块（即 IV），因此对（由数百块组成的）长报文而言增加的带宽用量微不足道。

举例来说，对表 8-1 中的 3 比特块密码，明文为 010010010 和 $IV = c(0) = 001$ ，我们现在来确定其密文。发送方首先使用 IV 来计算 $c(1) = K_S(m(1) \oplus c(0)) = 100$ 。发送方然后计算 $c(2) = K_S(m(2) \oplus c(1)) = K_S(010 \oplus 100) = 000$ ，并且 $c(3) = K_S(m(3) \oplus c(2)) =$

$K_s(010 \oplus 000) = 101$ 。读者可证实接收方若知道了 IV 和 K_s ，将能够恢复初始的明文。

当设计安全网络协议时，CBC 有一种重要的后果：我们需要在协议中提供一种机制，以从发送方向接收方分发 IV。在本章稍后我们将看到几个协议是如何这样做的。

8.2.2 公开密钥加密

从凯撒密码时代直到 20 世纪 70 年代的两千多年以来，加密通信都需要通信双方共享一个共同秘密，即用于加密和解密的对称密钥。这种方法的一个困难是两方必须就共享密钥达成一致；但是这样做的前提是需要通信（可假定是安全的）！可能是双方首先会面，人为协商确定密钥（例如，凯撒的两个百夫长在罗马浴室碰头），此后才能进行加密通信。但是，在网络世界中，通信各方之间可能从未见过面，也不会在网络以外的任何地方交谈。此时通信双方能够在没有预先商定的共享密钥的条件下进行加密通信吗？在 1976 年，Diffie 和 Hellman [Diffie 1976] 论证了一个解决这个问题的算法（现在称为 Diffie-Hellman 密钥交换），这是个完全不同、极为优雅的安全通信算法，开创了如今的公开密钥密码系统的发展之路。我们很快就会看到公开密钥密码系统也有许多很好的特性，使得它不仅可以用于加密，还可以用于鉴别和数字签名。有趣的是，最近发现 20 世纪 70 年代早期由英国通信电子安全团体的研究人员独立研究的一系列秘密报告中的思想，与 [Diffie 1976] 和 [RSA 1978] 中的思想类似 [Ellis 1987]。事实常常如此，伟大的想法通常会在许多地方独立地闪现；幸运的是，公钥的进展不仅秘密地发生，而且也在公众视野中发生。

公开密钥密码的使用在概念上相当简单。假设 Alice 要和 Bob 通信。如图 8-6 所示，这时 Alice 和 Bob 并未共享一个密钥（如同在对称密钥系统情况下），而 Bob（Alice 报文的接收方）则有两个密钥，一个是世界上的任何人（包括入侵者 Trudy）都可得到的公钥（public key），另一个是只有 Bob 知道的私钥（private key）。我们使用符号 K_B^+ 和 K_B^- 来分别表示 Bob 的公钥和私钥。为了与 Bob 通信，Alice 首先取得 Bob 的公钥，然后用这个公钥和一个众所周知的（例如，已标准化的）加密算法，加密她要传递给 Bob 的报文 m ；即 Alice 计算 $K_B^+(m)$ 。Bob 接收到 Alice 的加密报文后，用其私钥和一个众所周知的（例如，已标准化的）解密算法解密 Alice 的加密报文，即 Bob 计算 $K_B^-(K_B^+(m))$ 。后面我们将看到，存在着可以选择公钥和私钥的加密/解密算法和技术，使得 $K_B^-(K_B^+(m)) = m$ ；也就是说，用 Bob 的公钥 K_B^+ 加密报文 m （得到 $K_B^+(m)$ ），然后再用 Bob 的私钥 K_B^- 解密报文的密文形式（就是计算 $K_B^-(K_B^+(m))$ ）就能得到最初的明文 m 。这是个不寻常的结果！用这种办法，Alice 可以使用 Bob 公开可用的密钥给 Bob 发送机密信息，而他们任一方都无须分发任何密钥！我们很快能够看到，公钥和私钥加密相互交换同样能够得到不寻常的结果，即 $K_B^-(K_B^+(m)) = K_B^+(K_B^-(m)) = m$ 。

因此公开密钥密码体制的使用在概念上是简单的。但是有两点必须要注意。首先应关注的是，尽管入侵者截取到 Alice 的加密报文时看到的只是乱码，但是入侵者知道该公钥（显然 Bob 的公钥是全世界都可以使用的）和 Alice 加密所用的算法。Trudy 可以据此发起选择明文攻击，使用已知的标准加密算法和 Bob 的公开可用的加密密钥对她所选择的任意报文加密！例如，Trudy 可以尝试对她怀疑 Alice 可能发送的报文的全部或部分编码。很明显，如果公开密钥密码要能工作的话，密钥选择和加密/解密算法必须保证任一个入侵者

都不能（至少要困难得几乎不可能）确定出 Bob 的私钥或以某种方式解密或猜出 Alice 发给 Bob 的报文。第二个值得关注的问题是，既然 Bob 的加密密钥是公开的，任何人（包括 Alice 和其他声称自己是 Alice 的人）都可能向 Bob 发送一个已加密的报文。在单一共享密钥情况下，发送方知道共享秘密密钥的事实就已经向接收方隐含地证实了发送方的身份。然而在公钥体制中，这点就行不通了，因为任何一个人都可向 Bob 发送使用 Bob 的公开可用密钥加密的报文。这就需要用数字签名把发送方和报文绑定起来，数字签名是我们在 8.3 节中讨论的主题。

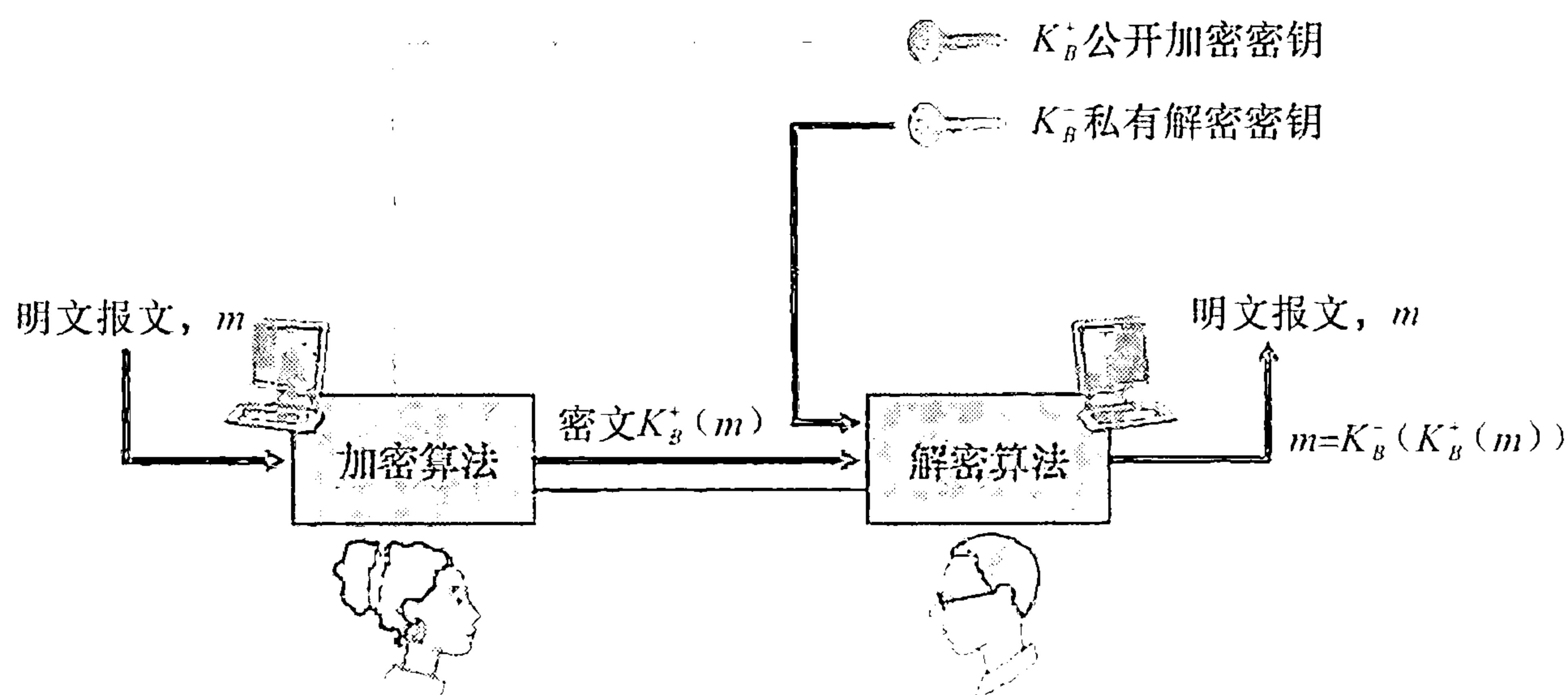


图 8-6 公开密钥密码

1. RSA

尽管可能有许多算法处理这些关注的问题，但 RSA 算法（RSA algorithm，取算法创立人 Ron Rivest、Adi Shamir 和 Leonard Adleman 的首字母命名）几乎已经成了公开密钥密码的代名词。我们首先来理解 RSA 是如何工作的，然后再考察 RSA 的工作原理。

RSA 广泛地使用了模 n 算术的算术运算。故我们简要地回顾一下模算术。前面讲过 $x \bmod n$ 只是表示被 n 除时 x 的余数；因此如 $19 \bmod 5 = 4$ 。在模算术中，人们执行通常的加法、乘法和指数运算。然而，每个运算的结果由整数余数代替，该余数是被 n 除后留下的数。对于模算术的加法和乘法可由下列便于施用的事实所简化：

$$[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$$

$$[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$$

$$[(a \bmod n) \cdot (b \bmod n)] \bmod n = (a \cdot b) \bmod n$$

从第三个事实能够有 $(a \bmod n)^d \bmod n = a^d \bmod n$ ，我们很快将会发现这个恒等式是非常有用的。

现在假设 Alice 要向 Bob 发送一个 RSA 加密的报文，如图 8-6 所示。在我们的讨论中，我们心中永远要记住一个报文只不过是一种比特模式，并且所有每种比特模式能唯一地被一个整数（连同该比特模式的长度）表示。例如，假设一个报文是比特模式 1001；这个报文能由十进制整数 9 来表示。所以，当用 RSA 加密一个报文时，等价于加密表示该报文的这个唯一的整数。

RSA 有两个互相关联的部分：

- 公钥和私钥的选择。
- 加密和解密算法。

为了生成 RSA 的公钥和私钥，Bob 执行如下步骤：

1) 选择两个大素数 p 和 q 。那么 p 和 q 应该多大呢？该值越大，破解 RSA 越困难，而执行加密和解密所用的时间也越长。RSA 实验室推荐，公司使用时， p 和 q 的乘积为 1024 比特的数量级。对于选择大素数的方法的讨论，参见 [Caldwell 2012]。

2) 计算 $n = pq$ 和 $z = (p - 1)(q - 1)$ 。

3) 选择小于 n 的一个数 e ，且使 e 和 z 没有（非 1 的）公因数。（这时称 e 与 z 互素。）使用字母 e 表示是因为这个值将被用于加密。

4) 求一个数 d ，使得 $ed - 1$ 可以被 z 整除（就是说，没有余数）。使用字母 d 表示是因为这个值将用于解密。换句话说，给定 e ，我们选择 d ，使得

$$ed \bmod z = 1$$

5) Bob 使外界可用的公钥 K_B^+ 是一对数 (n, e) ；其私钥 K_B^- 是一对数 (n, d) 。

Alice 执行的加密和 Bob 进行的解密过程如下：

- 假设 Alice 要给 Bob 发送一个由整数 m 表示的比特组合，且 $m < n$ 。为了进行编码，Alice 执行指数运算 m^e ，然后计算 m^e 被 n 除的整数余数。换言之，Alice 的明文报文 m 的加密的值 c 就是：

$$c = m^e \bmod n$$

对应于这个密文 c 的比特模式发送给 Bob。

- 为了对收到的密文报文 c 解密，Bob 计算：

$$m = c^d \bmod n$$

这要求使用他的私钥 (n, d) 。

举一个简单的 RSA 例子，假设 Bob 选择 $p = 5$ 和 $q = 7$ 。（坦率地讲，这样小的值无法保证安全。）则 $n = 35$ 和 $z = 24$ 。因为 5 和 24 没有公因数，所以 Bob 选择 $e = 5$ ；最后，因为 $5 \times 29 - 1$ （即 $ed - 1$ ）可以被 24 整除，所以 Bob 选择 $d = 29$ 。Bob 公开了 $n = 35$ 和 $e = 5$ 这两个值，并秘密保存了 $d = 29$ 。观察公开的这两个值，假定 Alice 要发送字母“l”、“o”、“v”和“e”给 Bob。用 1 ~ 26 之间的每个数表示一个字母，其中 1 表示“a”，…，26 表示“z”，Alice 和 Bob 分别执行如表 8-2 和表 8-3 所示的加密和解密运算。注意到在这个例子中，我们认为每四个字母作为一个不同报文。一个更为真实的例子是把这四个字母转换成它们的 8 比特 ASCII 表示形式，然后加密对应得到的 32 比特的比特模式的整数。（这样一个真实的例子产生了一些长得难以在教科书中打印出来的数！）

假定在表 8-2 和表 8-3 中的简单示例已经产生了某些极大的数，并且假定我们前面看到 p 和 q 每个都是数百比特长的数，这些都是实际使用 RSA 时必须牢记的。如何选择大素数？如何选择 e 和 d ？以及如何对大数进行指数运算？对这些重要问题的详细讨论超出了本书的范围，详情请参见 [Kaufman 1995] 以及其中的参考文献。

表 8-2 Alice 的 RSA 加密， $e = 5, n = 35$

明文字母	m : 数字表示	me	密文 $c = me \bmod n$
i	12	248832	17
o	15	759375	15
v	22	5153632	22
e	5	3125	10

表 8-3 Bob 的 RSA 解密, $d=29, n=35$

密文 c	c^d	$m = c^d \bmod n$	明文字母
17	481968572106750915091411825223071697	12	i
15	12783403948858939111232757568359375	15	o
22	851643319086537701956194499721106030592	22	v
10	10000000000000000000000000000000	5	e

2. 会话密钥

这里我们注意到，RSA 所要求的指数运算是相当耗费时间的过程。形成对比的是，DES 用软件实现要比 RSA 快 100 倍，用硬件实现则要快 1000 ~ 10 000 倍 [RSA Fast 2012]。所以，在实际应用中，RSA 通常与对称密钥密码结合起来使用。例如，如果 Alice 要向 Bob 发送大量的加密数据，她可以用下述方式来做。首先，Alice 选择一个用于加密数据本身的密钥；这个密钥有时称为会话密钥（session key），该会话密钥表示为 K_s 。Alice 必须把这个会话密钥告知 Bob，因为这是他们在对称密钥密码（如 DES 或 AES）中所使用的共享对称密钥。Alice 可以使用 Bob 的 RSA 公钥来加密该会话密钥，即计算 $c = (K_s)^e \bmod n$ 。Bob 收到了该 RSA 加密的会话密钥 c 后，解密得到会话密钥 K_s 。Bob 此时已经知道 Alice 将要用于她的加密数据传输的会话密钥了。

3. RSA 的工作原理

RSA 加密/解密看起来相当神奇。为什么那样应用加密算法，然后再运行解密算法，就能恢复出初始报文呢？要理解 RSA 的工作原理，我们仍将表示 $n = pq$ ，其中 p 和 q 是 RSA 算法中的大素数。

前面讲过在 RSA 加密过程中，一个报文 m （唯一地表示为整数）使用模 n 算术做 e 次幂运算，即

$$c = m^e \bmod n$$

解密则先对该值执行 d 次幂，再做模 n 运算。因此先加密再解密的结果是 $(m^e \bmod n)^d \bmod n$ 。下面我们来看，关于这个量我们能够得到什么。因为前面提到，模算术的一个重要性质是对于任意值 a 、 n 和 d 都有 $(a \bmod n)^d \bmod n = a^d \bmod n$ 。因此，在这个性质中使用 $a = m^e$ ，则有

$$(m^e \bmod n)^d \bmod n = m^{ed} \bmod n$$

因此剩下证明 $m^{ed} \bmod n = m$ 。尽管我们正试图揭开 RSA 工作原理的神秘面纱，但为了做到这一点，我们还需要用到数论中一个相当神奇的结果。具体而言，就是要用到数论中这样的结论：如果 p 和 q 是素数，且有 $n = pq$ 和 $z = (p - 1)(q - 1)$ ，则 $x^y \bmod n$ 与 $x^{(y \bmod z)} \bmod n$ 是等同的 [Kaufman 1995]。应用这个结论，对于 $x = m$ 和 $y = ed$ ，可得

$$m^{ed} \bmod n = m^{(ed \bmod z)} \bmod n$$

但是要记住，我们是这样选择 e 和 d 的，即 $ed \bmod z = 1$ 。这告诉我们

$$m^{ed} \bmod n = m^1 \bmod n = m$$

这正是我们希望得到的结果！先对 m 做 e 次幂（加密）再做 d 次幂（解密），然后做模 n 的算术运算（原文中没有这句，译者认为有必要补上。——译者注），就可得到初始的 m 。甚至更为奇妙之处是这样一个事实，如果我们先对 m 做 d 次幂（加密）再做 e 次幂，即颠倒加密和解密的次序，先执行解密操作再执行加密操作，我们也能得到初始值 m 。这个奇妙的结果完全遵循下列模算术：

$$(m^d \bmod n)^e \bmod n = m^{de} \bmod n = m^{ed} \bmod n = (m^e \bmod n)^d \bmod n$$

RSA 的安全性依赖于这样的事实：目前没有已知的算法可以快速进行一个数的因数分解，这种情况下公开值 n 无法快速分解成素数 p 和 q 。如果已知 p 和 q ，则给定公开值 e ，就很容易计算出秘密密钥 d 。在另一方面，也不确定是否存在因数分解一个数的快速算法，从这种意义上来说，RSA 的安全性也不是确保的。

另一种流行的公钥加密算法是 Differ-Hellman，我们将在课后习题中简要探讨它。Differ-Hellman 并不像 RSA 那样多功能，即它不能用于加密任意长度的报文；然而，它能够用来创建一个对称的会话密钥，该密钥再被用于加密长报文。

8.3 报文完整性和数字签名

在前面一节中我们看到了能够使用密码术为两个通信实体提供机密性。在本节中我们转向提供报文完整性（message integrity）这个同等重要的主题。报文完整性也称为报文鉴别。连同报文完整性，在本节中我们将讨论两个相关的主题：数字签名和端点鉴别。

我们再次使用 Alice 和 Bob 来定义报文完整性问题。假定 Bob 接收到一个报文（这可能已经加密或可能是明文），并且他认为这个报文是由 Alice 发送的。为了鉴别这个报文，Bob 需要证实：

- 1) 该报文的确源自 Alice。
- 2) 该报文在到 Bob 的途中没有被篡改。

我们将在 8.4 ~ 8.7 节中看到，报文完整性这个问题在所有安全网络协议中都是至关重要的。

举一个特定的例子，考虑一个使用链路状态路由选择算法（例如 OSPF）的计算机网络，在该网络中决定每对路由器之间的路由（参见第 4 章）。在一个链路状态算法中，每台路由器需要向该网络中的所有其他路由器广播一个链路状态报文。路由器的链路状态报文包括直接相连邻居的列表以及到这些邻居的直接费用。一旦某台路由器从其他所有路由器收到了链路状态报文，它能够生成该网络的全图，运行它的最小费用路由选择算法并配置它的转发表。对路由选择算法的一个相对容易的攻击是，Trudy 分发具有不正确状态信息的虚假链路状态报文。因此产生了报文完整性的需求：当路由器 B 收到来自路由器 A 的链路状态报文，路由器 B 应当证实路由器 A 实际生成了该报文，并且进一步证实在传输过程中该报文没有被篡改。

在本节中，我们描述一种由许多安全网络协议所使用的流行报文完整性技术。但在做此事之前，我们需要涉及密码学中的另一个重要主题，即密码散列函数。

8.3.1 密码散列函数

如图 8-7 所示，散列函数以 m 为输入，并计算得到一个称为散列的固定长度的字符串 $H(m)$ 。因特网检验和（第 3 章）和 CRC（第 4 章）都满足这个定义。密码散列函数（cryptographic hash function）要求具有下列附加的性质：

- 找到任意两个不同的报文 x 和 y 使得 $H(x) = H(y)$ ，在计算上是不可能的。

不严格地说，这种性质就意味着入侵者在计算上不可能用其他报文替换由散列函数保护的报文。这就是说，如果 $(m, H(m))$ 是报文和由发送方生成的报文散列的话，则入侵者不可能伪造另一个报文 y 的内容，使得该报文具有与原报文相同的散列值。

我们来证实一个简单的检验和（如因特网检验和）只能算作劣质的密码散列函数。不像在因特网检验和中执行反码运算那样，我们把每个字符看作一个字节，并把这些字节加到一起，一次用 4 字节的块来进行计算。假定 Bob 欠 Alice 100.99 美元并且向 Alice 发送一张借据，这个借据包含以下文本字符串“IOU100.99BOB”。这些字符的 ASCII 表示（以十六进制形式表示）为 49, 4F, 55, 31, 30, 30, 2E, 39, 39, 42, 4F, 42。

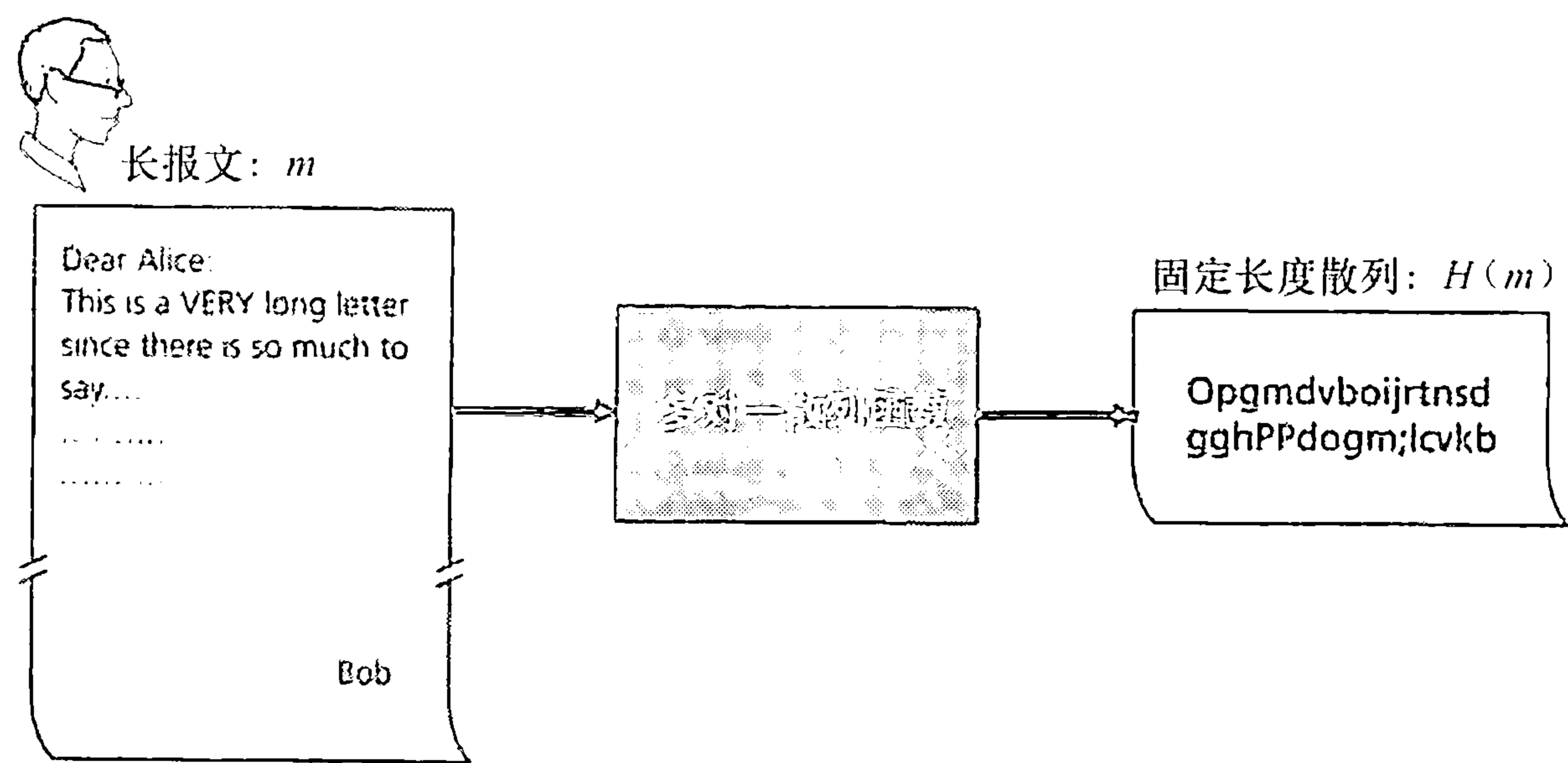


图 8-7 散列函数

图 8-8 上半部分显示了这个报文的 4 字节检验和是 B2 C1 D2 AC。图 8-8 下半部分显示了一条稍微不同的报文（但是 Bob 要付的钱却多了许多）。报文“IOU100.99BOB”和“IOU900.19BOB”有相同的检验和。因此，这种简单的检验和算法违反了上述的要求。给定初始数据，很容易找到有相同检验和的另一组数据。很明显，为了安全起见，我们需要比检验和更为强有力的散列函数。

Ron Rivest [RFC 1321] 的 MD5 散列算法如今正在广泛使用。这个算法通过 4 步过程计算得到 128 比特的散列。这 4 步过程由下列步骤组成：①填充——先填 1，然后填足够多的 0，直到报文长度满足一定的条件；②添加——在填充前添加一个用 64 比特表示的报文长度；③初始化累加器；④循环——在最后的循环步骤中，对报文的 16 字块进行 4 轮处理。MD5 的描述（包括一个 C 源代码实现）可参见 [RFC 1321]。

报文	ASCII表示				
I O U 1	49	4F	55	31	
0 0 . 9	30	30	2E	39	
9 B O B	39	42	4F	42	
	B2	C1	D2	AC	检验和

报文	ASCII表示				
I O U 9	49	4F	55	39	
0 0 . 1	30	30	2E	31	
9 B O B	39	42	4F	42	
	B2	C1	D2	AC	检验和

图 8-8 初始报文和欺诈报文具有相同的检验和

目前正使用的第二个主要报文摘要算法是安全散列算法 SHA-1（Security Hash Algorithm）[FIPS 1995]。这个算法的原理类似于 MD4 [RFC 1320] 设计中所使用的原理，而 MD4 是 MD5 的前身。SHA-1 是美国联邦政府的标准，任何联邦政府的应用程序如果需要使用密码散列算法的话，都要求使用 SHA-1。SHA-1 生成一个 160 比特的报文摘要。较长的输出长度可使 SHA-1 更安全。

8.3.2 报文鉴别码

我们现在再回到报文完整性的问题。既然我们理解了散列函数，就先来看一下将如何

执行报文完整性：

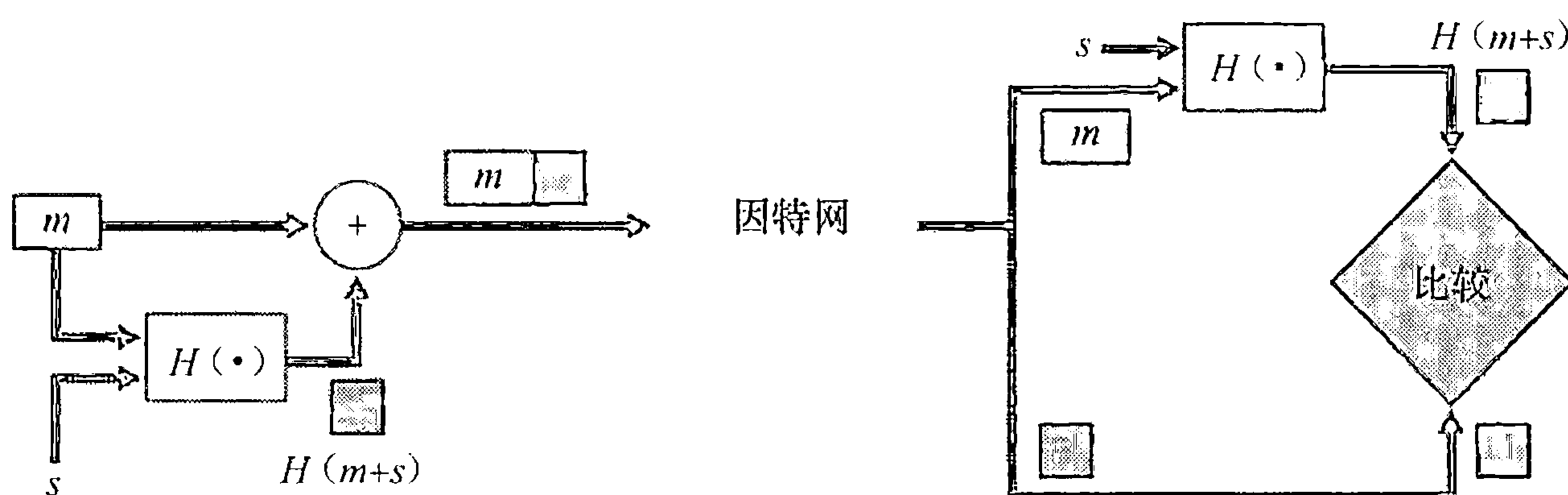
- 1) Alice 生成报文 m 并计算散列 $H(m)$ (例如使用 SHA-1)。
- 2) Alice 则将 $H(m)$ 附加到报文 m 上, 生成一个扩展报文 $(m, H(m))$, 并将该扩展报文发给 Bob。
- 3) Bob 接收到一个扩展报文 (m, h) 并计算 $H(m)$ 。如果 $H(m) = h$, Bob 得到结论：一切正常。

这种方法存在明显缺陷。Trudy 能够生成虚假报文 m' , 在其中声称她就是 Alice, 计算 $H(m')$ 并发送给 Bob $(m', H(m'))$ 。当 Bob 接收到该报文, 一切将在步骤 3 中核对通过, 并且 Bob 无法猜出这种不轨的行为。

为了执行报文完整性, 除了使用密码散列函数, Alice 和 Bob 将需要共享秘密 s 。这个共享的秘密只不过是一个比特串, 它被称为鉴别密钥 (authentication key)。使用这个共享秘密, 报文完整性能够执行如下：

- 1) Alice 生成报文 m , 用 s 级联 m 以生成 $m + s$, 并计算散列 $H(m + s)$ (例如使用 SHA-1)。 $H(m + s)$ 被称为报文鉴别码 (Message Authentication Code, MAC)。
- 2) Alice 则将 MAC 附加到报文 m 上, 生成扩展报文 $(m, H(m + s))$, 并将该扩展报文发送给 Bob。
- 3) Bob 接收到一个扩展报文 (m, h) , 由于知道 s , 计算出报文鉴别码 $H(m + s)$ 。如果 $H(m + s) = h$, Bob 得到结论：一切正常。

图 8-9 中显示了上述过程的总结。读者们应当注意到这里的 MAC (表示“报文鉴别码”) 与用于数据链路层中的 MAC (表示“媒体访问控制”) 是不一样的！



图例：
 m = 报文
 s = 共享秘密

图 8-9 报文鉴别码

MAC 的一个优良特点是它不要求一种加密算法。的确, 在许多应用中, 包括前面讨论的链路状态路由选择算法, 通信实体仅关心报文完整性, 并不关心报文机密性。使用一个 MAC, 实体能够鉴别它们相互发送的报文, 而不必在完整性过程中综合进复杂的加密过程。

如你所猜测, 多年来已经提出了若干种对 MAC 的不同标准。目前最为流行的标准是 HMAC, 它能够与 MD5 或 SHA-1 一道使用。HMAC 实际上通过散列函数运行数据和鉴别密钥两次 [Kaufman 1995; RFC 2104]。

这里还遗留下一个重要问题。我们怎样向通信实体分发这个共享的鉴别密钥呢? 例如, 在链路状态路由选择算法中, 我们将在某种程度上需要向自治系统中的每台路由器分

发该秘密鉴别密钥。（注意到所有路由器能够都使用相同的鉴别密钥。）一名网络管理员能够通过物理上访问每台路由器来实际完成这项工作。或者，如果这名网络管理员不够勤快，并且如果每台路由器都有它自己的公钥，那么该网络管理员能够用路由器的公钥加密该鉴别密钥并分发给任何一台路由器，从而通过网络向路由器发送加密的密钥。

8.3.3 数字签名

回想在过去的一周中，你在纸上已经签过多少次你的名字？你可能经常会在支票、信用卡收据、法律文件和信件上签名。你的签名证明你（而不是其他人）承认和/或同意这些文件的内容。在数字领域，人们通常需要指出一个文件的所有者或创作者，或者表明某人认可一个文件内容。数字签名（digital signature）就是在数字领域实现这些目标的一种密码技术。

正如人手签字一样，数字签名也应当以可鉴别的、不可伪造的方式进行。这就是说，必须能够证明由某个人在一个文档上的签名确实是由该人签署的（该签名必须是可证实的），且只有那个人能够签署那个文件（该签名无法伪造）。

我们现在来考虑怎样设计一个数字签名方案。当 Bob 签署一个报文时，可以观察到 Bob 必须将某些对他是独特的东西放置在该报文上。Bob 能够考虑附加一个 MAC 用作签名，其中 MAC 是由他的密钥（对他是独特的）作用到该报文上而生成的，然后得到该散列值。而 Alice 为了验证该签名，她必须也具有该密钥的副本，在这种情况下该密钥对 Bob 将不是唯一的。因此，此时 MAC 是无法胜任这项工作的。

前面讲过使用公钥密码，Bob 具有公钥和私钥，这两种密钥对 Bob 均为独特的。因此，公钥密码是一种提供数字签名的优秀候选者。我们现在来研究一下这是怎样完成的。

假设 Bob 要以数字方式签署一个文档 m 。我们能够想象这个文档是 Bob 打算签署并发送的一个文件或一个报文。如图 8-10 所示，为了签署这个文档，Bob 直接使用他的私钥 K_B^- 计算 $K_B^-(m)$ 。乍一看，会感觉很奇怪，Bob 怎么会用他的私钥（在 8.2 节中，我们用私钥解密用其公钥加密的报文）签署文档！但是回想加密和解密都只不过是数学运算而已（RSA 中所做的 e 或 d 指数幂运算；参见 8.2 节），并且 Bob 的目的不是弄乱或掩盖文档的内容，而只是要以可鉴别的、不可伪造的方式签署这个文档。Bob 对文档 m 签名之后所得的文档就是 $K_B^-(m)$ 。

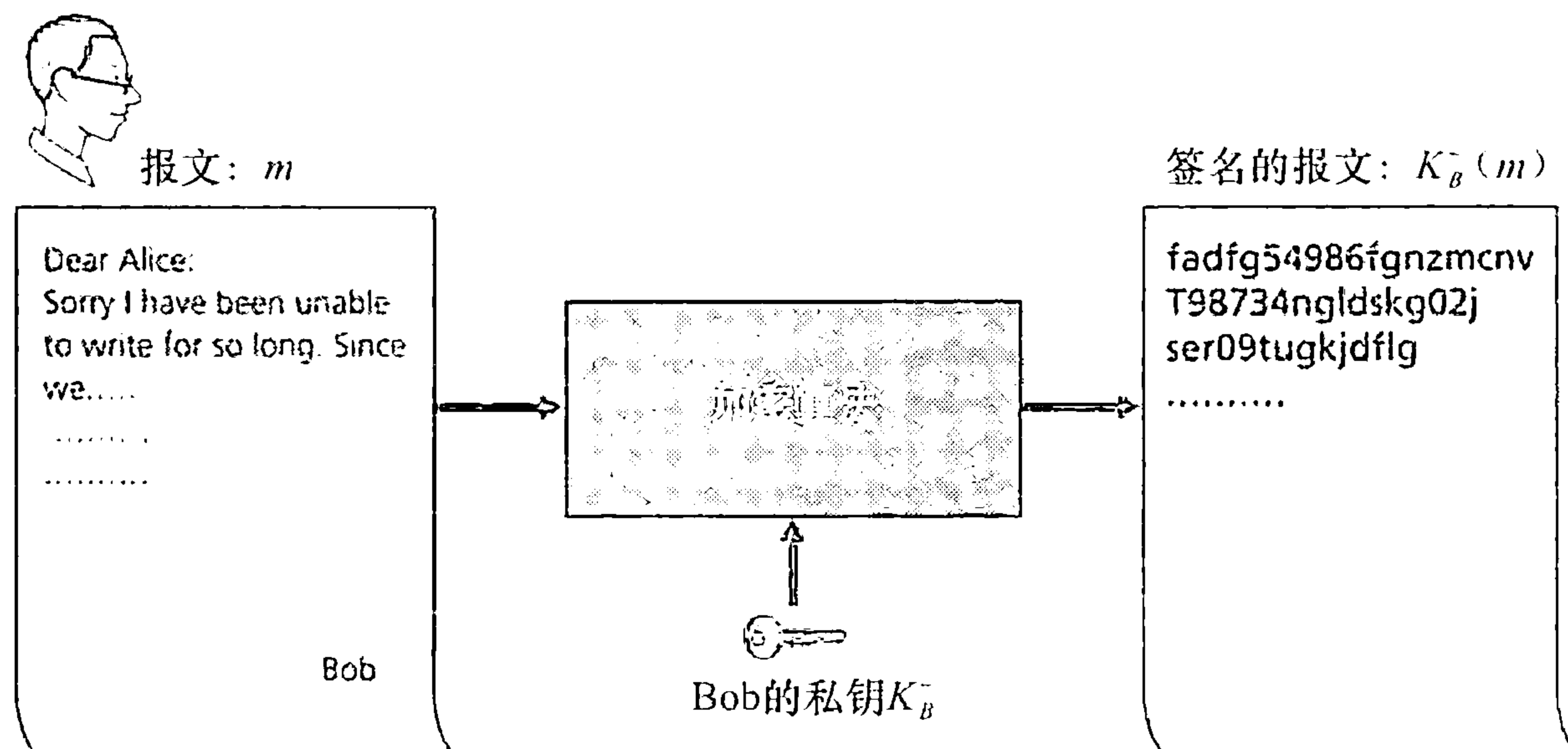


图 8-10 对一个文档生成一个数据签名

数字签名 $K_B^-(m)$ 是否满足了可鉴别的、不可伪造的需求？假设 Alice 有 m 和 $K_B^-(m)$ 。她要在法庭上证明（进行诉讼）Bob 确实签署过这个文档，他就是唯一能够签署该文档的人。Alice 持有 Bob 的公钥 K_B^+ ，并把它用于 Bob 的数字签名 $K_B^-(m)$ 上，从而得到了文档 m 。也就是说，Alice 计算 $K_B^+(K_B^-(m))$ ，瞧！在 Alice 经历了令人注目的慌乱后得到了 m ，它与初始文档完全一致。然后，Alice 就可以论证仅有 Bob 能够签署这个文档，基于如下理由：

- 无论是谁签署这个报文，都必定在计算签名 $K_B^-(m)$ 过程中使用了 K_B^- 这个私钥，使 $K_B^+(K_B^-(m)) = m$ 。
- 知道 K_B^- 这个私钥的唯一人只有 Bob。从 8.2 节我们对 RSA 的讨论中可知，知道公钥 K_B^+ 无助于得知私钥 K_B^- 的信息。因此，知道私钥 K_B^- 的人才是生成密钥对 (K_B^+, K_B^-) 的人，而这个人首当其冲就是 Bob。（注意到此处假设 Bob 没有把 K_B^- 泄露给任何人，也没有人从 Bob 处窃取到 K_B^- 。）

注意到下列问题是重要的，如果源文档 m 被修改过，比如改成了另一个文档 m' ，则 Bob 对 m 生成的签名对 m' 无效，因为 $K_B^+(K_B^-(m))$ 不等于 m' 。因此我们看到数字签名也提供完整性，使得接收方验证该报文未被篡改，同时也验证了该报文的源。

对用加密进行数据签名的担心是，加密和解密的计算代价昂贵。给定加解密的开销，通过完全加密/解密对数据签名是杀鸡用牛刀。更有效的方法是将散列函数引入数字签名 8.3.2 节中讲过，一种散列算法取一个任意长的报文 m ，计算生成该报文的一个固定长度的数据“指纹”，表示为 $H(m)$ 。使用散列函数，Bob 对报文的散列签名而不是对报文本身签名，即 Bob 计算 $K_B^-(H(m))$ 。因为 $H(m)$ 通常比报文 m 小得多，所以生成数字签名所需要的计算耗费大为降低。

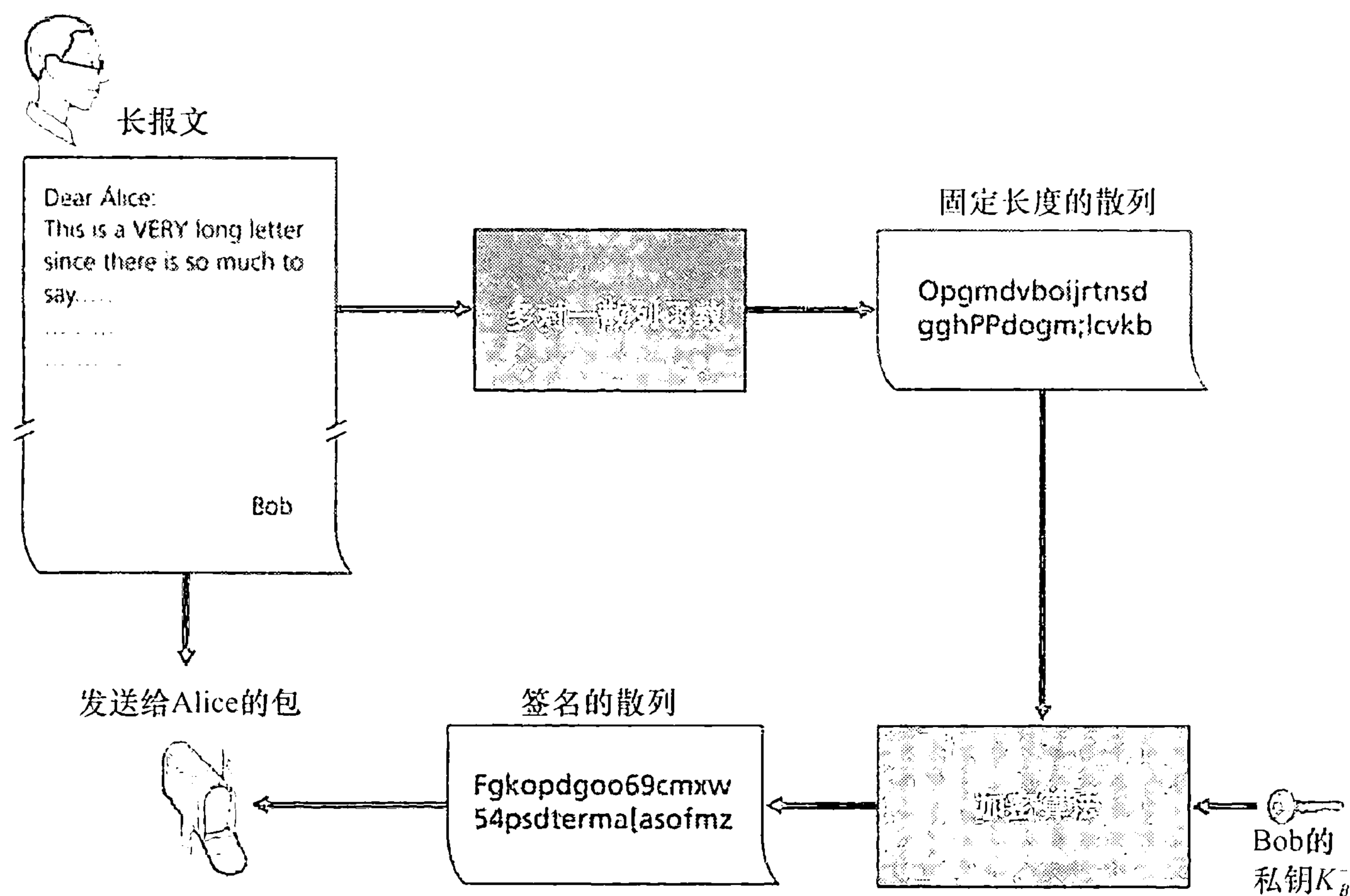


图 8-11 发送数字签名的报文

在 Bob 向 Alice 发送一个报文的情况下，图 8-11 提供了生成一个数字签名的操作过程的概览。Bob 让他的初始长报文通过一个散列函数。然后他用自己的私钥对得到的散列进行数字签名。明文形式的初始报文连同已经数字签名的报文摘要（从此以后可称为数字签

名) 一道被发送给 Alice。图 8-12 提供了鉴别报文完整性的操作过程的概览。Alice 先把发送方的公钥应用于报文获得一个散列结果。然后她再把该散列函数应用于明文报文以得到第二个散列结果。如果这两个散列匹配, 则 Alice 可以确信报文的完整性及其发送方。

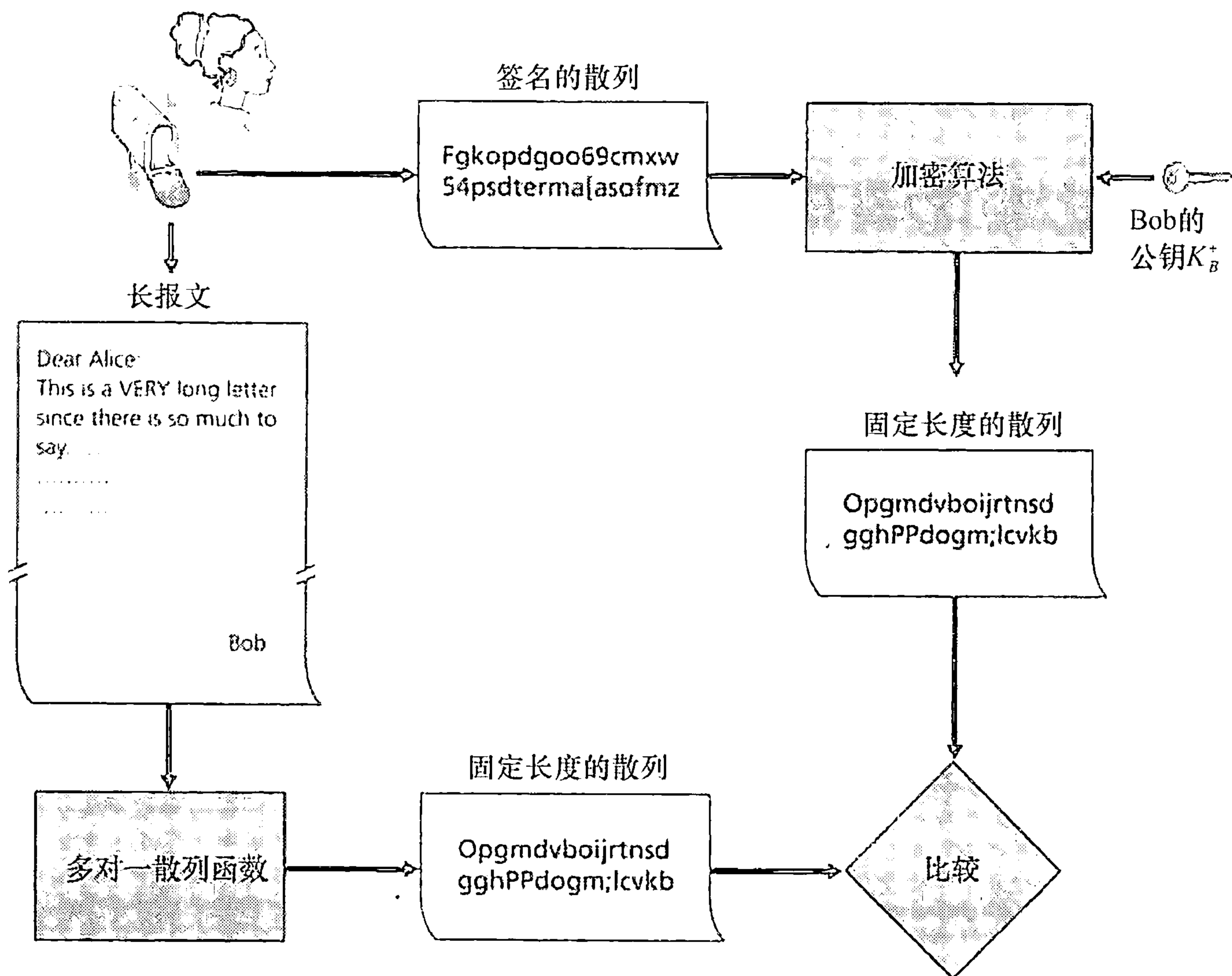


图 8-12 验证签名报文

在继续学习之前, 我们简要地将数字签名与 MAC 进行比较, 尽管它们有类似之处, 但也有重要的微妙差异。数字签名和 MAC 都以一个报文 (或一个文档) 开始。为了从该报文中生成一个 MAC, 我们为报文附加一个鉴别密钥, 然后取得该结果的散列。注意到在生成 MAC 过程中既不涉及公开密钥加密, 也不涉及对称密钥加密。为了生成一个数字签名, 我们首先取得该报文的散列, 然后用我们的私钥加密该报文 (使用公钥密码)。因此, 数字签名是一种“技术含量更高的”技术, 因为它需要一个如后面描述的、具有认证中心支撑的公钥基础设施 (PKI)。我们将在 8.4 节中看到, PGP 是一种流行的安全电子邮件系统, 为了报文完整性而使用数字签名。我们已经看到了 OSPF 为了报文完整性而使用 MAC。我们将在 8.5 节和 8.6 节中看到 MAC 也能用于流行的运输层和网络层安全协议。

公钥认证

数字签名的一个重要应用是公钥认证 (public key certification), 即证实一个公钥属于某个特定的实体。公钥认证被用于许多流行的安全网络协议中, 包括 IPsec 和 SSL。

为了深入理解这个问题, 我们考虑一个因特网商务版本的经典的“比萨恶作剧”。假定 Alice 正在从事比萨派送业务, 从因特网上接受订单。Bob 是一个爱吃比萨的人, 他向 Alice 发送了一份包含其家庭地址和他希望的比萨类型的明文报文。Bob 在这个报文中也包含一个数字签名 (即对原始明文报文的签名的散列), 以向 Alice 证实他是该报文的真正来源。为了验证这个数字签名, Alice 获得了 Bob 的公钥 (也许从公钥服务器或通过电子邮

件报文) 并核对该数字签名。通过这种方式, Alice 确信是 Bob 而不是某些青少年恶作剧者下的比萨订单。

在聪明的 Trudy 出现之前, 这一切看起来进行得相当好。如在图 8-13 中所示, Trudy 沉溺于一场恶作剧中。Trudy 向 Alice 发送一个报文, 在这个报文中她说她是 Bob, 给出了 Bob 家的地址并订购了一个比萨。在这个报文中, 她也包括了她 (Trudy) 的公钥, 虽然 Alice 自然地假定它就是 Bob 的公钥。Trudy 也附加了一个签名, 但是这是用她自己 (Trudy) 的私钥生成的。在收到该报文后, Alice 就会用 Trudy 的公钥 (Alice 认为它是 Bob 的公钥) 来解密该数字签名, 并得到结论: 这个明文报文确实是由 Bob 生成的。而当外送人员带着具有意大利辣香肠和凤尾鱼的比萨到达 Bob 家时, 他会感到非常惊讶!

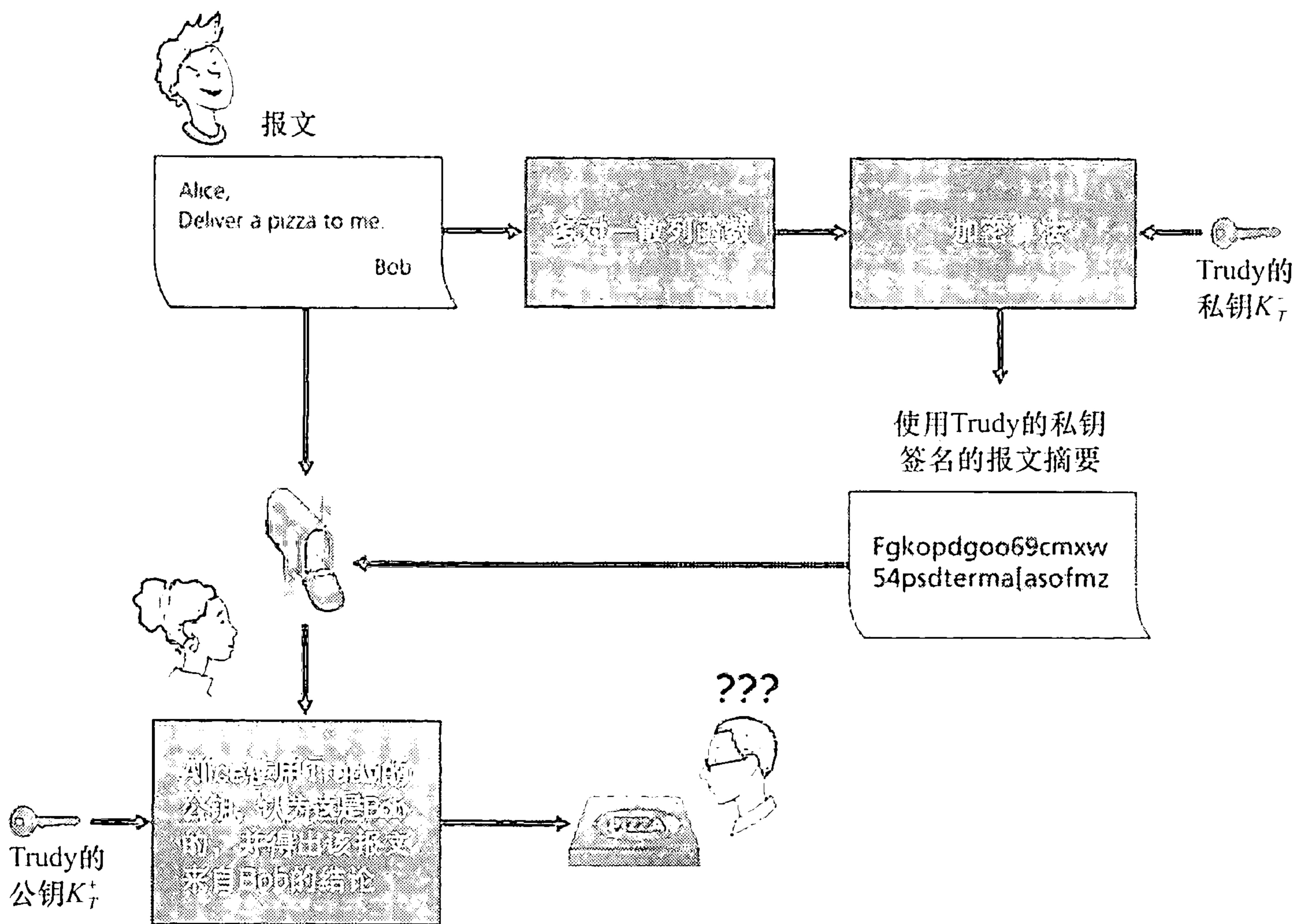


图 8-13 Trudy 用公钥密码冒充 Bob

从这个例子我们看到, 要使公钥密码有用, 需要能够证实你具有的公钥实际上就是与你要进行通信的实体 (人员、路由器、浏览器等) 的公钥。例如, 当 Alice 与 Bob 使用公钥密码通信时, 她需要证实她假定是 Bob 的那个公钥确实就是 Bob 的公钥。

将公钥与特定实体绑定通常是由认证中心 (Certification Authority, CA) 完成的, CA 的职责就是使识别和发行证书合法化。CA 具有下列作用:

1) CA 证实一个实体 (一个人、一台路由器等) 的真实身份。如何进行认证并没有强制的过程。当与一个 CA 打交道时, 一方必须信任这个 CA 能够执行适当的严格身份验证。例如, 如果 Trudy 可以走进名为 Fly-by-Night 的证书权威机构并只是宣称 “我是 Alice”, 就可以得到该机构颁发的与 Alice 的身份相关联证书的话, 则人们不会对 Fly-by-Night 证书权威机构所签发的公钥证书有太多的信任。在另一方面, 人们可能愿意 (或不愿意!) 信任某个 CA, 如果这个 CA 是联邦或州计划的一部分的话。你对与公钥相联系的身份的信任程度, 仅能达到你对 CA 及其身份验证技术的信任程度。我们编织了多么混乱

的信任关系网啊！

2) 一旦 CA 验证了某个实体的身份，这个 CA 会生成一个把其身份和实体的公钥绑定起来的**证书**（certificate）。这个证书包含这个公钥和公钥所有者全局唯一的身份标识信息（例如，一个人的名字或一个 IP 地址）。由 CA 对这个证书进行数字签名。这些步骤显示在图 8-14 中。

我们现在来看怎样使用认证来对抗“比萨订购”中的恶作剧者（如 Trudy）和其他意外情况。当 Bob 下订单的同时，他也发送了其 CA 签署的证书。Alice 使用 CA 的公钥来核对 Bob 证书的合法性并提取 Bob 的公钥。

国际电信联盟（International Telecommunication Union, ITU）和 IETF 都研发了用于 CA 的系列标准。ITU X. 509 [ITU 2005a] 规定了证书的鉴别服务以及特定语法。[RFC 1422] 描述了安全因特网电子邮件所用的基于 CA 的密钥管理。它和 X. 509 兼容，但比 X. 509 增加了密钥管理体系结构的创建过程和约定内容。表 8-4 显示了一份证书中的某些重要字段。

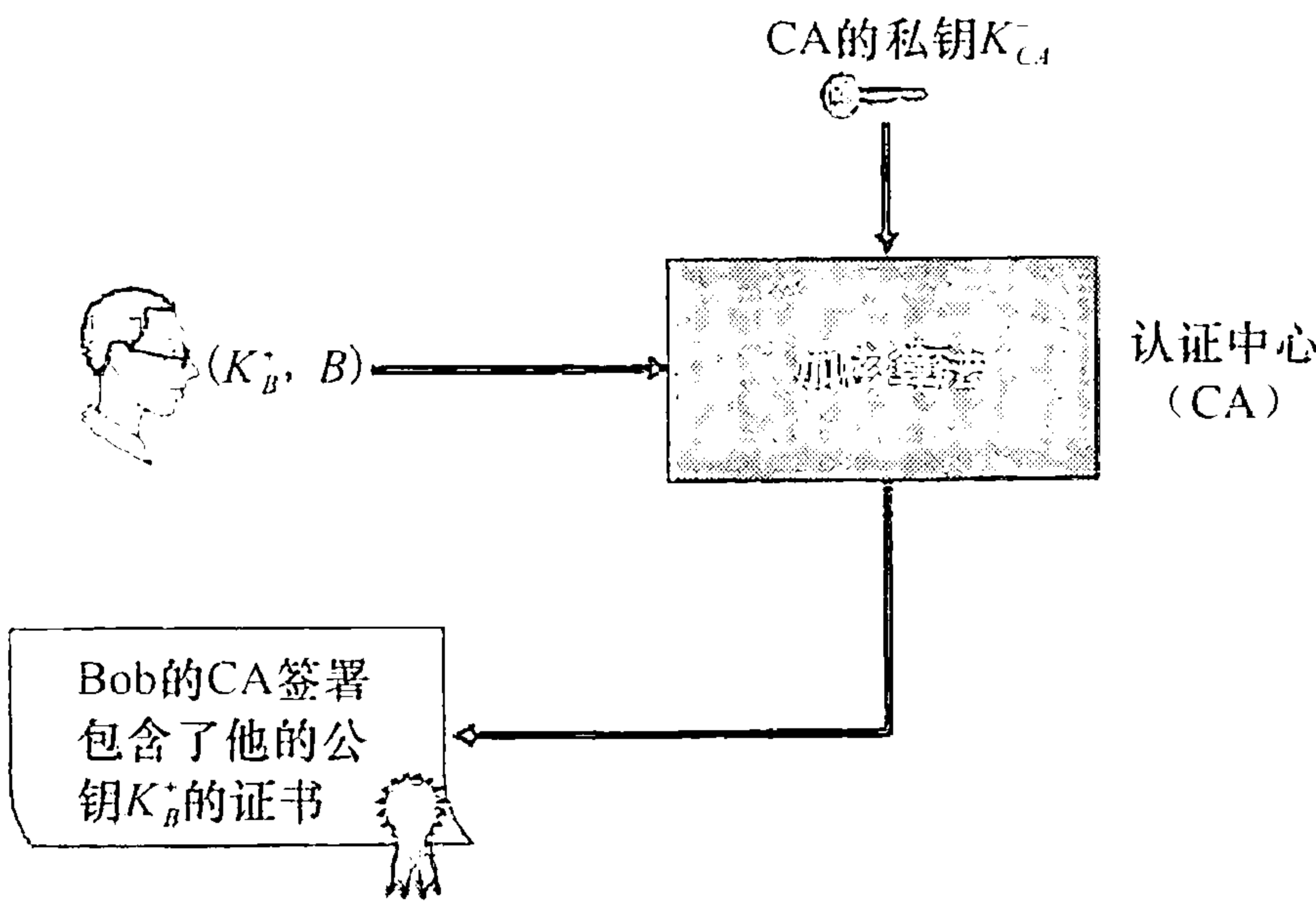


图 8-14 Bob 获得一份来自 CA 的证书

表 8-4 在 X. 509 和 RFC 1422 公钥证书中的部分字段

字段名	描述
版本（Version）	X. 509 规范的版本号
序列号（Serial Number）	CA 发布的证书的独特标识符
签名（Signature）	规定了由 CA 所用的对该证书签名的算法
颁发者名称（Issuer Name）	发行该证书的 CA 的标识符，用的是区别名（DN）格式 [RFC 4514]
有效期（Validity period）	证书合法性开始和结束的时间范围
主题名（Subject name）	其公钥与该证书相联系的实体标识符，用 DN 格式
主题公钥（Subject public key）	该主题的公钥以及该公钥使用的公钥算法（及其参数）的指示

8.4 端点鉴别

端点鉴别（end-point authentication）就是一个实体经过计算机网络向另一个实体证明其身份的过程，例如一个人向某个电子邮件服务器证明其身份。作为人类，我们通过多种方式互相鉴别：见面时我们互相识别对方的面容，打电话时我们分辨对方的声音，海关的检查官员通过护照上的照片对我们进行鉴别。

在本节中，我们讨论经网络通信的双方如何能够鉴别彼此。此处我们重点关注当通信实际发生时鉴别“活动的”实体。一个具体的例子是一个用户向某电子邮件服务器鉴别他或她自己。这与证明在过去的某点接收到的报文确实来自声称的发送方稍有不同，如 8.3 节所述。

当经网络进行鉴别时，通信各方不能依靠生物信息比如外表、声波纹等进行身份鉴别。的确，我们会在后面的实例研究中看到，诸如路由器、客户/服务器进程等网络元素